

LAPORAN TEKNIS KOMPREHENSIF CAPSTONE DATA SCIENCE

**Analisis Risiko Penyakit Kardiovaskular Menggunakan
Exploratory Data Analysis dan Dashboard Interaktif**

DAFTAR ISI

DAFTAR ISI.....	2
BAB I PENDAHULUAN.....	5
1.1 Latar Belakang.....	5
1.2 Problem Discovery.....	5
1.3 Tujuan Proyek.....	5
1.4 Ruang Lingkup Proyek	6
BAB II ALUR DAN METODOLOGI PROYEK	7
2.1 Alur Keseluruhan Proyek.....	7
2.2 Tools yang Digunakan	8
BAB III DATA WRANGLING.....	9
3.1 Gathering Data	9
3.2 Data Dictionary	9
3.3 Assessing Data	10
3.4 Cleaning Data.....	10
BAB IV PERTANYAAN BISNIS	12
BAB V EXPLORATORY DATA ANALYSIS	13
5.1 Distribusi Kategori Risiko	13
5.2 Distribusi Heart Disease Risk Score	13
5.3 Distribusi Variabel Numerik.....	15
5.4 Distribusi Variabel Kategorik	16
5.5 Korelasi Variabel Numerik	17
5.6 Korelasi Variabel Numerik dengan Heart Disease Risk Score.....	18
5.7 Hubungan Variabel Numerik dengan Risk Category	19
5.8 Hubungan Variabel Kategorik dengan Risk Category.....	20

BAB VI FEATURE ENGINEERING	22
6.1 Tujuan Feature Engineering.....	22
6.2 6.2 Fitur yang Dibuat	22
6.3 Hasil Feature Engineering.....	23
BAB VII DASHBOARD INTERAKTIF	24
7.1 Tujuan Dashboard.....	24
7.2 Tools Dashboard	24
7.3 7.3 Struktur Dashboard	24
7.4 Fitur Filter Dashboard.....	24
7.5 KPI Dashboard.....	25
7.6 Visualisasi pada Halaman Dashboard.....	25
7.7 Halaman Relationship Features	26
7.8 Halaman Prioritas Edukasi.....	26
7.9 Deployment Dashboard	26
7.10 Tampilan Dashboard.....	27
BAB VIII HASIL AKHIR DAN INSIGHT UTAMA.....	32
8.1 Hasil Akhir Proyek	32
8.2 Insight Utama.....	32
8.3 Jawaban Pertanyaan Bisnis	33
BAB IX A/B TESTING.....	37
9.1 Tujuan A/B Testing.....	37
9.2 Desain Pengujian	37
9.3 Metode Pengujian	37
9.4 Hasil dan Interpretasi	37
9.5 Kesimpulan	37

BAB I

PENDAHULUAN

1.1 Latar Belakang

Penyakit kardiovaskular merupakan salah satu masalah kesehatan yang perlu mendapatkan perhatian serius karena dapat dipengaruhi oleh berbagai faktor, baik faktor klinis maupun faktor gaya hidup. Beberapa faktor yang sering berkaitan dengan risiko penyakit kardiovaskular antara lain usia, indeks massa tubuh atau Body Mass Index (BMI), tekanan darah, kadar kolesterol, detak jantung saat istirahat, kebiasaan merokok, aktivitas fisik, kualitas tidur, konsumsi alkohol, tingkat stres, kualitas diet, serta riwayat keluarga dengan penyakit jantung.

Dalam konteks data science, data pasien dapat dimanfaatkan untuk memahami pola risiko kardiovaskular. Analisis terhadap data tersebut dapat membantu mengidentifikasi kelompok pasien dengan risiko rendah, sedang, dan tinggi. Selain itu, analisis data juga dapat digunakan untuk mengetahui faktor klinis dan gaya hidup apa saja yang paling menonjol pada pasien dengan kategori risiko tinggi.

Proyek ini dilakukan untuk menganalisis dataset risiko penyakit kardiovaskular menggunakan proses data science secara menyeluruh. Tahapan yang dilakukan meliputi data wrangling, exploratory data analysis, feature engineering, explanatory analysis, hingga pembuatan dashboard interaktif. Dashboard dibuat agar hasil analisis dapat disajikan secara lebih informatif dan mudah dipahami.

1.2 Problem Discovery

Permasalahan utama dalam proyek ini adalah bagaimana memahami karakteristik pasien berdasarkan tingkat risiko penyakit kardiovaskular. Dataset yang digunakan memiliki berbagai variabel klinis dan gaya hidup, tetapi data tersebut perlu dianalisis lebih lanjut agar dapat menghasilkan informasi yang berguna.

1.3 Tujuan Proyek

Tujuan dari proyek ini adalah sebagai berikut:

1. Melakukan proses data wrangling pada dataset risiko penyakit kardiovaskular.
2. Mengevaluasi kualitas data melalui pengecekan struktur data, missing value, duplikasi, dan outlier.
3. Membersihkan data agar siap digunakan untuk analisis lanjutan.
4. Membuat data dictionary untuk menjelaskan variabel dalam dataset.
5. Melakukan Exploratory Data Analysis (EDA) untuk memperoleh insight dari data.

6. Menganalisis hubungan antara variabel klinis, variabel gaya hidup, dan kategori risiko.
7. Membuat fitur tambahan melalui proses feature engineering.
8. Membuat dashboard interaktif menggunakan Streamlit.
9. Menyusun rekomendasi berdasarkan hasil analisis.

1.4 Ruang Lingkup Proyek

Ruang lingkup proyek ini meliputi:

1. Penggunaan dataset risiko penyakit kardiovaskular.
2. Analisis terhadap variabel klinis dan gaya hidup pasien.
3. Pembersihan data melalui pengecekan missing value, duplikasi, dan outlier.
4. Analisis eksploratif menggunakan visualisasi data.
5. Pembuatan fitur tambahan yang relevan dengan risiko kesehatan.
6. Pembuatan dashboard interaktif untuk menyajikan hasil analisis.
7. Penyusunan insight dan rekomendasi berdasarkan hasil analisis.

BAB II

ALUR DAN METODOLOGI PROYEK

2.1 Alur Keseluruhan Proyek

Proyek ini dilakukan melalui beberapa tahapan utama. Alur pengerjaan proyek dimulai dari problem discovery hingga pembuatan dashboard interaktif. Tahapan tersebut adalah sebagai berikut:

- 1. Problem Discovery**

Mengidentifikasi permasalahan utama yang ingin diselesaikan, yaitu memahami faktor-faktor yang berkaitan dengan risiko penyakit kardiovaskular.

- 2. Gathering Data**

Mengumpulkan dataset yang relevan untuk analisis risiko penyakit kardiovaskular.

- 3. Assessing Data**

Mengevaluasi kualitas data melalui pengecekan struktur data, tipe data, missing value, data duplikat, dan outlier.

- 4. Cleaning Data**

Membersihkan data agar lebih siap digunakan untuk analisis. Pada tahap ini dilakukan penanganan outlier menggunakan metode IQR Capping.

- 5. Data Dictionary**

Menyusun penjelasan setiap variabel dalam dataset agar pembaca memahami makna masing-masing kolom.

- 6. Exploratory Data Analysis**

Melakukan eksplorasi data menggunakan statistik deskriptif dan visualisasi data.

- 7. Feature Engineering**

Membuat fitur baru yang dapat memperkaya analisis, seperti pulse pressure, kategori BMI, kategori kolesterol, kelompok usia, lifestyle risk score, dan clinical risk score.

- 8. Explanatory Analysis**

Menjawab pertanyaan bisnis berdasarkan hasil visualisasi dan analisis.

- 9. A/B Testing**

A/B Testing dilakukan untuk membandingkan dua scenario yang berbeda.

- 10. Dashboard Development**

Membuat dashboard interaktif menggunakan Streamlit untuk menyajikan hasil analisis secara lebih komunikatif.

11. Kesimpulan dan Rekomendasi

Menyusun kesimpulan utama dan rekomendasi berdasarkan hasil analisis.

2.2 Tools yang Digunakan

Tools dan library yang digunakan dalam proyek ini antara lain:

1. **Python**

Digunakan sebagai bahasa pemrograman utama dalam proses analisis data.

2. **Pandas**

Digunakan untuk membaca, mengolah, membersihkan, dan memanipulasi data.

3. **NumPy**

Digunakan untuk operasi numerik.

4. **Matplotlib dan Seaborn**

Digunakan untuk membuat visualisasi data pada tahap EDA.

5. **Plotly**

Digunakan untuk membuat visualisasi interaktif pada dashboard.

6. **Streamlit**

Digunakan untuk membangun dashboard interaktif berbasis web.

7. **Jupyter Notebook atau Google Colab**

Digunakan sebagai environment untuk proses data wrangling, EDA, dan dokumentasi analisis.

BAB III

DATA WRANGLING

3.1 Gathering Data

Dataset yang digunakan dalam proyek ini adalah dataset risiko penyakit kardiovaskular. Dataset ini berisi informasi pasien yang mencakup faktor klinis, faktor gaya hidup, skor risiko penyakit jantung, serta kategori risiko.

Data yang digunakan memiliki format CSV dan terdiri dari 5.500 baris data dengan 17 kolom utama. Setiap baris merepresentasikan satu pasien. Variabel dalam dataset mencakup usia, BMI, tekanan darah sistolik, tekanan darah diastolik, kadar kolesterol, detak jantung saat istirahat, status merokok, jumlah langkah harian, tingkat stres, aktivitas fisik per minggu, durasi tidur, riwayat keluarga penyakit jantung, kualitas diet, konsumsi alkohol, skor risiko penyakit jantung, dan kategori risiko.

Dataset ini digunakan karena sesuai dengan tujuan proyek, yaitu menganalisis faktor-faktor yang berkaitan dengan risiko penyakit kardiovaskular.

3.2 Data Dictionary

Data dictionary disusun untuk menjelaskan setiap variabel yang terdapat dalam dataset. Data dictionary berisi nama variabel, tipe data, deskripsi, dan keterangan nilai atau satuan. Dengan adanya data dictionary, proses analisis menjadi lebih mudah dipahami karena setiap kolom memiliki definisi yang jelas.

Variabel utama dalam dataset antara lain:

1. Patient_ID: identitas unik pasien.
2. age: usia pasien dalam tahun.
3. bmi: indeks massa tubuh pasien.
4. systolic_bp: tekanan darah sistolik dalam mmHg.
5. diastolic_bp: tekanan darah diastolik dalam mmHg.
6. cholesterol_mg_dl: kadar kolesterol dalam mg/dL.
7. resting_heart_rate: detak jantung saat istirahat dalam bpm.
8. smoking_status: status merokok pasien.
9. daily_steps: jumlah langkah harian pasien.
10. stress_level: tingkat stres pasien.
11. physical_activity_hours_per_week: durasi aktivitas fisik per minggu.
12. sleep_hours: durasi tidur pasien per hari.
13. family_history_heart_disease: riwayat keluarga penyakit jantung.

14. `diet_quality_score`: skor kualitas diet.
15. `alcohol_units_per_week`: konsumsi alkohol per minggu.
16. `heart_disease_risk_score`: skor risiko penyakit jantung.
17. `risk_category`: kategori risiko penyakit jantung.

Selain variabel utama, beberapa variabel baru juga dibuat melalui proses feature engineering, seperti `pulse_pressure`, `blood_pressure_ratio`, `hypertension_stage`, `bmi_category`, `cholesterol_category`, `activity_level`, `sleep_category`, `alcohol_category`, `lifestyle_risk_score`, `clinical_risk_score`, dan `age_group`.

3.3 Assessing Data

Tahap assessing data dilakukan untuk mengevaluasi kualitas dan struktur dataset sebelum masuk ke tahap analisis. Pemeriksaan dilakukan terhadap jumlah data, tipe data, missing value, data duplikat, serta outlier.

Berdasarkan hasil pengecekan struktur data, dataset memiliki 5.500 baris dan 17 kolom. Tipe data pada dataset terdiri dari tipe integer, float, dan object. Variabel numerik mencakup usia, BMI, tekanan darah, kolesterol, detak jantung, jumlah langkah harian, tingkat stres, aktivitas fisik, durasi tidur, konsumsi alkohol, dan skor risiko penyakit jantung. Sementara itu, variabel kategorik mencakup status merokok, riwayat keluarga penyakit jantung, dan kategori risiko.

Hasil pengecekan missing value menunjukkan bahwa seluruh kolom memiliki jumlah data yang lengkap. Dengan demikian, tidak terdapat nilai kosong pada dataset dan tidak diperlukan proses imputasi data.

Hasil pengecekan data duplikat juga menunjukkan bahwa tidak terdapat baris data yang terduplikasi. Artinya, setiap data pasien bersifat unik dan tidak ada data yang tercatat lebih dari satu kali.

Selain itu, dilakukan pengecekan outlier pada variabel numerik. Beberapa variabel memiliki outlier, seperti BMI, tekanan darah, kolesterol, detak jantung, langkah harian, aktivitas fisik, durasi tidur, dan konsumsi alkohol. Outlier tersebut perlu ditangani agar tidak terlalu memengaruhi hasil analisis.

3.4 Cleaning Data

Tahap cleaning data dilakukan untuk memastikan dataset berada dalam kondisi yang siap dianalisis. Berdasarkan hasil pengecekan, tidak ditemukan missing value dan data duplikat. Oleh karena itu, proses cleaning difokuskan pada penanganan outlier.

Outlier ditangani menggunakan metode **IQR Capping**. Metode ini bekerja dengan menentukan batas bawah dan batas atas berdasarkan nilai kuartil pertama, kuartil ketiga, dan Interquartile Range. Nilai yang berada di bawah batas bawah akan disesuaikan menjadi nilai

batas bawah, sedangkan nilai yang berada di atas batas atas akan disesuaikan menjadi nilai batas atas.

Metode IQR Capping dipilih karena dapat mengurangi pengaruh nilai ekstrem tanpa menghapus data. Dengan demikian, jumlah data tetap sama, tetapi nilai ekstrem yang berpotensi mengganggu hasil analisis dapat dikendalikan.

Setelah proses penanganan outlier dilakukan, seluruh variabel numerik memiliki jumlah outlier sebesar 0. Hal ini menunjukkan bahwa nilai ekstrem pada dataset telah berhasil ditangani dan data menjadi lebih siap digunakan untuk analisis lanjutan.

BAB IV

PERTANYAAN BISNIS

Pertanyaan bisnis digunakan untuk mengarahkan proses analisis agar hasil yang diperoleh lebih terfokus dan dapat diukur. Pertanyaan bisnis dalam proyek ini adalah sebagai berikut:

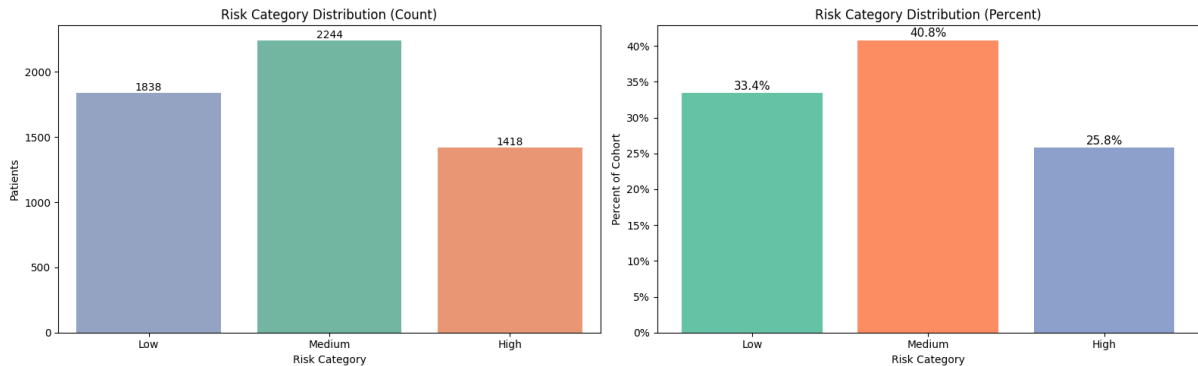
1. Berapa persentase pasien pada setiap kategori risiko kardiovaskular Low, Medium, dan High dalam dataset?
2. Kelompok usia mana yang memiliki proporsi pasien High Risk paling besar?
3. Bagaimana rata-rata tekanan darah, kadar kolesterol, dan heart disease risk score pada setiap kategori risiko?
4. Bagaimana pola gaya hidup pasien berdasarkan kategori risiko, dilihat dari aktivitas fisik, jumlah langkah harian, durasi tidur, kualitas diet, dan konsumsi alkohol?
5. Bagaimana proporsi status merokok dan riwayat keluarga penyakit jantung pada setiap kategori risiko?
6. Fitur klinis dan gaya hidup apa yang paling menonjol pada pasien kategori High Risk berdasarkan hasil EDA?
7. Kelompok pasien mana yang perlu menjadi prioritas edukasi dan pencegahan penyakit jantung?

Pertanyaan-pertanyaan tersebut dijawab melalui proses EDA, visualisasi data, dan dashboard interaktif.

BAB V

EXPLORATORY DATA ANALYSIS

5.1 Distribusi Kategori Risiko

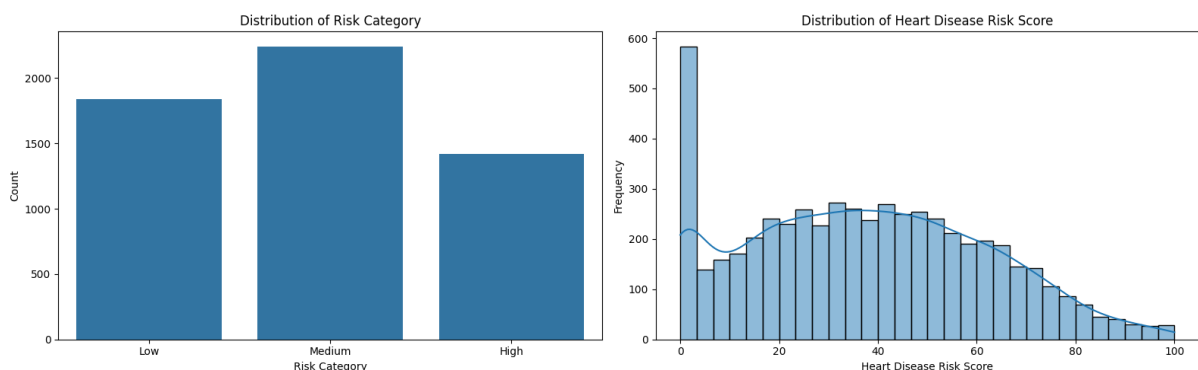


Distribusi kategori risiko menunjukkan bahwa pasien terbagi ke dalam tiga kategori, yaitu Low Risk, Medium Risk, dan High Risk. Berdasarkan hasil analisis, kategori Medium Risk memiliki jumlah pasien paling banyak, diikuti oleh kategori Low Risk, sedangkan kategori High Risk memiliki jumlah pasien paling sedikit.

Kategori Medium Risk terdiri dari 2.244 pasien atau sekitar 40,8% dari total data. Kategori Low Risk terdiri dari 1.838 pasien atau sekitar 33,4%. Sementara itu, kategori High Risk terdiri dari 1.418 pasien atau sekitar 25,8%.

Hasil ini menunjukkan bahwa sebagian besar pasien berada pada tingkat risiko sedang. Kelompok Medium Risk penting untuk diperhatikan karena dapat berpotensi meningkat menjadi High Risk apabila faktor klinis dan gaya hidup tidak dikendalikan. Meskipun jumlah pasien High Risk paling sedikit, persentasenya tetap cukup besar sehingga kelompok ini perlu menjadi prioritas dalam edukasi dan pencegahan.

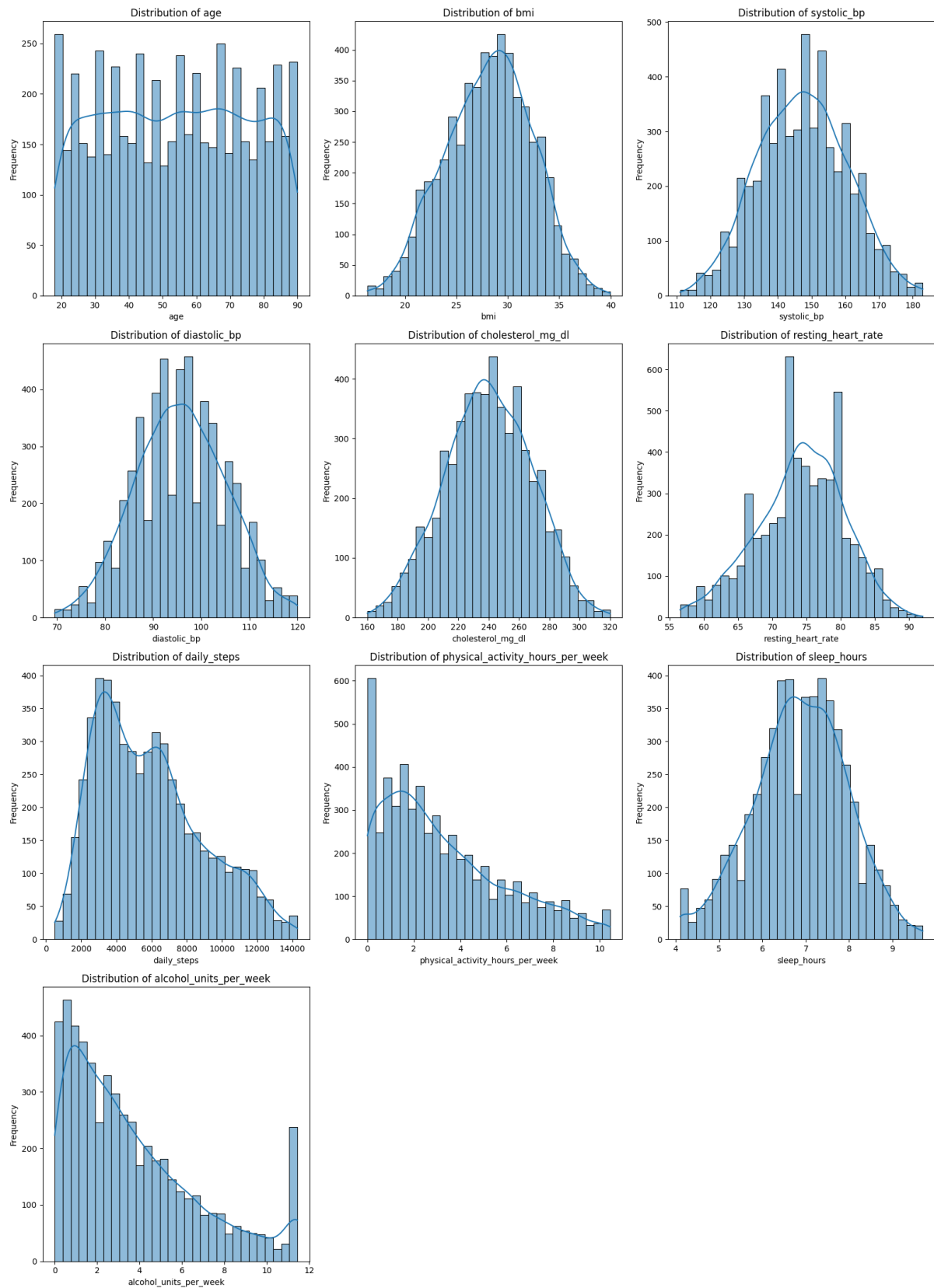
5.2 Distribusi Heart Disease Risk Score



Heart disease risk score memiliki rentang nilai dari rendah hingga tinggi. Sebagian besar skor risiko berada pada tingkat rendah hingga sedang. Frekuensi pasien cenderung menurun pada skor risiko yang lebih tinggi.

Pola ini menunjukkan bahwa pasien dengan skor risiko sangat tinggi jumlahnya relatif lebih sedikit. Namun, kelompok tersebut tetap penting untuk dianalisis karena memiliki potensi risiko kesehatan yang lebih serius. Analisis terhadap heart disease risk score juga membantu memahami hubungan antara variabel klinis dan gaya hidup terhadap tingkat risiko pasien.

5.3 Distribusi Variabel Numerik



Distribusi variabel numerik menunjukkan karakteristik umum pasien dalam dataset. Variabel usia tersebar cukup merata dari usia muda hingga usia lanjut. Variabel BMI

cenderung membentuk distribusi mendekati normal, dengan mayoritas nilai berada pada rentang menengah.

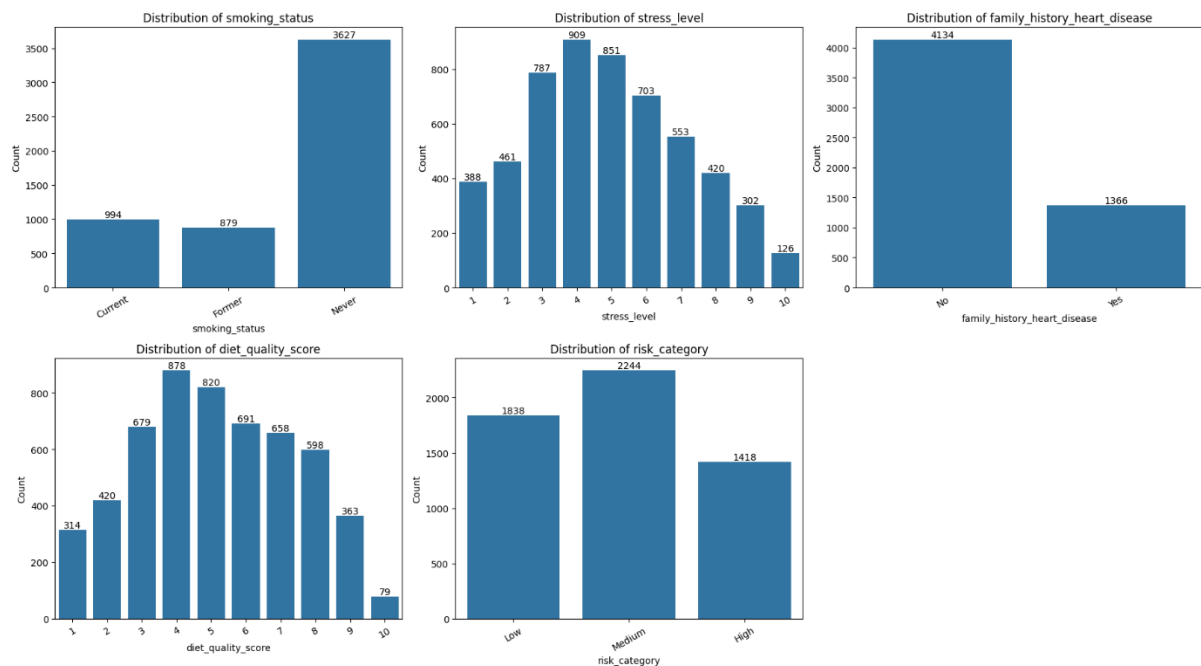
Tekanan darah sistolik dan diastolik juga menunjukkan pola distribusi yang cenderung normal. Hal ini menunjukkan bahwa mayoritas pasien memiliki tekanan darah pada rentang tertentu, tetapi tetap terdapat variasi antar pasien. Kadar kolesterol juga memiliki pola distribusi yang cukup normal, dengan konsentrasi nilai pada rentang menengah.

Variabel resting heart rate paling banyak berada pada kisaran menengah. Sementara itu, variabel daily steps dan physical activity hours per week cenderung condong ke kanan, menunjukkan bahwa sebagian besar pasien memiliki jumlah langkah harian dan aktivitas fisik yang relatif rendah hingga sedang.

Variabel sleep hours menunjukkan distribusi yang relatif normal, dengan mayoritas pasien memiliki durasi tidur sekitar 6 sampai 8 jam. Variabel alcohol units per week cenderung condong ke kanan, menunjukkan bahwa sebagian besar pasien mengonsumsi alkohol dalam jumlah rendah, meskipun terdapat sebagian kecil pasien dengan konsumsi alkohol lebih tinggi.

Secara umum, variabel klinis seperti BMI, tekanan darah, kolesterol, dan durasi tidur memiliki distribusi yang relatif stabil. Sementara itu, variabel gaya hidup seperti aktivitas fisik, langkah harian, dan konsumsi alkohol menunjukkan variasi yang lebih besar.

5.4 Distribusi Variabel Kategorik



Distribusi variabel kategorik menunjukkan bahwa mayoritas pasien memiliki status merokok Never. Artinya, sebagian besar pasien dalam dataset tidak memiliki kebiasaan

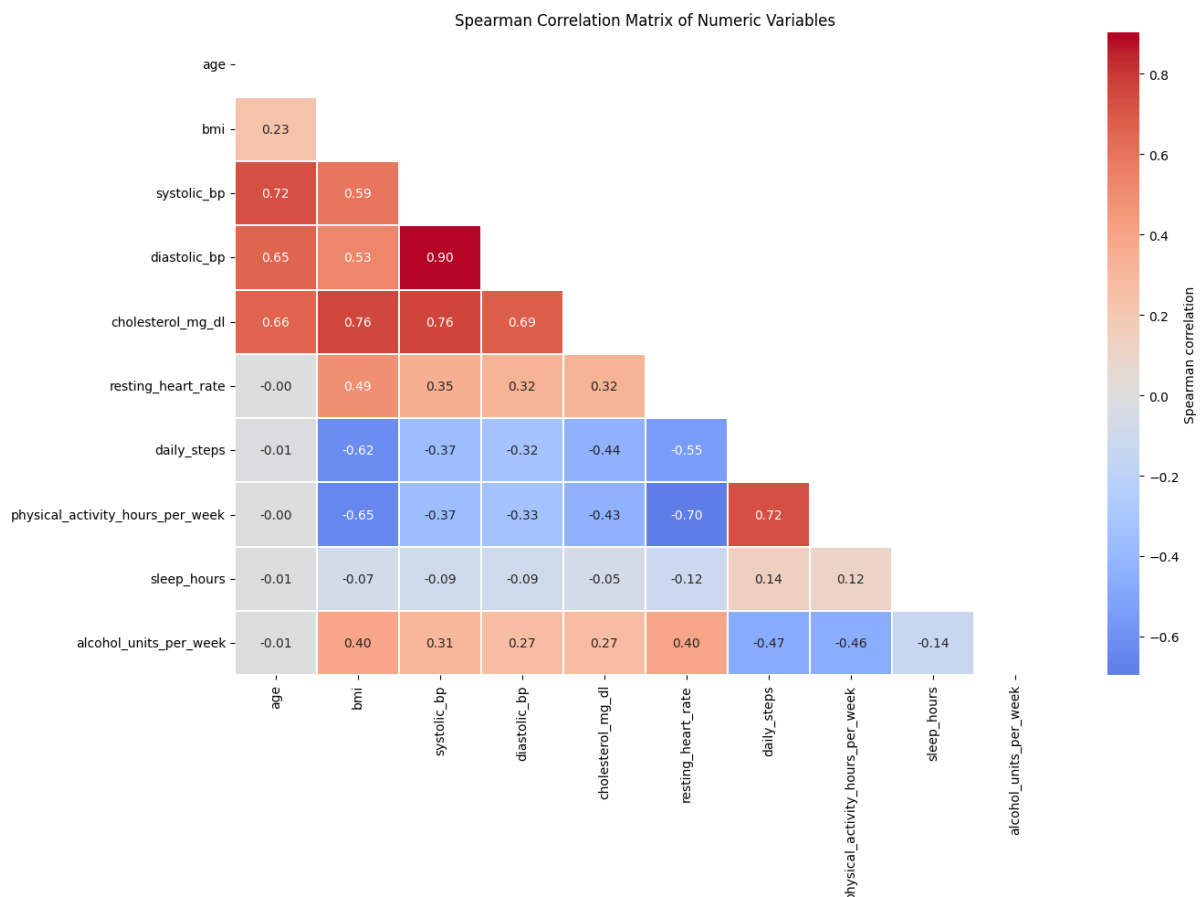
merokok. Jumlah pasien dengan status Current smoker dan Former smoker lebih sedikit dibandingkan pasien yang tidak pernah merokok.

Pada variabel tingkat stres, mayoritas pasien berada pada tingkat stres menengah. Hal ini menunjukkan bahwa tingkat stres dalam dataset tidak hanya terkonsentrasi pada kategori rendah atau tinggi, tetapi tersebar dengan konsentrasi utama di bagian tengah.

Pada variabel family history heart disease, mayoritas pasien tidak memiliki riwayat keluarga penyakit jantung. Namun, masih terdapat sejumlah pasien yang memiliki riwayat keluarga penyakit jantung, sehingga variabel ini tetap penting untuk dianalisis.

Pada variabel diet quality score, mayoritas nilai berada pada skor menengah. Hal ini menunjukkan bahwa kualitas diet pasien dalam dataset cenderung berada pada tingkat sedang.

5.5 Korelasi Variabel Numerik



Analisis korelasi Spearman digunakan untuk melihat hubungan antarvariabel numerik. Hasil korelasi menunjukkan bahwa tekanan darah sistolik dan tekanan darah diastolik memiliki hubungan positif yang sangat kuat. Hal ini wajar karena kedua variabel tersebut sama-sama merepresentasikan kondisi tekanan darah pasien.

Kadar kolesterol memiliki korelasi positif dengan BMI, tekanan darah sistolik, dan tekanan darah diastolik. Hal ini menunjukkan bahwa pasien dengan BMI dan tekanan darah yang lebih tinggi cenderung memiliki kadar kolesterol yang lebih tinggi.

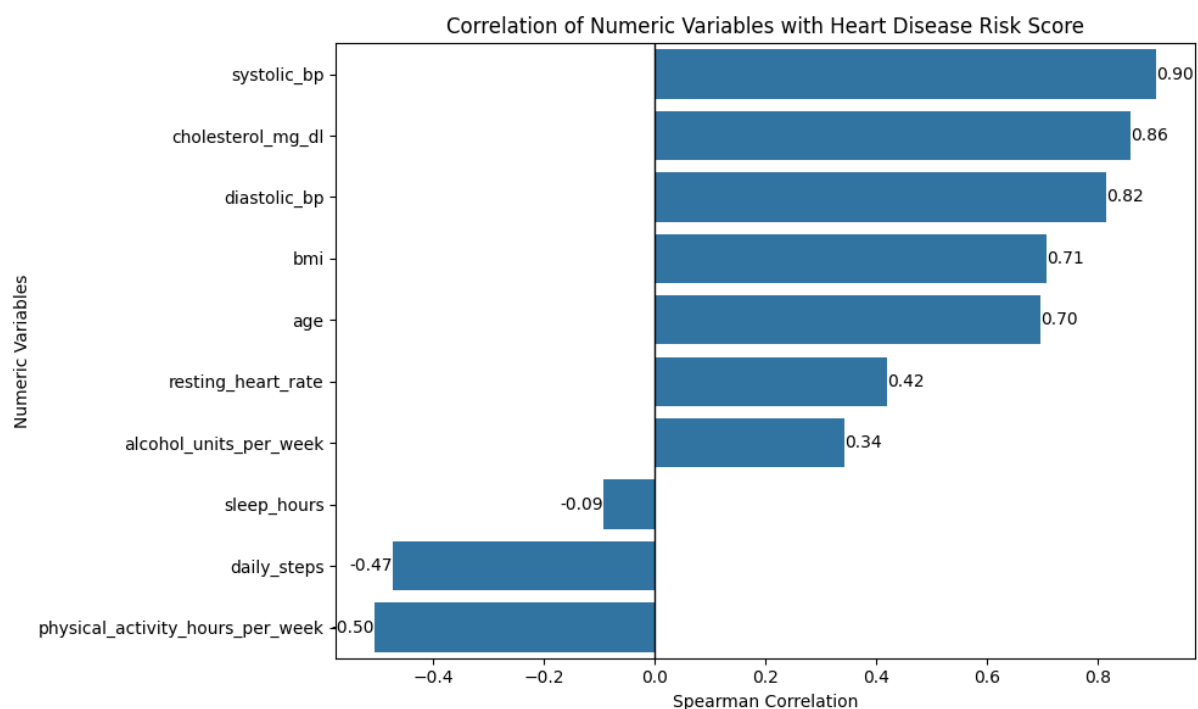
Usia memiliki korelasi positif dengan tekanan darah dan kolesterol. Artinya, semakin tinggi usia pasien, tekanan darah dan kolesterol cenderung meningkat.

Variabel daily steps dan physical activity hours per week memiliki korelasi positif. Hal ini menunjukkan bahwa pasien yang lebih aktif secara fisik cenderung memiliki jumlah langkah harian yang lebih tinggi.

Sebaliknya, physical activity hours per week memiliki korelasi negatif dengan resting heart rate. Artinya, semakin tinggi aktivitas fisik, detak jantung saat istirahat cenderung lebih rendah.

Secara umum, variabel klinis seperti tekanan darah, BMI, dan kolesterol saling berkorelasi positif. Sementara itu, variabel aktivitas fisik cenderung berkorelasi negatif dengan faktor risiko kesehatan.

5.6 Korelasi Variabel Numerik dengan Heart Disease Risk Score



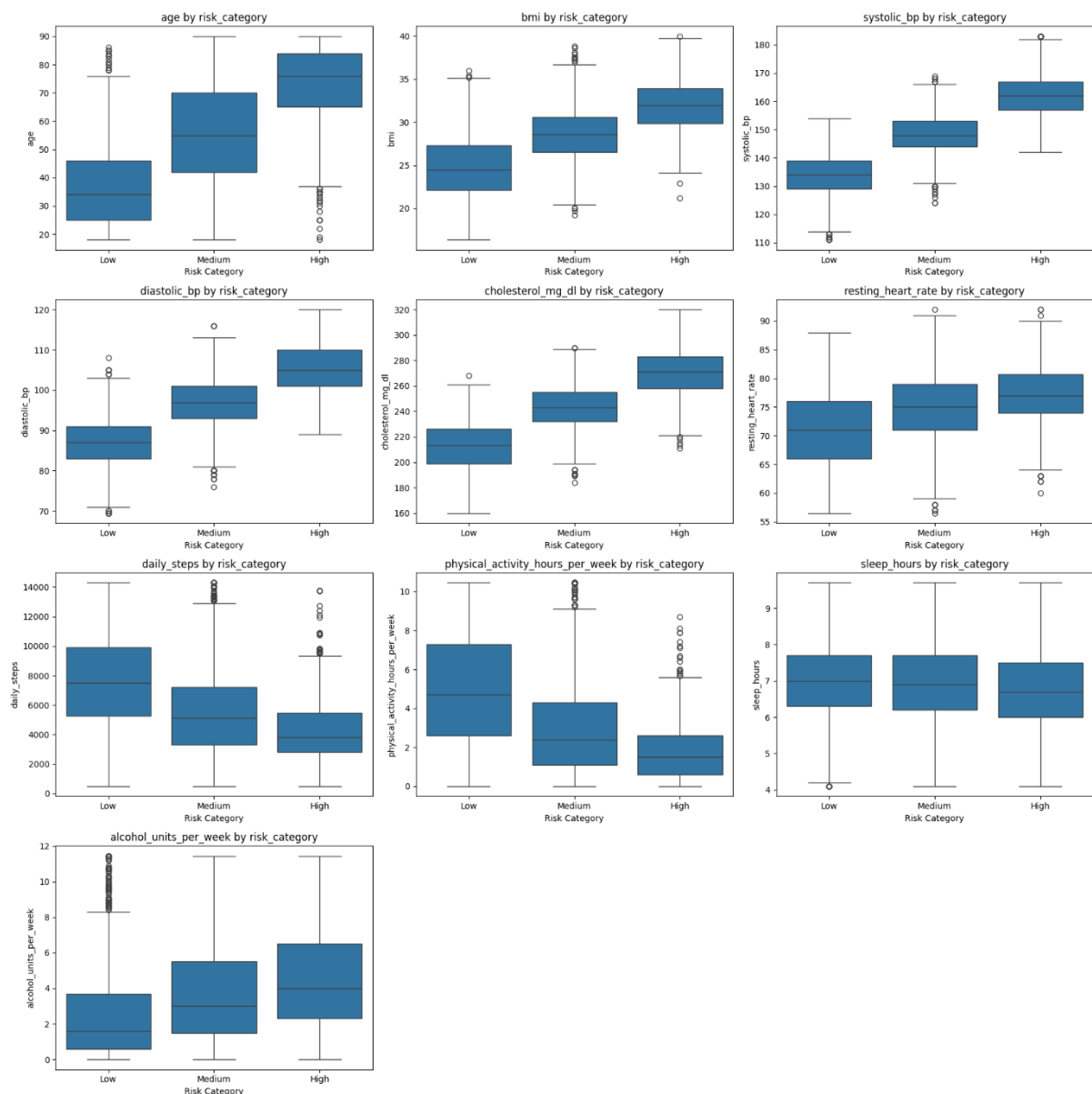
Hasil analisis menunjukkan bahwa variabel yang memiliki korelasi positif paling kuat dengan heart disease risk score adalah tekanan darah sistolik. Selain itu, kadar kolesterol dan tekanan darah diastolik juga memiliki korelasi positif yang kuat.

Variabel BMI dan usia juga memiliki hubungan positif yang cukup kuat dengan skor risiko penyakit jantung. Artinya, semakin tinggi tekanan darah, kolesterol, BMI, dan usia, maka skor risiko penyakit jantung cenderung meningkat.

Resting heart rate dan alcohol units per week memiliki hubungan positif, tetapi tidak sekuat tekanan darah, kolesterol, BMI, dan usia. Sementara itu, physical activity hours per week dan daily steps memiliki korelasi negatif dengan heart disease risk score. Artinya, semakin tinggi aktivitas fisik dan jumlah langkah harian, maka skor risiko penyakit jantung cenderung lebih rendah.

Sleep hours memiliki korelasi negatif yang sangat lemah, sehingga hubungannya dengan skor risiko tidak terlalu kuat dalam dataset ini.

5.7 Hubungan Variabel Numerik dengan Risk Category



Analisis boxplot berdasarkan risk category menunjukkan adanya perbedaan karakteristik antara pasien Low Risk, Medium Risk, dan High Risk. Pasien kategori High Risk cenderung memiliki usia yang lebih tinggi dibandingkan kategori lainnya.

Nilai BMI juga meningkat dari kategori Low Risk ke Medium Risk dan High Risk. Hal ini menunjukkan bahwa pasien dengan kategori risiko lebih tinggi cenderung memiliki BMI yang lebih besar.

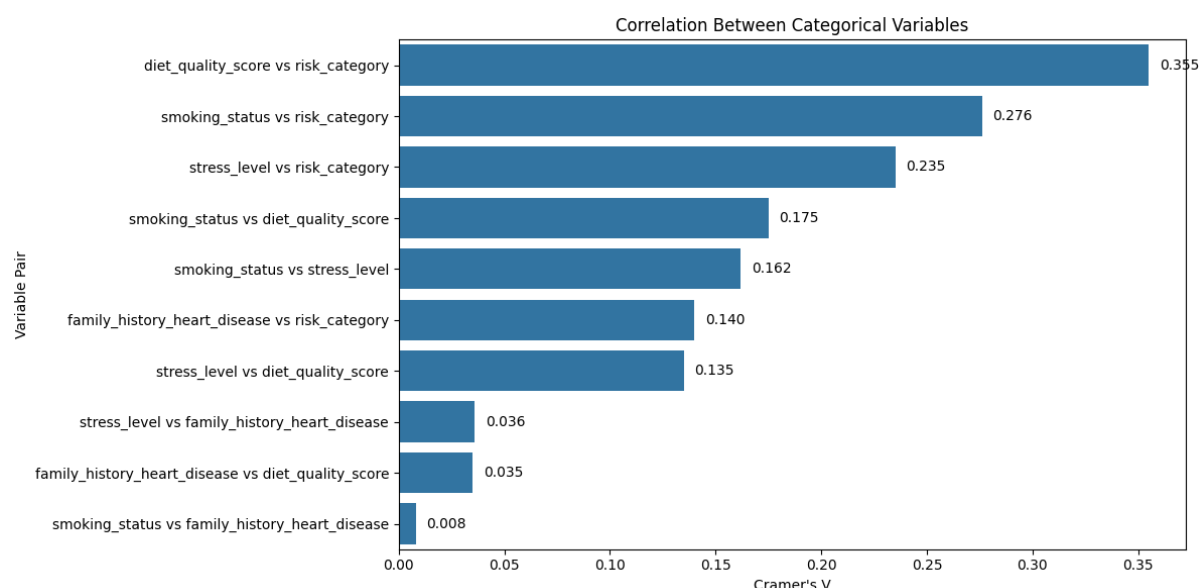
Tekanan darah sistolik dan diastolik terlihat semakin tinggi pada kategori risiko yang lebih tinggi. Kadar kolesterol juga meningkat seiring naiknya kategori risiko. Selain itu, resting heart rate pada pasien High Risk cenderung lebih tinggi dibandingkan kategori lainnya.

Sebaliknya, daily steps dan physical activity hours per week cenderung lebih rendah pada pasien dengan kategori risiko lebih tinggi. Hal ini menunjukkan bahwa pasien High Risk cenderung memiliki aktivitas fisik yang lebih rendah.

Variabel sleep hours terlihat relatif mirip antar kategori risiko, sehingga perbedaannya tidak terlalu kuat. Sementara itu, alcohol units per week cenderung lebih tinggi pada kategori High Risk.

Secara keseluruhan, pasien High Risk memiliki karakteristik klinis yang lebih buruk, seperti usia, BMI, tekanan darah, kolesterol, dan detak jantung yang lebih tinggi. Di sisi lain, pasien Low Risk cenderung memiliki aktivitas fisik dan jumlah langkah harian yang lebih tinggi.

5.8 Hubungan Variabel Kategorik dengan Risk Category



Analisis hubungan variabel kategorik dengan risk category dilakukan menggunakan ukuran asosiasi Cramer's V. Hasil analisis menunjukkan bahwa variabel diet quality score memiliki hubungan paling kuat dengan risk category dibandingkan variabel kategorik lainnya.

Variabel smoking status juga memiliki hubungan dengan kategori risiko. Hal ini menunjukkan bahwa status merokok berkaitan dengan perbedaan kategori risiko pasien. Selain itu, stress level juga memiliki hubungan dengan risk category, sehingga tingkat stres perlu diperhatikan dalam analisis risiko kardiovaskular.

Variabel family history heart disease memiliki hubungan yang lebih lemah dibandingkan diet quality score, smoking status, dan stress level. Meskipun demikian, riwayat keluarga tetap menjadi variabel penting karena dapat menjadi salah satu indikator risiko.

Secara umum, variabel kategorik yang paling berkaitan dengan risk category adalah kualitas diet, status merokok, dan tingkat stres.

BAB VI

FEATURE ENGINEERING

6.1 Tujuan Feature Engineering

Feature engineering dilakukan untuk membuat variabel baru yang dapat memperkaya proses analisis. Fitur baru dibuat berdasarkan kombinasi atau pengelompokan dari variabel yang sudah ada. Tujuannya adalah agar informasi dalam dataset dapat disajikan secara lebih informatif dan mudah dianalisis.

Feature engineering juga membantu dalam memahami risiko pasien dari sisi klinis dan gaya hidup. Beberapa fitur baru dibuat untuk mengelompokkan nilai numerik ke dalam kategori tertentu, sedangkan fitur lainnya dibuat dalam bentuk skor gabungan.

6.2 Fitur yang Dibuat

Fitur baru yang dibuat dalam proyek ini antara lain:

1. **pulse_pressure**

Fitur ini diperoleh dari selisih antara tekanan darah sistolik dan tekanan darah diastolik. Pulse pressure digunakan untuk melihat perbedaan tekanan darah atas dan bawah.

2. **blood_pressure_ratio**

Fitur ini diperoleh dari rasio antara tekanan darah sistolik dan tekanan darah diastolik. Fitur ini digunakan untuk melihat perbandingan antara dua indikator tekanan darah.

3. **hypertension_stage**

Fitur ini digunakan untuk mengelompokkan tingkat tekanan darah pasien berdasarkan nilai tekanan darah sistolik dan diastolik.

4. **bmi_category**

Fitur ini digunakan untuk mengelompokkan nilai BMI pasien ke dalam kategori tertentu, seperti normal, overweight, atau obese.

5. **cholesterol_category**

Fitur ini digunakan untuk mengelompokkan kadar kolesterol pasien berdasarkan rentang nilai tertentu.

6. **activity_level**

Fitur ini digunakan untuk mengelompokkan tingkat aktivitas pasien berdasarkan aktivitas fisik atau jumlah langkah harian.

7. **sleep_category**

Fitur ini digunakan untuk mengelompokkan durasi tidur pasien ke dalam kategori tertentu.

8. **alcohol_category**

Fitur ini digunakan untuk mengelompokkan konsumsi alkohol pasien berdasarkan jumlah unit alkohol per minggu.

9. **lifestyle_risk_score**

Fitur ini digunakan untuk menggambarkan risiko berdasarkan faktor gaya hidup, seperti aktivitas fisik, tidur, alkohol, merokok, dan kualitas diet.

10. **clinical_risk_score**

Fitur ini digunakan untuk menggambarkan risiko berdasarkan faktor klinis, seperti usia, BMI, tekanan darah, kolesterol, dan detak jantung.

11. **age_group**

Fitur ini digunakan untuk mengelompokkan usia pasien ke dalam beberapa kelompok usia, seperti kurang dari 30 tahun, 30-39 tahun, 40-49 tahun, dan seterusnya.

6.3 Hasil Feature Engineering

Setelah dilakukan feature engineering, dataset memiliki informasi tambahan yang dapat digunakan untuk analisis lanjutan. Fitur-fitur baru tersebut membantu memperjelas pola risiko pasien, terutama dalam membandingkan karakteristik pasien Low Risk, Medium Risk, dan High Risk.

Fitur seperti clinical risk score dan lifestyle risk score dapat membantu membedakan risiko berdasarkan aspek klinis dan gaya hidup. Sementara itu, fitur kategorik seperti age group, bmi category, dan cholesterol category membantu menyajikan analisis dalam bentuk yang lebih mudah dipahami.

BAB VII

DASHBOARD INTERAKTIF

7.1 Tujuan Dashboard

Dashboard interaktif dibuat sebagai media penyajian hasil analisis data. Dashboard membantu pengguna untuk memahami distribusi risiko, karakteristik pasien, hubungan antarvariabel, serta rekomendasi prioritas edukasi pencegahan penyakit jantung.

Dengan dashboard, hasil analisis tidak hanya disajikan dalam bentuk tabel dan grafik statis, tetapi juga dapat dieksplorasi secara interaktif menggunakan filter. Hal ini membuat pengguna lebih mudah memahami pola risiko berdasarkan kategori tertentu.

7.2 Tools Dashboard

Dashboard dibuat menggunakan Streamlit sebagai framework utama. Library yang digunakan meliputi Pandas, NumPy, Plotly Express, Plotly Graph Objects, dan Plotly Subplots.

Streamlit digunakan untuk membangun tampilan dashboard berbasis web. Plotly digunakan untuk membuat visualisasi interaktif, seperti pie chart, boxplot, bar chart, scatterplot, gauge chart, dan visualisasi lainnya.

7.3 Struktur Dashboard

Dashboard terdiri dari tiga halaman utama, yaitu:

1. **Dashboard**
2. **Relationship Features**
3. **Prioritas Edukasi**

Setiap halaman memiliki fungsi yang berbeda. Halaman Dashboard digunakan untuk menyajikan ringkasan utama dan visualisasi inti. Halaman Relationship Features digunakan untuk melihat hubungan antara variabel numerik dengan heart disease risk score. Halaman Prioritas Edukasi digunakan untuk menampilkan rekomendasi pencegahan berdasarkan hasil analisis.

7.4 Fitur Filter Dashboard

Dashboard dilengkapi dengan beberapa filter interaktif, yaitu:

1. Filter kategori risiko.
2. Filter riwayat keluarga penyakit jantung.
3. Filter kelompok usia.
4. Filter rentang BMI.

Filter ini memungkinkan pengguna untuk melihat hasil analisis berdasarkan kelompok pasien tertentu. Misalnya, pengguna dapat memilih hanya pasien High Risk, pasien dengan riwayat keluarga penyakit jantung, kelompok usia tertentu, atau rentang BMI tertentu.

7.5 KPI Dashboard

Dashboard menampilkan beberapa indikator utama atau Key Performance Indicators (KPI), yaitu:

1. Jumlah pasien.
2. Rata-rata kolesterol.
3. Rata-rata langkah harian.
4. Rata-rata heart disease risk score.
5. Rata-rata aktivitas fisik per minggu.

KPI ini membantu pengguna memperoleh gambaran cepat mengenai kondisi data berdasarkan filter yang dipilih.

7.6 Visualisasi pada Halaman Dashboard

Halaman Dashboard menampilkan beberapa visualisasi utama, yaitu:

1. Persentase Pasien per Kategori Risiko

Visualisasi ini menunjukkan proporsi pasien pada kategori Low Risk, Medium Risk, dan High Risk.

2. Distribusi Heart Risk Score berdasarkan Kelompok Usia

Visualisasi ini menunjukkan persebaran skor risiko penyakit jantung pada setiap kelompok usia.

3. Rata-rata Tekanan Darah, Kolesterol, dan Heart Risk

Visualisasi ini digunakan untuk membandingkan indikator klinis antar kategori risiko.

4. Riwayat Keluarga Penyakit Jantung

Visualisasi ini menunjukkan proporsi pasien dengan dan tanpa riwayat keluarga penyakit jantung pada setiap kategori risiko.

5. Fitur yang Menonjol pada Pasien High Risk

Visualisasi ini menunjukkan perbedaan relatif antara pasien High Risk dan Low Risk pada beberapa fitur penting.

6. Pola Gaya Hidup per Kategori Risiko

Visualisasi ini menampilkan perbandingan aktivitas fisik, langkah harian, durasi tidur, kualitas diet, dan konsumsi alkohol pada setiap kategori risiko.

7.7 Halaman Relationship Features

Halaman Relationship Features digunakan untuk menampilkan hubungan antara variabel numerik dengan heart disease risk score. Visualisasi yang digunakan berupa scatterplot dengan garis tren.

Variabel yang dianalisis antara lain usia, BMI, tekanan darah sistolik, tekanan darah diastolik, kadar kolesterol, resting heart rate, daily steps, physical activity hours per week, sleep hours, dan alcohol units per week.

Melalui halaman ini, pengguna dapat melihat pola hubungan antara setiap variabel numerik dengan skor risiko penyakit jantung. Variabel seperti tekanan darah, kolesterol, BMI, dan usia cenderung memiliki hubungan positif dengan skor risiko. Sementara itu, aktivitas fisik dan jumlah langkah harian cenderung memiliki hubungan negatif dengan skor risiko.

7.8 Halaman Prioritas Edukasi

Halaman Prioritas Edukasi berisi rekomendasi berdasarkan hasil analisis. Rekomendasi ini ditujukan untuk membantu menentukan kelompok pasien yang perlu mendapatkan perhatian lebih.

Prioritas edukasi yang ditampilkan pada dashboard meliputi:

1. Prioritaskan pasien kategori High Risk.
2. Kontrol tekanan darah dan kolesterol secara rutin.
3. Dorong aktivitas fisik dan peningkatan langkah harian.
4. Perbaiki kualitas diet dan tidur pasien.
5. Fokus pada pasien dengan riwayat keluarga penyakit jantung.

Halaman ini menjadi bagian penting karena mengubah hasil analisis menjadi rekomendasi yang lebih mudah dipahami dan dapat ditindaklanjuti.

7.9 Deployment Dashboard

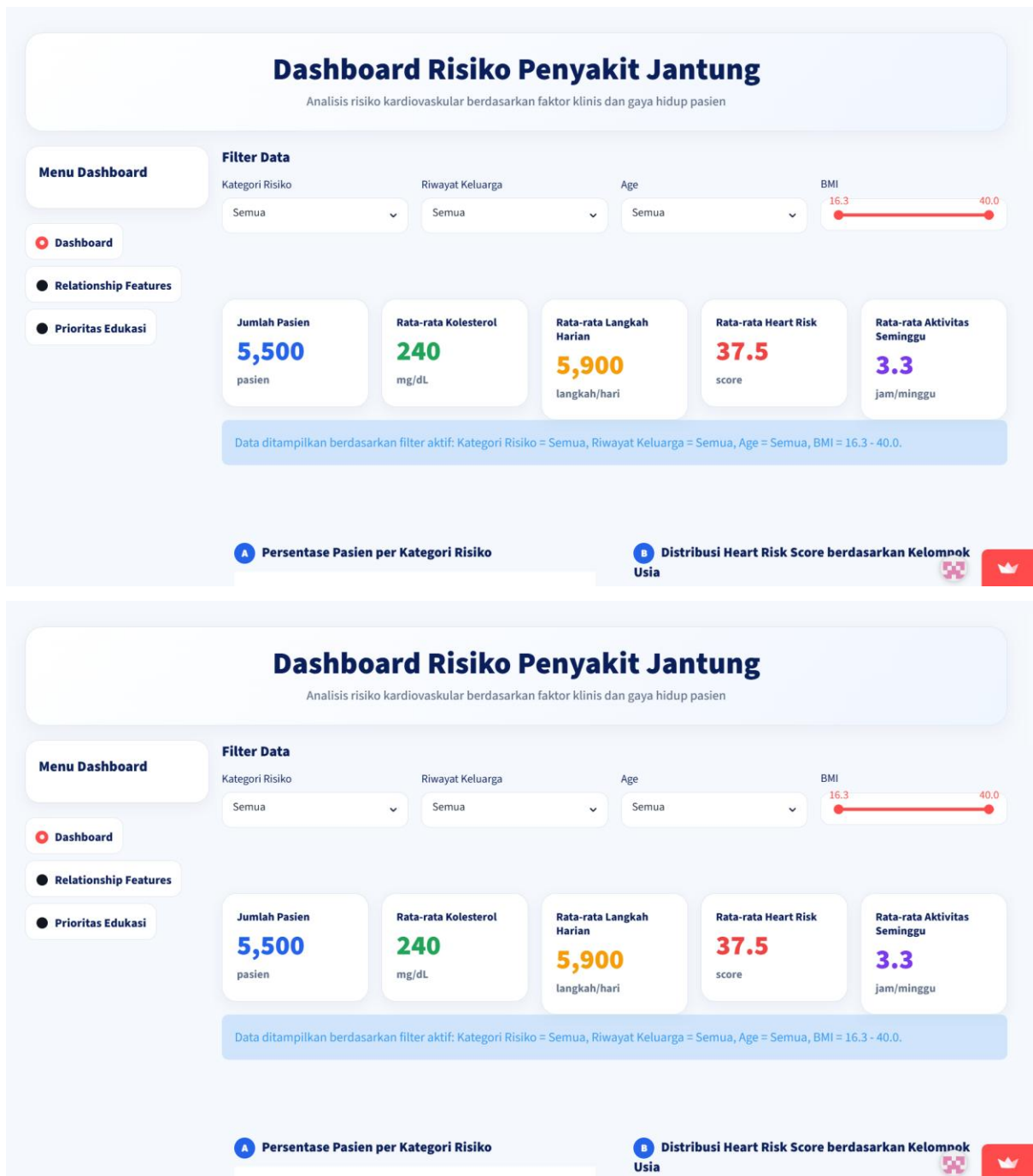
Dashboard interaktif yang telah dibuat kemudian dideploy menggunakan Streamlit Cloud agar dapat diakses secara publik melalui browser. Deployment ini bertujuan agar hasil analisis risiko penyakit kardiovaskular dapat digunakan dan ditampilkan secara interaktif tanpa harus menjalankan kode secara lokal.

Dashboard dapat diakses melalui tautan berikut:

<https://capstonedashboardheart.streamlit.app/>

Melalui dashboard tersebut, pengguna dapat melihat ringkasan data, distribusi kategori risiko, hubungan antar fitur numerik dengan skor risiko penyakit jantung, pola gaya hidup pasien, serta rekomendasi prioritas edukasi pencegahan.

7.10 Tampilan Dashboard



Dashboard Risiko Penyakit Jantung

Analisis risiko kardiovaskular berdasarkan faktor klinis dan gaya hidup pasien

Menu Dashboard

Dashboard

Relationship Features

Prioritas Edukasi

Filter Data

Kategori Risiko

Semua

Riwayat Keluarga

Semua

Age

Semua

BMI

16.3

40.0

Jumlah Pasien

5,500

pasien

Rata-rata Kolesterol

240

mg/dL

Rata-rata Langkah
Harian

5,900

langkah/hari

Rata-rata Heart Risk

37.5

score

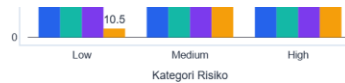
Rata-rata Aktivitas
Minggu

3.3

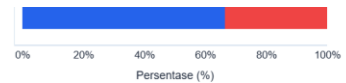
jam/minggu

Data ditampilkan berdasarkan filter aktif: Kategori Risiko = Semua, Riwayat Keluarga = Semua, Age = Semua, BMI = 16.3 - 40.0.

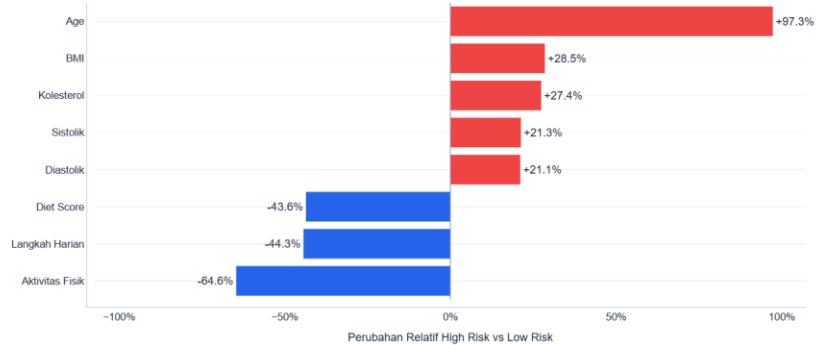
A Persentase Pasien per Kategori Risiko



B Distribusi Heart Risk Score berdasarkan Kelompok Usia



E Fitur yang Menonjol pada Pasien High Risk



F Pola Gaya Hidup per Kategori Risiko

Rata-rata Aktivitas Fisik

Rata-rata Langkah Harian

8000 7577.5

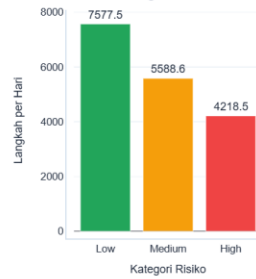
Rata-rata Durasi Tidur

F Pola Gaya Hidup per Kategori Risiko

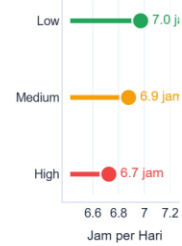
Rata-rata Aktivitas Fisik



Rata-rata Langkah Harian



Rata-rata Durasi Tidur

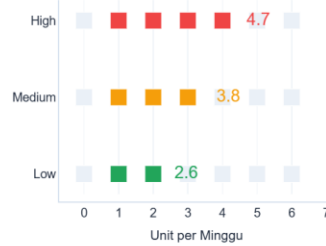


Skor Kualitas Diet

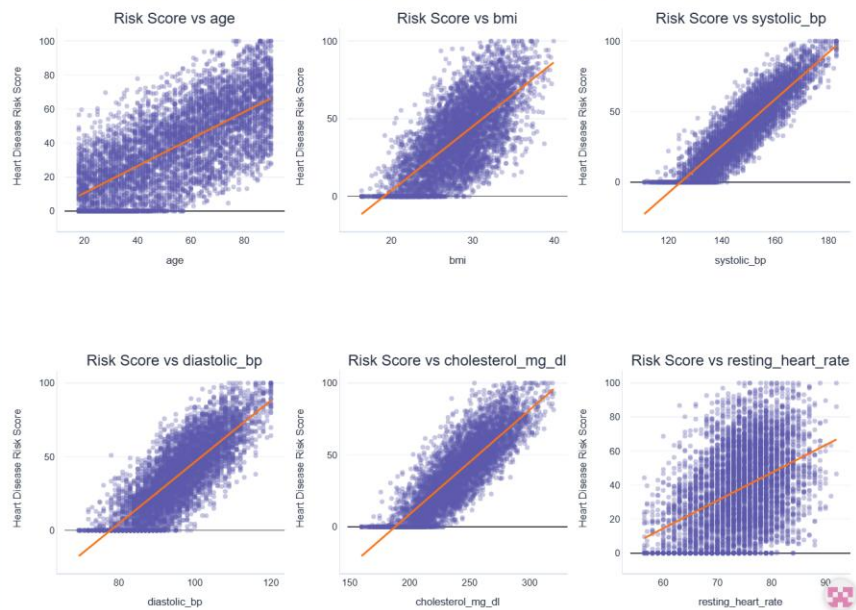


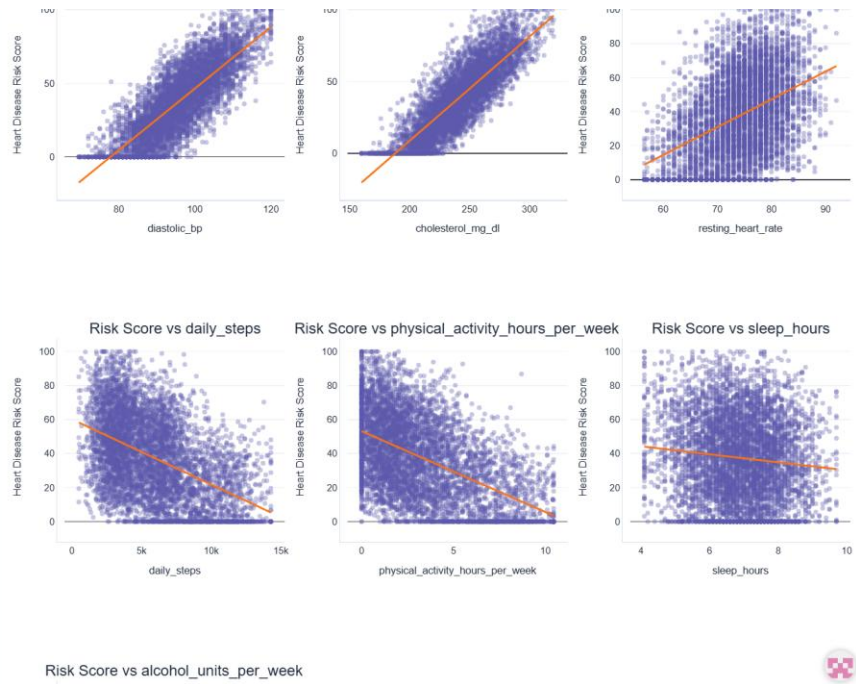
Skor Diet (0-10). Semakin tinggi skor, semakin baik kualitas diet.

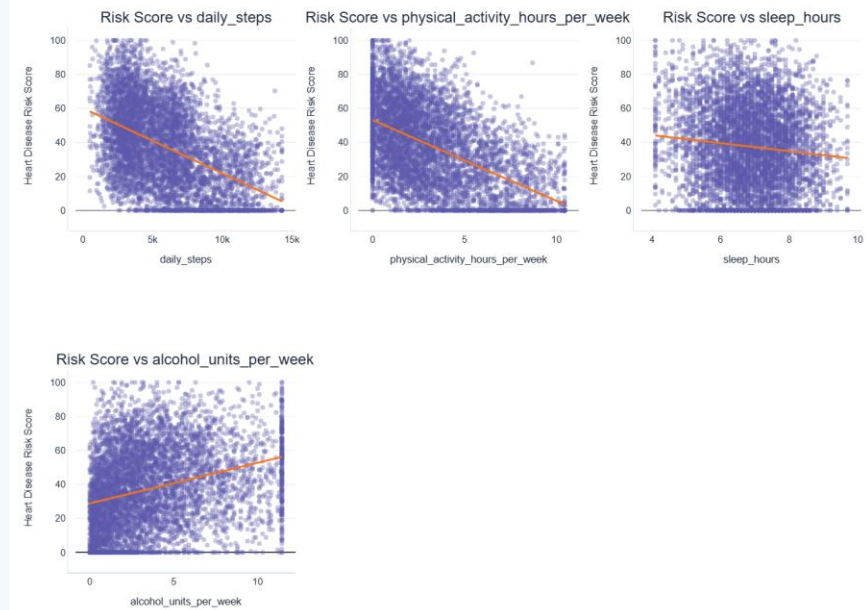
Konsumsi Alkohol per Kategori Risiko



Relationship Between Numeric Features and Heart Disease Risk Score







langkah/hari jam/minggu

Data ditampilkan berdasarkan filter aktif: Kategori Risiko = Semua, Riwayat Keluarga = Semua, Age = Semua, BMI = 16.3 - 40.0.

Prioritas Edukasi Pencegahan

1. Prioritaskan pasien kategori High Risk.

Kelompok ini paling membutuhkan monitoring dan edukasi pencegahan.

2. Kontrol tekanan darah dan kolesterol secara rutin.

Dua indikator ini dominan pada pasien berisiko tinggi.

3. Dorong aktivitas fisik dan peningkatan langkah harian.

Gaya hidup kurang aktif berkaitan dengan risiko yang lebih tinggi.

4. Perbaiki kualitas diet dan tidur pasien.

Pola hidup sehat membantu menurunkan faktor risiko kardiovaskular.

5. Fokus pada pasien dengan riwayat keluarga penyakit jantung.

Kelompok ini perlu skrining dan edukasi lebih dini.

BAB VIII

HASIL AKHIR DAN INSIGHT UTAMA

8.1 Hasil Akhir Proyek

Hasil akhir dari proyek ini adalah laporan analisis data dan dashboard interaktif risiko penyakit kardiovaskular. Proyek ini berhasil melakukan tahapan data science secara end-to-end, mulai dari problem discovery, data wrangling, exploratory data analysis, feature engineering, hingga pembuatan dashboard.

Dataset telah melalui proses pengecekan kualitas data, termasuk pengecekan missing value, data duplikat, dan outlier. Data juga telah dibersihkan menggunakan metode IQR Capping untuk menangani nilai ekstrem. Selain itu, dilakukan feature engineering untuk menambahkan informasi baru yang mendukung analisis.

Dashboard interaktif berhasil dibuat menggunakan Streamlit. Dashboard ini memungkinkan pengguna untuk mengeksplorasi data berdasarkan kategori risiko, riwayat keluarga penyakit jantung, kelompok usia, dan rentang BMI.

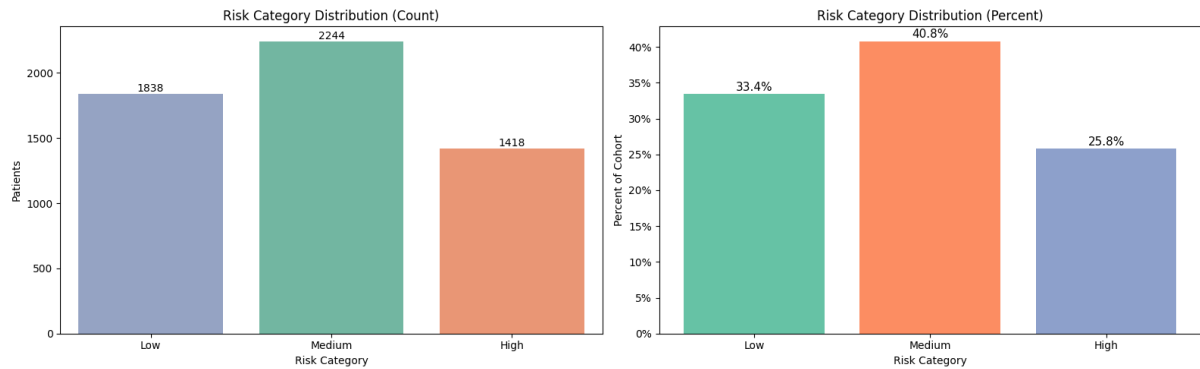
8.2 Insight Utama

Berdasarkan hasil analisis, diperoleh beberapa insight utama sebagai berikut:

1. Mayoritas pasien berada pada kategori Medium Risk.
2. Pasien High Risk tetap memiliki proporsi yang cukup besar sehingga perlu menjadi perhatian.
3. Pasien High Risk cenderung memiliki usia yang lebih tinggi.
4. Pasien High Risk cenderung memiliki BMI yang lebih tinggi.
5. Tekanan darah sistolik dan diastolik meningkat seiring dengan naiknya kategori risiko.
6. Kadar kolesterol lebih tinggi pada pasien dengan kategori risiko lebih tinggi.
7. Resting heart rate cenderung lebih tinggi pada pasien High Risk.
8. Jumlah langkah harian lebih rendah pada pasien dengan kategori risiko lebih tinggi.
9. Aktivitas fisik per minggu lebih rendah pada pasien High Risk.
10. Konsumsi alkohol cenderung lebih tinggi pada pasien High Risk.
11. Kualitas diet, status merokok, dan tingkat stres memiliki hubungan dengan kategori risiko.
12. Aktivitas fisik dan jumlah langkah harian memiliki hubungan negatif dengan skor risiko penyakit jantung.

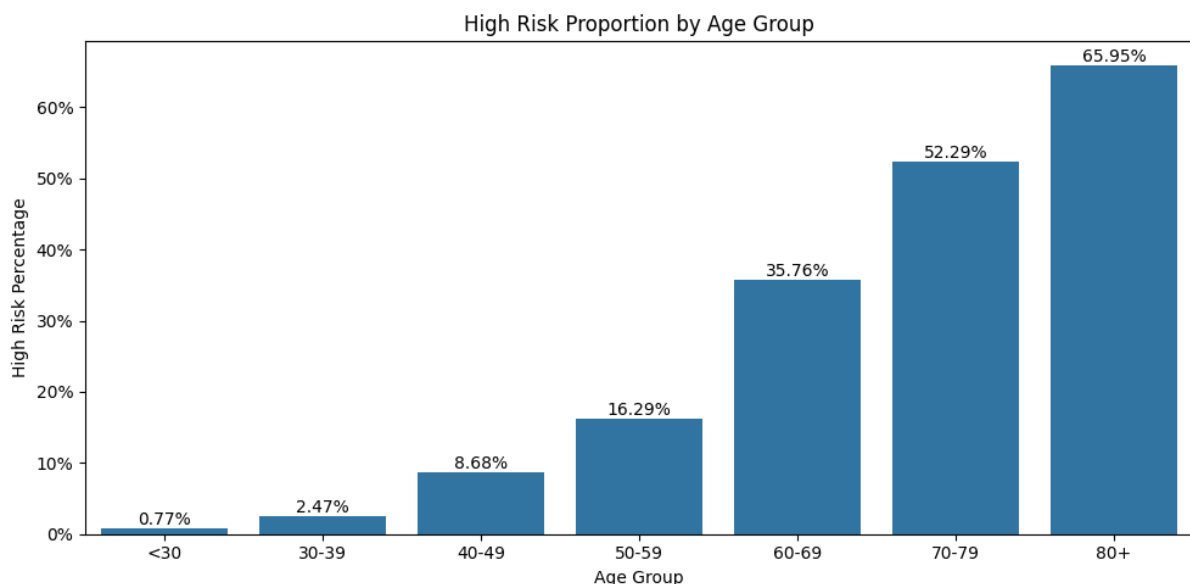
8.3 Jawaban Pertanyaan Bisnis

1. Berapa persentase pasien pada setiap kategori risiko kardiovaskular?



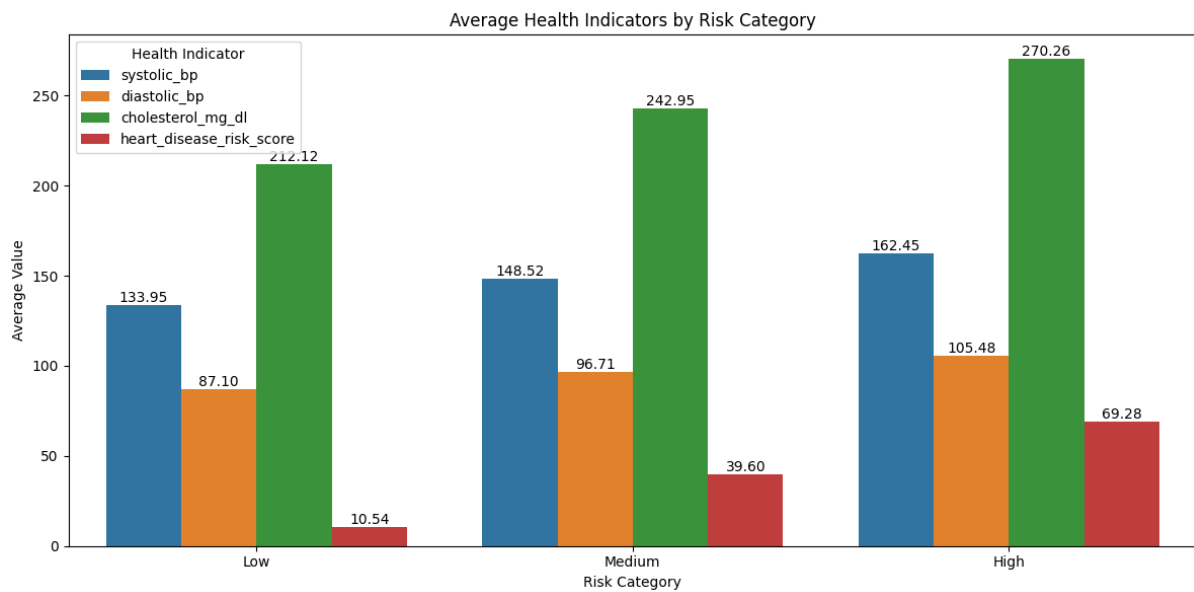
Pasien kategori Medium Risk memiliki persentase paling besar, yaitu sekitar 40,8%. Kategori Low Risk memiliki persentase sekitar 33,4%, sedangkan kategori High Risk memiliki persentase sekitar 25,8%.

2. Kelompok usia mana yang memiliki proporsi pasien High Risk paling besar?



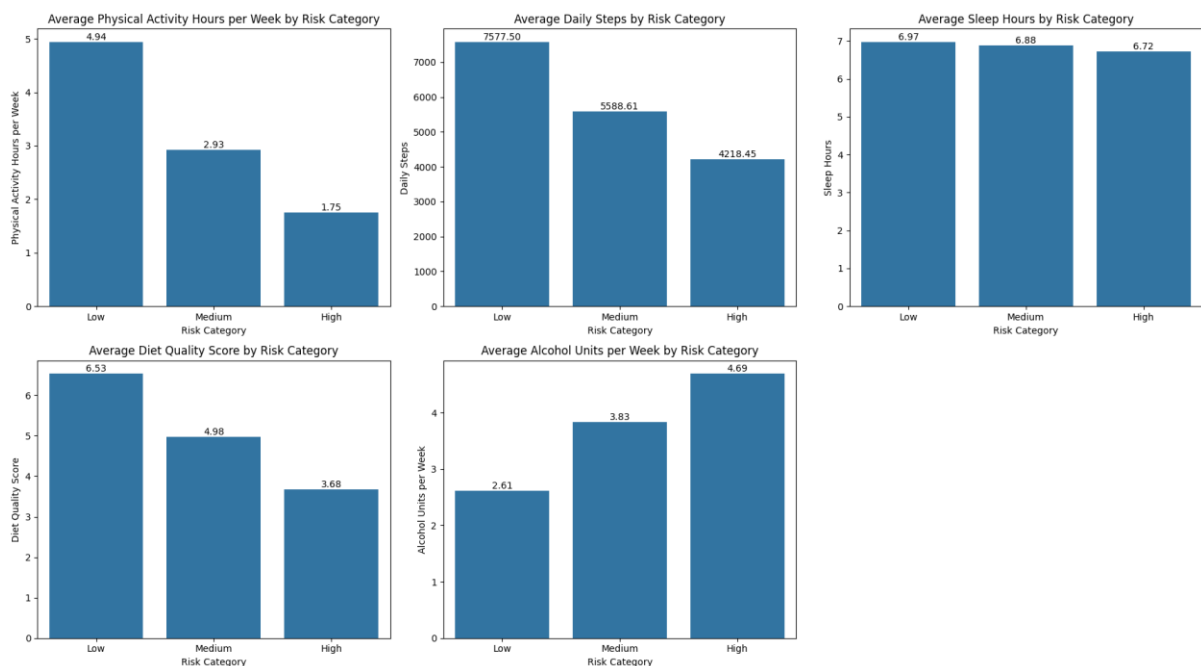
Berdasarkan hasil analisis, pasien dengan usia lebih tinggi cenderung memiliki risiko yang lebih besar. Hal ini terlihat dari peningkatan skor risiko dan kategori risiko pada kelompok usia lanjut.

3. Bagaimana rata-rata tekanan darah, kadar kolesterol, dan heart disease risk score pada setiap kategori risiko?



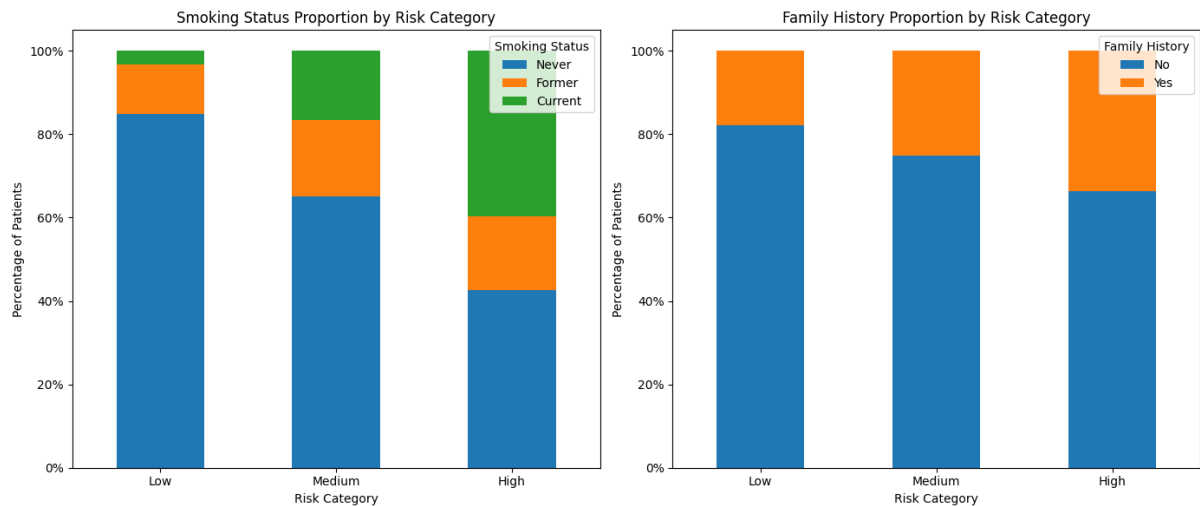
Rata-rata tekanan darah, kadar kolesterol, dan heart disease risk score cenderung meningkat dari kategori Low Risk ke Medium Risk dan High Risk. Hal ini menunjukkan bahwa faktor klinis memiliki hubungan yang kuat dengan tingkat risiko kardiovaskular.

4. Bagaimana pola gaya hidup pasien berdasarkan kategori risiko?



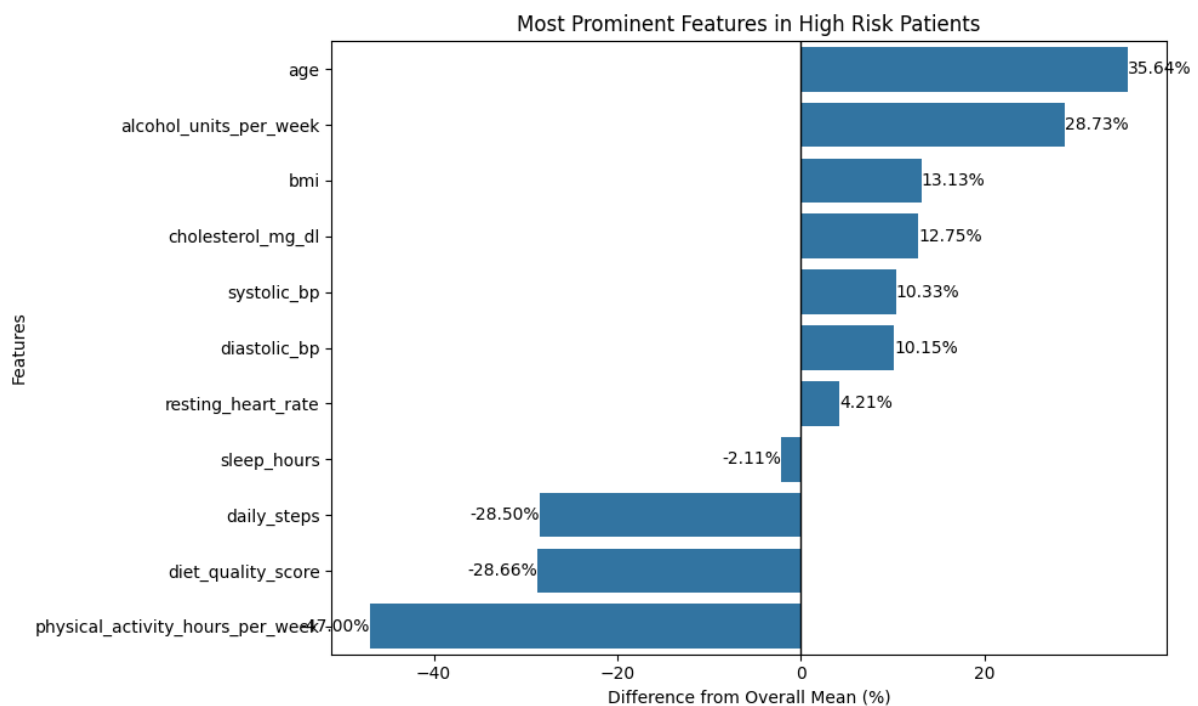
Pasien High Risk cenderung memiliki aktivitas fisik dan jumlah langkah harian yang lebih rendah. Selain itu, konsumsi alkohol cenderung lebih tinggi pada kelompok risiko tinggi. Kualitas diet juga berkaitan dengan kategori risiko.

5. Bagaimana proporsi status merokok dan riwayat keluarga penyakit jantung pada setiap kategori risiko?



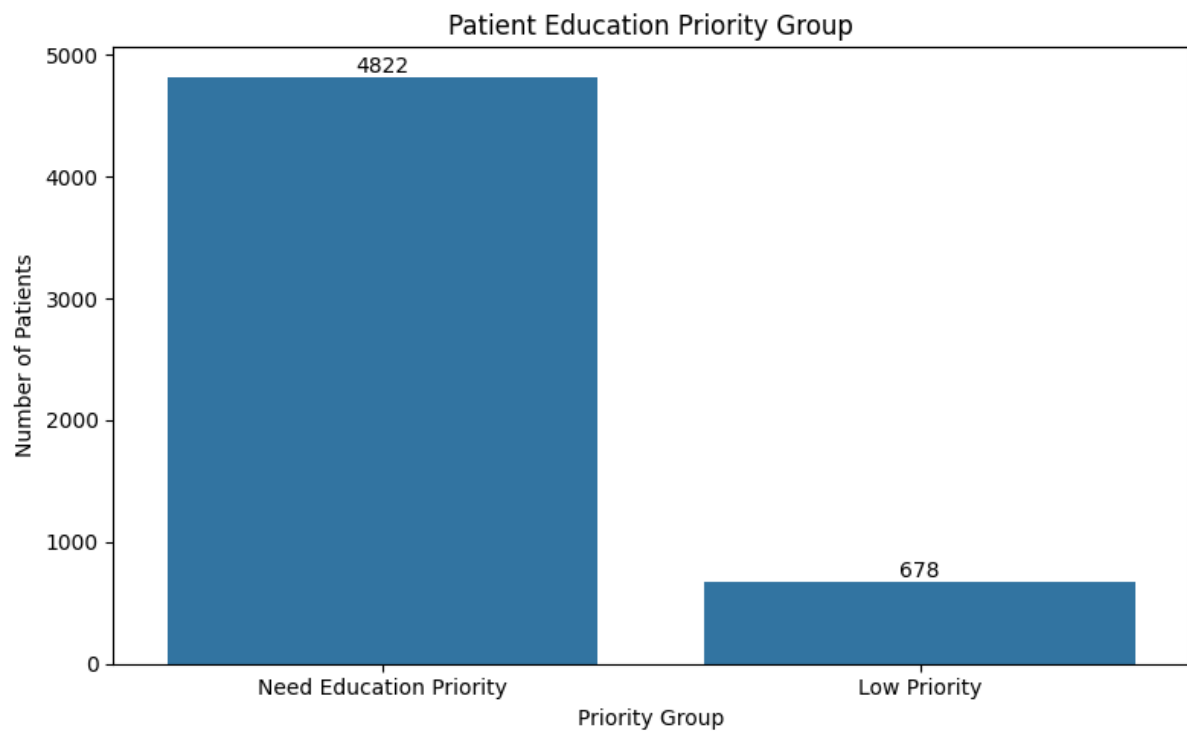
Status merokok memiliki hubungan dengan kategori risiko. Pasien dengan riwayat keluarga penyakit jantung juga perlu diperhatikan, meskipun asosiasinya tidak sekuat variabel lain seperti kualitas diet, status merokok, dan tingkat stres.

6. Fitur klinis dan gaya hidup apa yang paling menonjol pada pasien High Risk?



Fitur yang menonjol pada pasien High Risk adalah usia, BMI, tekanan darah, kolesterol, resting heart rate, aktivitas fisik yang rendah, langkah harian yang rendah, serta konsumsi alkohol yang lebih tinggi.

7. Kelompok pasien mana yang perlu diprioritaskan dalam edukasi dan pencegahan penyakit jantung?



Kelompok pasien yang perlu diprioritaskan adalah pasien High Risk, pasien dengan tekanan darah dan kolesterol tinggi, pasien dengan aktivitas fisik rendah, pasien dengan konsumsi alkohol tinggi, serta pasien dengan riwayat keluarga penyakit jantung.

BAB IX

A/B TESTING

9.1 Tujuan A/B Testing

A/B Testing dilakukan untuk membandingkan skor risiko antara dua kelompok input berdasarkan variabel tertentu. Pada proyek ini, pengujian difokuskan pada variabel **riwayat penyakit** dan **kolesterol** karena keduanya menunjukkan perbedaan skor risiko yang signifikan.

Output yang dibandingkan adalah skor risiko yang dihasilkan oleh model website.

9.2 Desain Pengujian

A/B Testing dilakukan dengan membagi data menjadi dua kelompok pada setiap variabel.

Variabel	Kelompok A	Kelompok B
Riwayat Penyakit	Tidak ada riwayat penyakit	Ada riwayat penyakit
Kolesterol	Kolesterol normal (kolestrol < 200)	Kolesterol tinggi (kolestrol >= 200)

9.3 Metode Pengujian

Pengujian dilakukan menggunakan **Mann-Whitney U Test** karena data berjumlah kecil dan digunakan untuk membandingkan dua kelompok independen.

Hipotesis yang digunakan:

- H_0 : Tidak terdapat perbedaan skor risiko antara Kelompok A dan Kelompok B.
- H_1 : Terdapat perbedaan skor risiko antara Kelompok A dan Kelompok B.

9.4 Hasil dan Interpretasi

Pada variabel **riwayat penyakit**, kelompok tanpa riwayat penyakit memiliki rata-rata skor risiko sebesar **0,1579**, sedangkan kelompok dengan riwayat penyakit memiliki rata-rata skor risiko sebesar **0,3409**. Nilai **p-value** = **0,0150** lebih kecil dari **0,05**, sehingga terdapat perbedaan skor risiko yang signifikan antara kedua kelompok.

Pada variabel **kolesterol**, kelompok kolesterol normal memiliki rata-rata skor risiko sebesar **0,1536**, sedangkan kelompok kolesterol tinggi memiliki rata-rata skor risiko sebesar **0,2663**. Nilai **p-value** = **0,0368** lebih kecil dari **0,05**, sehingga terdapat perbedaan skor risiko yang signifikan antara kedua kelompok.

9.5 Kesimpulan

Berdasarkan hasil A/B Testing, variabel **riwayat penyakit** dan **kolesterol** menunjukkan perbedaan skor risiko yang signifikan. Kelompok dengan riwayat penyakit dan kelompok

dengan kolesterol tinggi memiliki rata-rata skor risiko yang lebih besar dibandingkan kelompok pembandingnya.

Dengan demikian, riwayat penyakit dan kolesterol menjadi faktor penting yang perlu diperhatikan dalam analisis risiko penyakit kardiovaskular.

CLEANING_DATA

June 5, 2026

1 GATHERING DATASET

2 Dataset yang Digunakan

Dataset yang digunakan pada proyek ini adalah **Heart Attack risk Dataset**. Dataset ini berisi data kesehatan pasien yang berkaitan dengan risiko penyakit kardiovaskular. Data tersebut digunakan untuk membangun model prediksi tingkat risiko penyakit jantung berdasarkan beberapa parameter kesehatan.

2.1 Sumber Dataset

Dataset diperoleh dari Kaggle melalui tautan berikut: [cardiovascular_risk_dataset.csv](#) - Kaggle

3 DATA DICTIONARY

3.1 Data Dictionary - Heart Risk Progression Dataset

Dataset ini merupakan data sintetis yang digunakan untuk penelitian dan eksperimen machine learning terkait risiko penyakit jantung.

No	Feature	Tipe Data	Jenis Variabel	Deskripsi	Rentang / Nilai	Contoh
1	Patient_ID	Integer	Identifier	ID unik untuk setiap pasien	1 – 5500	1, 2, 3
2	age	Integer	Numerical	Usia pasien dalam tahun	18 – 90	62, 54, 46

No	Feature	Tipe Data	Jenis Variabel	Deskripsi	Rentang / Nilai	Contoh
3	bmi	Float	Numerical	Body Mass Index (BMI), indikator lemak tubuh berdasarkan tinggi dan berat badan	~16 – 40	25.0, 29.7
4	systolic_bp	Integer	Numerical	Tekanan darah sistolik pasien (mmHg)	~111 – 183	142, 158
5	diastolic_bp	Float	Numerical	Tekanan darah diastolik pasien (mmHg)	~69 – 120	93, 101
6	cholesterol_mg/dL	Integer	Numerical	Total kadar kolesterol dalam darah (mg/dL)	~160 – 320	247, 254
7	resting_heart_rate	Float	Numerical	Detak jantung saat istirahat (Beats per minute (BPM))	~56 – 92	72, 74
8	smoking_status	Object	Nominal	Status kebiasaan merokok pasien	Never, Former, Current	Never
9	daily_steps	Integer	Numerical	Rata-rata jumlah langkah per hari	~500 – 14,000	11565, 4036
10	stress_level	Integer	Ordinal	Tingkat stres pasien	1 – 10	3, 8

No	Feature	Tipe Data	Jenis Variabel	Deskripsi	Rentang / Nilai	Contoh
11	physical_activity_hours_per_week	Float	Numerical	Jumlah jam aktivitas fisik per minggu	0 – 10	5.6, 0.5
12	sleep_hours	Float	Numerical	Rata-rata durasi tidur per malam (jam)	~4 – 9	8.2, 6.7
13	family_history_of_heart_disease	Object	Binary	Menunjukkan apakah pasien memiliki riwayat keluarga penyakit jantung	Yes, No	Yes
14	diet_quality_score	Integer	Ordinal	Skor kualitas pola makan pasien	1 – 10	7, 5
15	alcohol_units_per_week	Float	Numerical	Konsumsi alkohol per minggu	0 – ~11	0.7, 4.5
16	heart_disease_risk_score	Float	Numerical (Target)	Skor risiko penyakit jantung yang dihitung	0 – 100	28.1, 63.0
17	risk_category	Object	Ordinal (Target)	Kategori risiko penyakit jantung	Low, Medium, High	Medium

3.2 Library

```
[13]: import os
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

4 Assessing Data

4.1 Import Data

```
[14]: df = pd.read_csv('/content/cardiovascular_risk_dataset.csv')
display(df.head())
```

	Patient_ID	age	bmi	systolic_bp	diastolic_bp	cholesterol_mg_dl	\
0	1	62	25.0	142	93	247	
1	2	54	29.7	158	101	254	
2	3	46	36.2	170	113	276	
3	4	48	30.4	153	98	230	
4	5	46	25.3	139	87	206	

	resting_heart_rate	smoking_status	daily_steps	stress_level	\
0	72	Never	11565	3	
1	74	Current	4036	8	
2	80	Current	3043	9	
3	73	Former	5604	5	
4	69	Current	7464	1	

	physical_activity_hours_per_week	sleep_hours	family_history_heart_disease	\
0	5.6	8.2	No	
1	0.5	6.7	No	
2	0.4	4.0	No	
3	0.6	8.0	No	
4	2.0	6.1	No	

	diet_quality_score	alcohol_units_per_week	heart_disease_risk_score	\
0	7	0.7	28.1	
1	5	4.5	63.0	
2	1	20.8	73.1	
3	4	8.5	39.5	
4	5	3.6	29.3	

	risk_category
0	Medium
1	High
2	High
3	Medium
4	Medium

```
[15]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5500 entries, 0 to 5499
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
#   ...
```

```

---  -----
0  Patient_ID      5500 non-null  int64
1  age             5500 non-null  int64
2  bmi             5500 non-null  float64
3  systolic_bp     5500 non-null  int64
4  diastolic_bp    5500 non-null  int64
5  cholesterol_mg_dl 5500 non-null  int64
6  resting_heart_rate 5500 non-null  int64
7  smoking_status   5500 non-null  object
8  daily_steps      5500 non-null  int64
9  stress_level     5500 non-null  int64
10 physical_activity_hours_per_week 5500 non-null  float64
11 sleep_hours      5500 non-null  float64
12 family_history_heart_disease 5500 non-null  object
13 diet_quality_score 5500 non-null  int64
14 alcohol_units_per_week 5500 non-null  float64
15 heart_disease_risk_score 5500 non-null  float64
16 risk_category    5500 non-null  object
dtypes: float64(5), int64(9), object(3)
memory usage: 730.6+ KB

```

- Dataset memiliki **5.500 baris data** dan **17 kolom**.
- Setiap baris merepresentasikan data satu pasien.
- Semua kolom memiliki **5.500 data non-null**, sehingga tidak terdapat missing value.

```
[16]: df.head()
```

```

[16]:  Patient_ID  age  bmi  systolic_bp  diastolic_bp  cholesterol_mg_dl  \
0          1    62  25.0          142           93             247
1          2    54  29.7          158          101             254
2          3    46  36.2          170          113             276
3          4    48  30.4          153           98             230
4          5    46  25.3          139           87             206

      resting_heart_rate  smoking_status  daily_steps  stress_level  \
0                   72         Never      11565           3
1                   74         Current       4036           8
2                   80         Current       3043           9
3                   73         Former       5604           5
4                   69         Current       7464           1

      physical_activity_hours_per_week  sleep_hours  family_history_heart_disease  \
0                                5.6           8.2                        No
1                                0.5           6.7                        No
2                                0.4           4.0                        No
3                                0.6           8.0                        No
4                                2.0           6.1                        No

```

	diet_quality_score	alcohol_units_per_week	heart_disease_risk_score	\
0	7	0.7	28.1	
1	5	4.5	63.0	
2	1	20.8	73.1	
3	4	8.5	39.5	
4	5	3.6	29.3	

	risk_category
0	Medium
1	High
2	High
3	Medium
4	Medium

```
[17]: df.shape
```

```
[17]: (5500, 17)
```

```
[18]: df.describe()
```

```
[18]:
```

	Patient_ID	age	bmi	systolic_bp	diastolic_bp	\
count	5500.000000	5500.000000	5500.000000	5500.000000	5500.000000	
mean	2750.500000	53.872000	28.170818	147.248182	95.756727	
std	1587.857571	21.196017	4.189877	13.222701	9.451559	
min	1.000000	18.000000	15.000000	108.000000	64.000000	
25%	1375.750000	36.000000	25.200000	138.000000	89.000000	
50%	2750.500000	54.000000	28.400000	147.000000	96.000000	
75%	4125.250000	72.000000	31.100000	156.000000	102.000000	
max	5500.000000	90.000000	40.900000	192.000000	120.000000	

	cholesterol_mg_dl	resting_heart_rate	daily_steps	stress_level	\
count	5500.000000	5500.000000	5500.000000	5500.000000	
mean	239.684182	74.075091	5902.929455	4.907091	
std	28.570177	6.392166	3041.084590	2.298173	
min	147.000000	48.000000	500.000000	1.000000	
25%	220.000000	70.000000	3428.000000	3.000000	
50%	240.000000	74.000000	5460.000000	5.000000	
75%	260.000000	79.000000	7772.000000	7.000000	
max	331.000000	92.000000	16793.000000	10.000000	

	physical_activity_hours_per_week	sleep_hours	diet_quality_score	\
count	5500.000000	5500.000000	5500.000000	
mean	3.299364	6.869364	5.162909	
std	2.672457	1.091263	2.286134	
min	0.000000	4.000000	1.000000	
25%	1.200000	6.200000	3.000000	
50%	2.600000	6.900000	5.000000	

75%	4.900000	7.600000	7.000000
max	12.900000	10.000000	10.000000

	alcohol_units_per_week	heart_disease_risk_score
count	5500.000000	5500.000000
mean	3.782200	37.540455
std	3.515594	24.287026
min	0.000000	0.000000
25%	1.200000	18.400000
50%	2.800000	36.700000
75%	5.300000	55.500000
max	29.200000	100.000000

5 CLEANING DATA

5.1 Missing value

```
[19]: df.isnull().sum()
```

```
[19]: Patient_ID      0
      age            0
      bmi            0
      systolic_bp    0
      diastolic_bp   0
      cholesterol_mg_dl 0
      resting_heart_rate 0
      smoking_status 0
      daily_steps    0
      stress_level   0
      physical_activity_hours_per_week 0
      sleep_hours    0
      family_history_heart_disease    0
      diet_quality_score              0
      alcohol_units_per_week          0
      heart_disease_risk_score        0
      risk_category                   0
      dtype: int64
```

- Semua kolom memiliki **5.500 data non-null**, sehingga tidak terdapat missing value.

5.2 Cek Duplicate

```
[20]: df.duplicated().sum()
```

```
[20]: np.int64(0)
```

- Berdasarkan hasil pengecekan, tidak ditemukan data duplikat pada dataset.

- Artinya, setiap baris data memiliki informasi yang unik dan tidak ada data pasien yang tercatat lebih dari satu kali.
- Karena tidak terdapat duplikasi, tidak perlu dilakukan proses penghapusan data duplikat.
- Kondisi ini menunjukkan bahwa dataset sudah cukup baik dari sisi keunikan data dan dapat dilanjutkan ke tahap analisis berikutnya.

5.3 Outlier

```
[21]: kolom_numerik = [
    'age',
    'bmi',
    'systolic_bp',
    'diastolic_bp',
    'cholesterol_mg_dl',
    'resting_heart_rate',
    'daily_steps',
    'physical_activity_hours_per_week',
    'sleep_hours',
    'alcohol_units_per_week'
]
```

```
[22]: outlier_summary = []

for col in kolom_numerik:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outlier_count = ((df[col] < lower_bound) | (df[col] > upper_bound)).sum()
    outlier_percentage = (outlier_count / len(df)) * 100

    outlier_summary.append({
        "Column": col,
        "Outlier Count": outlier_count,
        "Outlier Percentage (%)": round(outlier_percentage, 2)
    })

outlier_df = pd.DataFrame(outlier_summary)
outlier_df
```

```
[22]:
```

	Column	Outlier Count	Outlier Percentage (%)
0	age	0	0.00
1	bmi	9	0.16
2	systolic_bp	15	0.27

3	diastolic_bp	5	0.09
4	cholesterol_mg_dl	7	0.13
5	resting_heart_rate	17	0.31
6	daily_steps	21	0.38
7	physical_activity_hours_per_week	42	0.76
8	sleep_hours	81	1.47
9	alcohol_units_per_week	214	3.89

- Berdasarkan hasil deteksi outlier, sebagian besar variabel memiliki jumlah outlier yang relatif kecil.

5.3.1 Penanganan outlier

```
[23]: # Kolom numerik yang ingin ditangani outlier-nya
numeric_cols = [
    'bmi',
    'systolic_bp',
    'diastolic_bp',
    'cholesterol_mg_dl',
    'resting_heart_rate',
    'daily_steps',
    'physical_activity_hours_per_week',
    'sleep_hours',
    'alcohol_units_per_week'
]

# Salinan dataframe
df_clean = df.copy()

# Capping outlier menggunakan metode IQR
for col in numeric_cols:
    Q1 = df_clean[col].quantile(0.25)
    Q3 = df_clean[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    df_clean[col] = df_clean[col].clip(lower=lower_bound, upper=upper_bound)
```

Outlier pada dataset ditangani menggunakan metode **IQR Capping**. Metode ini bekerja dengan membatasi nilai ekstrem berdasarkan batas bawah dan batas atas dari perhitungan **Interquartile Range (IQR)**.

```
[24]: outlier_summary_df_clean = []

for col in kolom_numerik:
```

```

Q1 = df_clean[col].quantile(0.25)
Q3 = df_clean[col].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outlier_count = ((df_clean[col] < lower_bound) | (df_clean[col] >
↪upper_bound)).sum()
outlier_percentage = (outlier_count / len(df_clean)) * 100

outlier_summary_df_clean.append({
    "Column": col,
    "Outlier Count": outlier_count,
    "Outlier Percentage (%)": round(outlier_percentage, 2)
})

outlier_df_clean = pd.DataFrame(outlier_summary_df_clean)
outlier_df_clean

```

```

[24]:
      Column  Outlier Count  Outlier Percentage (%)
0         age              0                0.0
1         bmi              0                0.0
2    systolic_bp           0                0.0
3    diastolic_bp           0                0.0
4 cholesterol_mg_dl        0                0.0
5 resting_heart_rate        0                0.0
6    daily_steps           0                0.0
7 physical_activity_hours_per_week  0                0.0
8         sleep_hours        0                0.0
9 alcohol_units_per_week        0                0.0

```

- Setelah dilakukan proses penanganan outlier, seluruh variabel numerik memiliki jumlah outlier sebesar **0**.
- Persentase outlier pada semua variabel juga menjadi **0,00%**.
- Hal ini menunjukkan bahwa nilai ekstrem pada dataset sudah berhasil ditangani.

6 PERTANYAAN BISNIS

6.1 Business Questions

1. Berapa persentase pasien pada setiap kategori risiko kardiovaskular (Low, Medium, dan High) dalam dataset?
2. Kelompok usia mana yang memiliki proporsi pasien High Risk paling besar?
3. Bagaimana rata-rata tekanan darah, kadar kolesterol, dan heart_disease_risk_score pada setiap kategori risiko?

4. Bagaimana pola gaya hidup pasien berdasarkan kategori risiko, dilihat dari aktivitas fisik, jumlah langkah harian, durasi tidur, kualitas diet, dan konsumsi alkohol?
5. Bagaimana proporsi status merokok dan riwayat keluarga penyakit jantung pada setiap kategori risiko?
6. Fitur klinis dan gaya hidup apa yang paling menonjol pada pasien kategori High Risk berdasarkan hasil EDA?
7. Kelompok pasien mana yang perlu diprioritaskan dalam edukasi dan pencegahan penyakit jantung?

7 EDA

[25]: `df_clean.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5500 entries, 0 to 5499
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Patient_ID                           5500 non-null   int64
1   age                                   5500 non-null   int64
2   bmi                                   5500 non-null   float64
3   systolic_bp                          5500 non-null   int64
4   diastolic_bp                         5500 non-null   float64
5   cholesterol_mg_dl                   5500 non-null   int64
6   resting_heart_rate                  5500 non-null   float64
7   smoking_status                      5500 non-null   object
8   daily_steps                         5500 non-null   int64
9   stress_level                        5500 non-null   int64
10  physical_activity_hours_per_week     5500 non-null   float64
11  sleep_hours                         5500 non-null   float64
12  family_history_heart_disease         5500 non-null   object
13  diet_quality_score                   5500 non-null   int64
14  alcohol_units_per_week               5500 non-null   float64
15  heart_disease_risk_score             5500 non-null   float64
16  risk_category                       5500 non-null   object
dtypes: float64(7), int64(7), object(3)
memory usage: 730.6+ KB
```

7.1 Target Variable Exploration

[26]: `fig, axes = plt.subplots(1, 2, figsize=(16, 5))`

```
# Plot 1: Countplot risk category
sns.countplot(
    data=df_clean,
```

```

x='risk_category',
order=['Low', 'Medium', 'High'],
ax=axes[0]
)

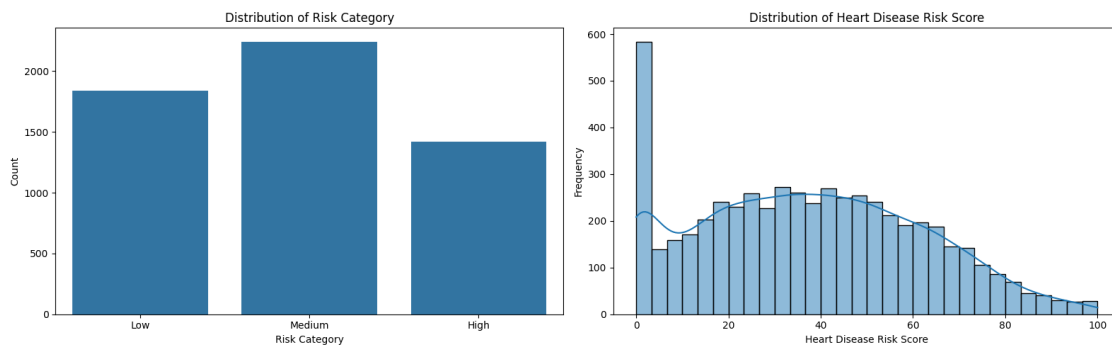
axes[0].set_title('Distribution of Risk Category')
axes[0].set_xlabel('Risk Category')
axes[0].set_ylabel('Count')

# Plot 2: Histogram risk score
sns.histplot(
    data=df_clean,
    x='heart_disease_risk_score',
    bins=30,
    kde=True,
    ax=axes[1]
)

axes[1].set_title('Distribution of Heart Disease Risk Score')
axes[1].set_xlabel('Heart Disease Risk Score')
axes[1].set_ylabel('Frequency')

plt.tight_layout()
plt.show()

```



- Kategori **Medium Risk** memiliki jumlah pasien paling banyak.
- Kategori **Low Risk** berada di posisi kedua.
- Kategori **High Risk** memiliki jumlah pasien paling sedikit.
- Hal ini menunjukkan bahwa mayoritas pasien berada pada tingkat risiko sedang.
- Kelompok **Medium Risk** perlu diperhatikan karena berpotensi meningkat menjadi **High Risk**.
- Distribusi **heart disease risk score** berada pada rentang sekitar 0 sampai 100.
- Sebagian besar skor berada pada kategori rendah hingga sedang.
- Frekuensi pasien semakin menurun pada skor risiko yang tinggi.
- Pasien dengan skor risiko tinggi tetap perlu menjadi prioritas analisis dan pencegahan.

```

[27]: import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.ticker import PercentFormatter

risk_order = ['Low', 'Medium', 'High']

risk_counts = df_clean['risk_category'].value_counts().reindex(risk_order)
risk_pct = (risk_counts / risk_counts.sum() * 100).round(2)

fig, axes = plt.subplots(1, 2, figsize=(16, 5))

sns.countplot(
    data=df_clean,
    x='risk_category',
    hue='risk_category',
    order=risk_order,
    palette='Set2',
    legend=False,
    ax=axes[0]
)

axes[0].set_title('Risk Category Distribution (Count)')
axes[0].set_xlabel('Risk Category')
axes[0].set_ylabel('Patients')

for container in axes[0].containers:
    axes[0].bar_label(container)

axes[1].bar(
    risk_order,
    risk_pct.values,
    color=sns.color_palette('Set2', 3)
)

axes[1].set_title('Risk Category Distribution (Percent)')
axes[1].set_xlabel('Risk Category')
axes[1].set_ylabel('Percent of Cohort')

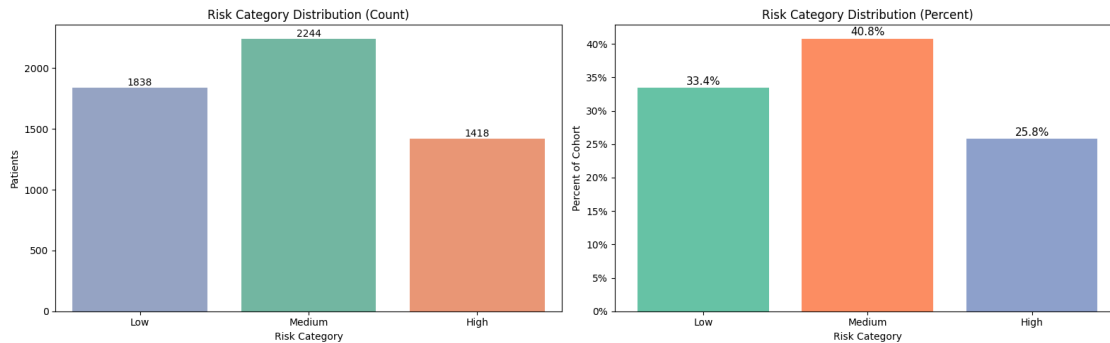
axes[1].yaxis.set_major_formatter(PercentFormatter(decimals=0))

for i, p in enumerate(risk_pct.values):
    axes[1].text(
        i,
        p + 0.5,
        f'{p:.1f}%',
        ha='center',
        fontsize=11
    )

```

```
)

plt.tight_layout()
plt.show()
```



- Kategori **Medium Risk** memiliki jumlah pasien terbanyak, yaitu **2.244 pasien** atau **40,8%**.
- Kategori **Low Risk** berada di posisi kedua dengan **1.838 pasien** atau **33,4%**.
- Kategori **High Risk** memiliki jumlah paling sedikit, yaitu **1.418 pasien** atau **25,8%**.
- Mayoritas pasien dalam dataset berada pada tingkat risiko **sedang**.
- Meskipun jumlah **High Risk** paling kecil, persentasenya tetap cukup besar sehingga perlu menjadi perhatian dalam analisis lanjutan.
- Kelompok **Medium Risk** juga penting diperhatikan karena berpotensi meningkat menjadi **High Risk** apabila faktor risiko tidak dikendalikan.

7.2 Distribusi Variabel Numerik

```
[28]: df_clean.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5500 entries, 0 to 5499
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Patient_ID                           5500 non-null   int64
1   age                                   5500 non-null   int64
2   bmi                                   5500 non-null   float64
3   systolic_bp                          5500 non-null   int64
4   diastolic_bp                         5500 non-null   float64
5   cholesterol_mg_dl                   5500 non-null   int64
6   resting_heart_rate                   5500 non-null   float64
7   smoking_status                       5500 non-null   object
8   daily_steps                          5500 non-null   int64
9   stress_level                         5500 non-null   int64
10  physical_activity_hours_per_week     5500 non-null   float64
11  sleep_hours                          5500 non-null   float64
```

```

12 family_history_heart_disease      5500 non-null    object
13 diet_quality_score                5500 non-null    int64
14 alcohol_units_per_week            5500 non-null    float64
15 heart_disease_risk_score           5500 non-null    float64
16 risk_category                     5500 non-null    object
dtypes: float64(7), int64(7), object(3)
memory usage: 730.6+ KB

```

```

[29]: kolom_numerik = [
    'age',
    'bmi',
    'systolic_bp',
    'diastolic_bp',
    'cholesterol_mg_dl',
    'resting_heart_rate',
    'daily_steps',
    'physical_activity_hours_per_week',
    'sleep_hours',
    'alcohol_units_per_week'
]

fig, axes = plt.subplots(4, 3, figsize=(16, 22))
axes = axes.flatten()

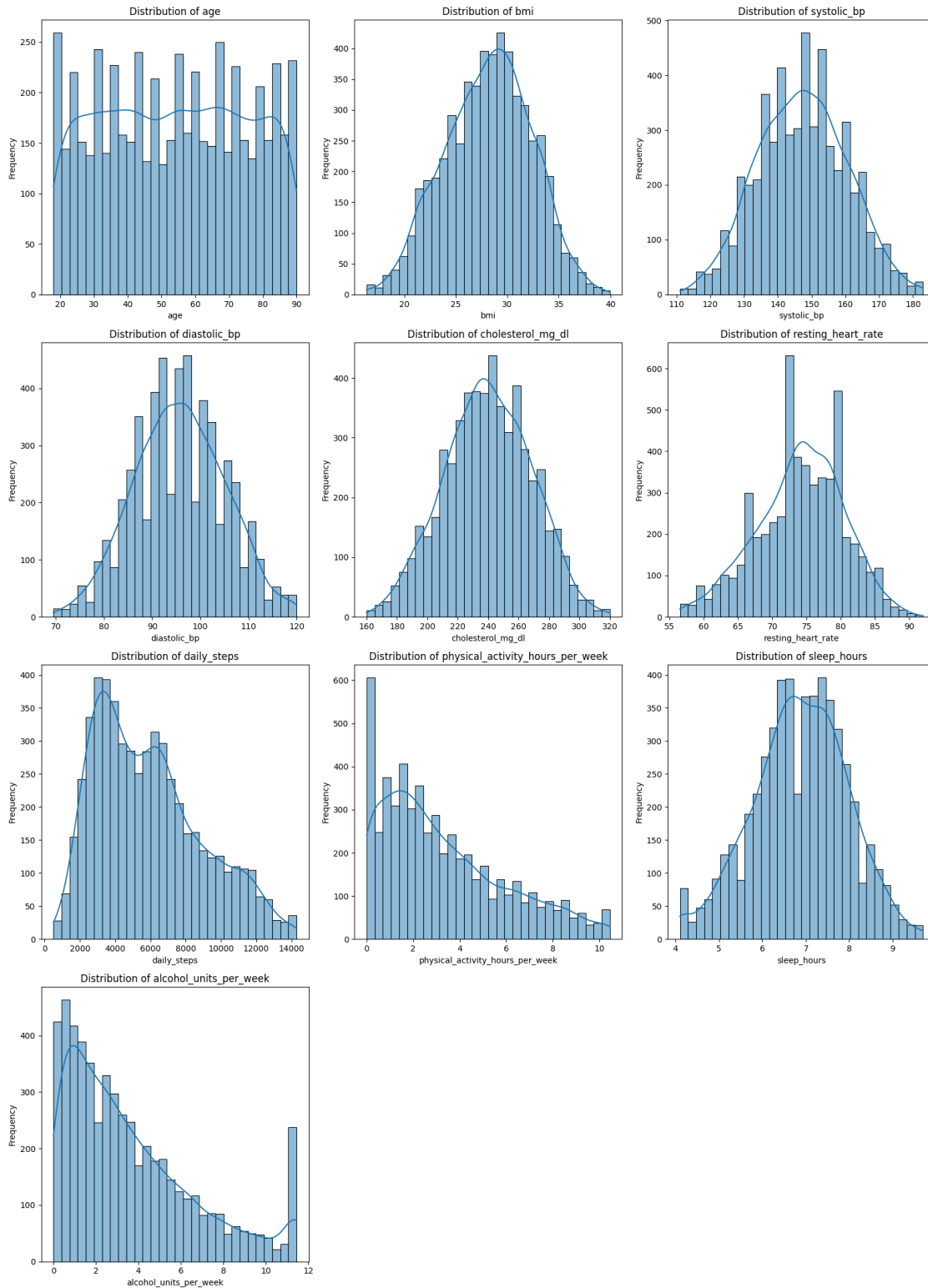
for i, col in enumerate(kolom_numerik):
    sns.histplot(
        data=df_clean,
        x=col,
        bins=30,
        kde=True,
        ax=axes[i]
    )

    axes[i].set_title(f'Distribution of {col}')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Frequency')

# Hapus subplot kosong
for j in range(len(kolom_numerik), len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()

```



- Variabel **age** tersebar cukup merata dari usia muda hingga usia lanjut.

- Variabel `bmi` cenderung membentuk distribusi normal dengan mayoritas nilai berada di sekitar 25–32.
- Variabel `systolic_bp` dan `diastolic_bp` juga cenderung normal, dengan sebagian besar pasien berada pada tekanan darah menengah.
- Variabel `cholesterol_mg_dl` memiliki pola distribusi cukup normal, dengan nilai terbanyak berada di sekitar 220–270 mg/dL.
- Variabel `resting_heart_rate` paling banyak berada pada kisaran 70–80 bpm.
- Variabel `daily_steps` cenderung condong ke kanan, artinya sebagian besar pasien memiliki jumlah langkah harian rendah hingga sedang.
- Variabel `physical_activity_hours_per_week` juga condong ke kanan, menunjukkan banyak pasien memiliki aktivitas fisik mingguan yang rendah.
- Variabel `sleep_hours` cenderung normal, dengan mayoritas pasien tidur sekitar 6–8 jam.
- Variabel `alcohol_units_per_week` condong ke kanan, menunjukkan sebagian besar pasien mengonsumsi alkohol dalam jumlah rendah.
- Secara umum, sebagian variabel klinis memiliki distribusi mendekati normal, sedangkan variabel gaya hidup lebih bervariasi dan cenderung tidak merata.

7.3 Distribusi Varibel Katagorik

```
[30]: kolom_kategorik = [
    'smoking_status',
    'stress_level',
    'family_history_heart_disease',
    'diet_quality_score',
    'risk_category'
]

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

for i, col in enumerate(kolom_kategorik):
    if col == 'risk_category':
        order = ['Low', 'Medium', 'High']
    else:
        order = sorted(df_clean[col].unique())

    sns.countplot(
        data=df_clean,
        x=col,
        order=order,
        ax=axes[i]
    )

    axes[i].set_title(f'Distribution of {col}')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Count')
    axes[i].tick_params(axis='x', rotation=30)
```

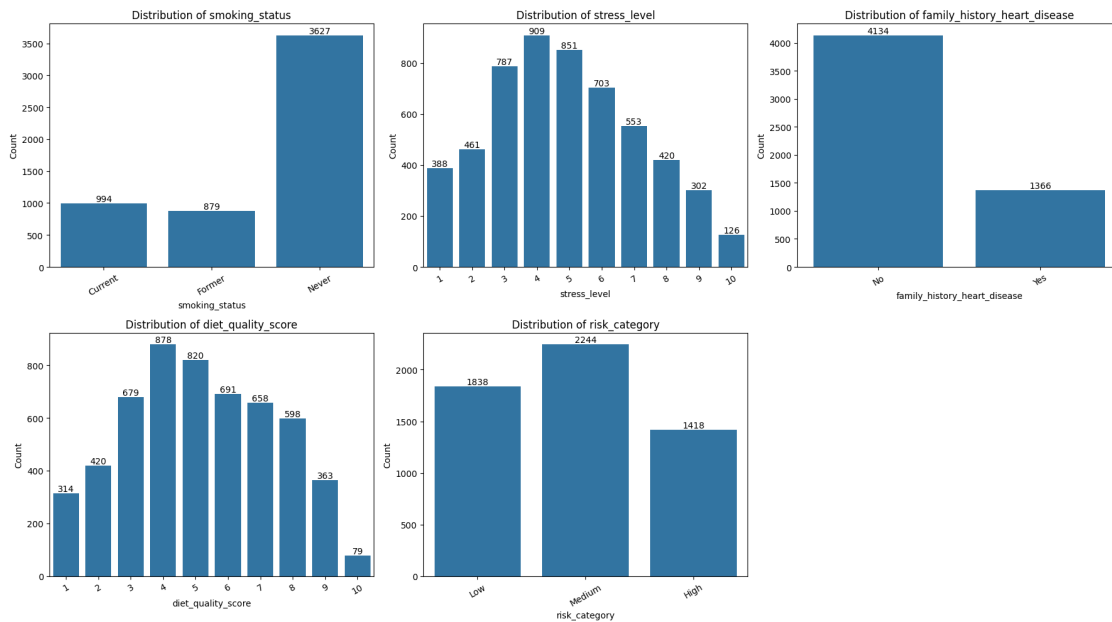
```

for container in axes[i].containers:
    axes[i].bar_label(container)

# Hapus subplot kosong
for j in range(len(kolom_kategorik), len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()

```



- Mayoritas pasien memiliki status merokok **Never**, yaitu sebanyak **3.627 pasien**.
- Jumlah pasien **Current smoker** sebanyak **994 pasien**, sedangkan **Former smoker** sebanyak **879 pasien**.
- Tingkat stres paling banyak berada pada level **4** dan **5**, sehingga mayoritas pasien memiliki tingkat stres sedang.
- Sebagian besar pasien tidak memiliki riwayat keluarga penyakit jantung, yaitu **4.134 pasien**.
- Pasien yang memiliki riwayat keluarga penyakit jantung berjumlah **1.366 pasien**.
- Skor kualitas diet paling banyak berada pada nilai **4** dan **5**, menunjukkan kualitas diet pasien cenderung sedang.
- Kategori risiko terbanyak adalah **Medium Risk** dengan **2.244 pasien**.
- Kategori **Low Risk** berjumlah **1.838 pasien**, sedangkan **High Risk** berjumlah **1.418 pasien**.
- Secara umum, dataset didominasi oleh pasien yang tidak merokok, tidak memiliki riwayat keluarga penyakit jantung, dan berada pada kategori risiko sedang.

7.4 Korelasi variabel numerik

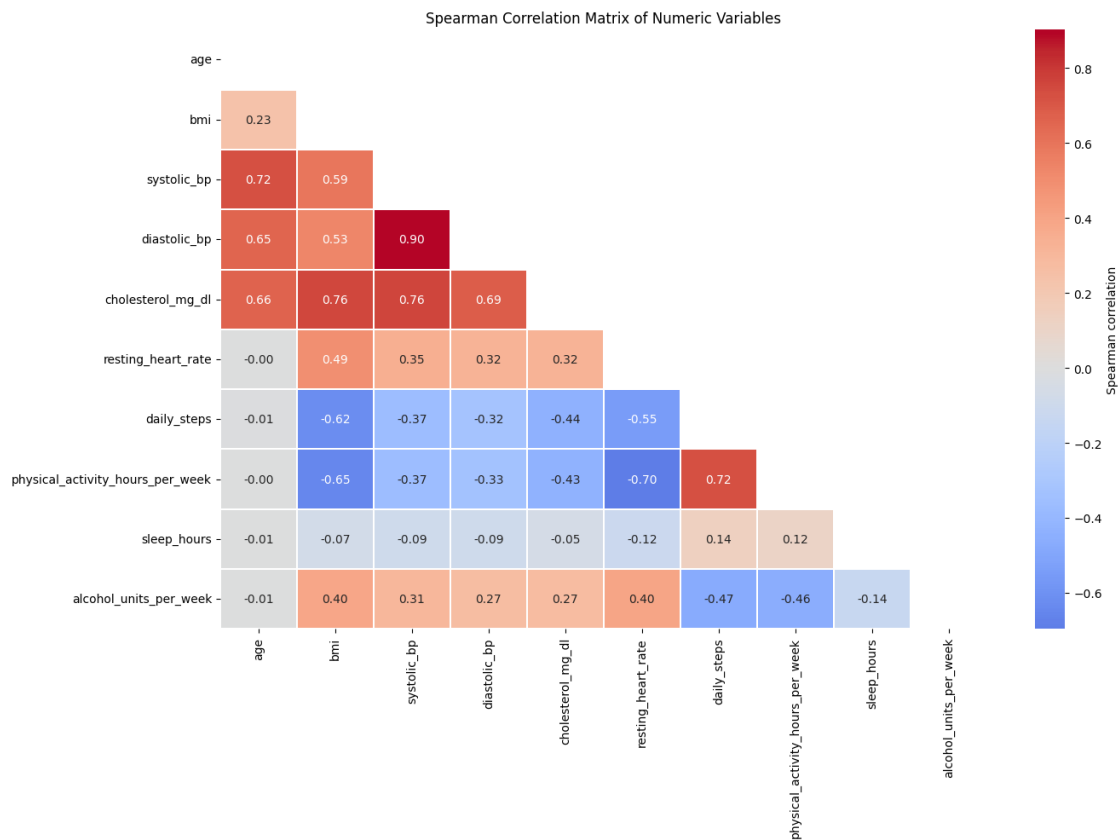
```
[31]: corr = df_clean[kolom_numerik].corr(method='spearman')

plt.figure(figsize=(14, 10))

mask = np.triu(np.ones_like(corr, dtype=bool))

sns.heatmap(
    corr,
    mask=mask,
    cmap='coolwarm',
    center=0,
    annot=True,
    fmt='.2f',
    linewidths=0.3,
    cbar_kws={'label': 'Spearman correlation'}
)

plt.title('Spearman Correlation Matrix of Numeric Variables')
plt.tight_layout()
plt.show()
```



- Korelasi terkuat terdapat antara `systolic_bp` dan `diastolic_bp` sebesar **0,90**, menunjukkan hubungan positif yang sangat kuat.
- `cholesterol_mg_dl` memiliki korelasi positif cukup kuat dengan `bmi`, `systolic_bp`, dan `diastolic_bp`.
- `age` berkorelasi positif dengan tekanan darah dan kolesterol, artinya semakin tinggi usia, nilai tekanan darah dan kolesterol cenderung meningkat.
- `daily_steps` dan `physical_activity_hours_per_week` memiliki korelasi positif sebesar **0,72**, menunjukkan bahwa pasien yang lebih aktif cenderung memiliki jumlah langkah harian lebih tinggi.
- `physical_activity_hours_per_week` berkorelasi negatif dengan `resting_heart_rate` sebesar **-0,70**, artinya semakin tinggi aktivitas fisik, detak jantung istirahat cenderung lebih rendah.
- `bmi` memiliki korelasi negatif dengan `daily_steps` dan `physical_activity_hours_per_week`, menunjukkan bahwa pasien dengan BMI lebih tinggi cenderung memiliki aktivitas fisik lebih rendah.
- `alcohol_units_per_week` berkorelasi negatif dengan `daily_steps` dan aktivitas fisik, sehingga konsumsi alkohol yang lebih tinggi cenderung berkaitan dengan gaya hidup yang kurang aktif.
- `sleep_hours` memiliki korelasi yang lemah dengan sebagian besar variabel, sehingga hubungannya tidak terlalu kuat dalam dataset ini.
- Secara umum, variabel klinis seperti tekanan darah, BMI, dan kolesterol saling berkorelasi positif, sedangkan variabel aktivitas fisik cenderung berkorelasi negatif dengan faktor risiko kesehatan.

7.4.1 Korelasi Variabel Numerik dengan Target Numerik

```
[32]: target = 'heart_disease_risk_score'

corr_target = (
    df_clean[kolom_numerik + [target]]
    .corr(method='spearman')[target]
    .drop(target)
    .sort_values(ascending=False)
)

corr_target

plt.figure(figsize=(10, 6))

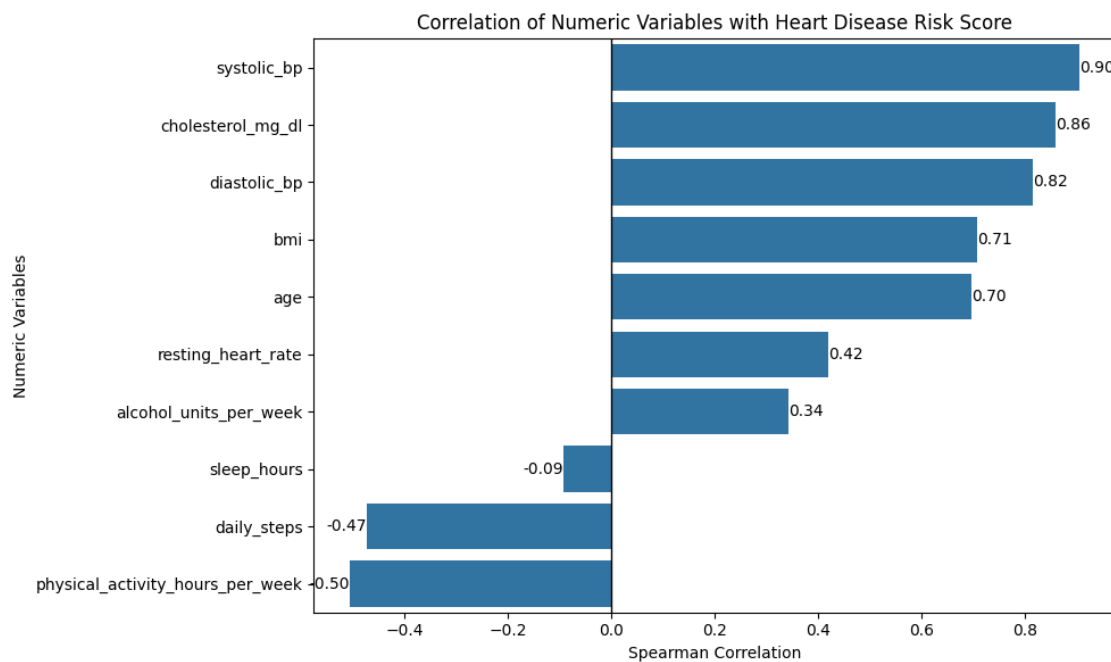
sns.barplot(
    x=corr_target.values,
    y=corr_target.index
)

plt.axvline(0, color='black', linewidth=1)
```

```
plt.title('Correlation of Numeric Variables with Heart Disease Risk Score')
plt.xlabel('Spearman Correlation')
plt.ylabel('Numeric Variables')

for i, value in enumerate(corr_target.values):
    plt.text(
        value,
        i,
        f'{value:.2f}',
        va='center',
        ha='left' if value >= 0 else 'right'
    )

plt.tight_layout()
plt.show()
```



- systolic_bp memiliki korelasi positif paling kuat dengan skor risiko penyakit jantung, yaitu **0,90**.
- cholesterol_mg_dl juga memiliki korelasi positif sangat kuat sebesar **0,86**.
- diastolic_bp berkorelasi positif kuat dengan nilai **0,82**.
- bmi dan age juga memiliki hubungan positif yang cukup kuat dengan skor risiko.
- Artinya, semakin tinggi tekanan darah, kolesterol, BMI, dan usia, maka skor risiko penyakit jantung cenderung meningkat.
- resting_heart_rate dan alcohol_units_per_week memiliki korelasi positif sedang hingga lemah.
- physical_activity_hours_per_week memiliki korelasi negatif sebesar **-0,50**.

- `daily_steps` juga memiliki korelasi negatif sebesar **-0,47**.
- Artinya, semakin tinggi aktivitas fisik dan jumlah langkah harian, maka skor risiko penyakit jantung cenderung menurun.
- `sleep_hours` memiliki korelasi negatif sangat lemah, sehingga hubungannya dengan skor risiko tidak terlalu kuat.

```
[33]: RANDOM_STATE = 42

n_cols = 3
n_rows = int(np.ceil(len(kolom_numerik) / n_cols))

fig, axes = plt.subplots(n_rows, n_cols, figsize=(18, 4.5 * n_rows))
axes = axes.flatten()

sample_df = df_clean.sample(min(len(df_clean), 2500), random_state=RANDOM_STATE)

for i, col in enumerate(kolom_numerik):
    sns.scatterplot(
        data=sample_df,
        x=col,
        y='heart_disease_risk_score',
        alpha=0.3,
        s=25,
        color='#7570b3',
        ax=axes[i]
    )

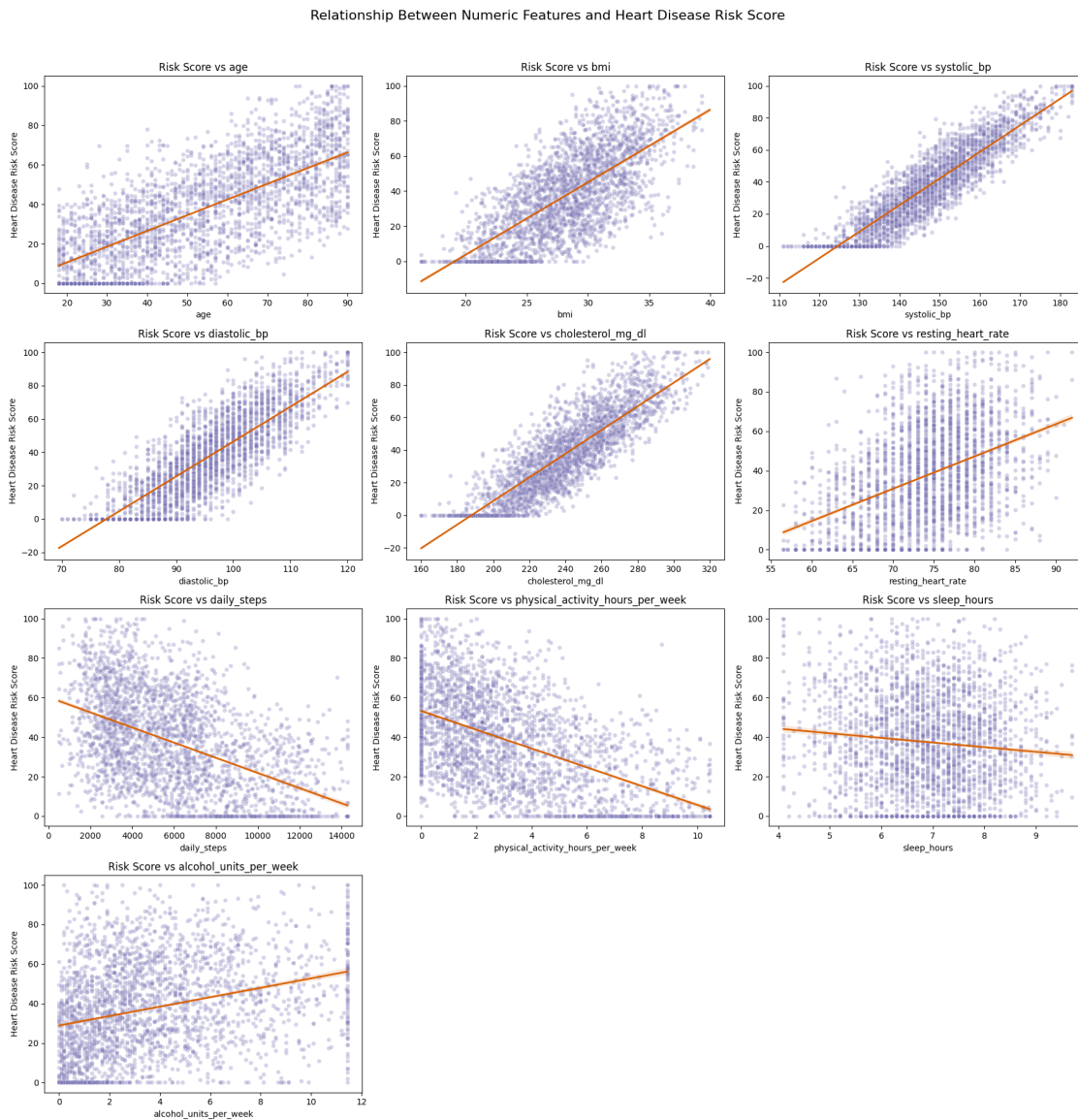
    sns.regplot(
        data=df_clean,
        x=col,
        y='heart_disease_risk_score',
        scatter=False,
        color='#d95f02',
        line_kws={'linewidth': 2},
        ax=axes[i]
    )

    axes[i].set_title(f'Risk Score vs {col}')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Heart Disease Risk Score')

for j in range(len(kolom_numerik), len(axes)):
    fig.delaxes(axes[j])

plt.suptitle('Relationship Between Numeric Features and Heart Disease Risk_↵
↵Score', y=1.02, fontsize=16)
plt.tight_layout()
```

```
plt.show()
```



- **age** memiliki hubungan positif dengan skor risiko, artinya semakin tinggi usia maka risiko penyakit jantung cenderung meningkat.
- **bmi** juga menunjukkan hubungan positif, sehingga BMI yang lebih tinggi cenderung berkaitan dengan skor risiko yang lebih besar.
- **systolic_bp** dan **diastolic_bp** memiliki hubungan positif yang kuat dengan skor risiko.
- **cholesterol_mg_dl** juga menunjukkan pola positif yang jelas terhadap peningkatan risiko penyakit jantung.
- **resting_heart_rate** memiliki hubungan positif, tetapi tidak sekuat tekanan darah dan kolesterol.
- **daily_steps** memiliki hubungan negatif, artinya semakin banyak langkah harian maka skor

risiko cenderung lebih rendah.

- `physical_activity_hours_per_week` juga memiliki hubungan negatif dengan skor risiko.
- `sleep_hours` menunjukkan hubungan negatif yang lemah, sehingga pengaruhnya tidak terlalu kuat.
- `alcohol_units_per_week` memiliki hubungan positif, artinya konsumsi alkohol yang lebih tinggi cenderung berkaitan dengan peningkatan skor risiko.
- Secara umum, faktor klinis seperti tekanan darah, kolesterol, BMI, dan usia cenderung meningkatkan risiko, sedangkan aktivitas fisik cenderung menurunkan risiko.

7.4.2 Korelasi Varibel Numerik dengan Target Katagorik

```
[34]: target = 'risk_category'

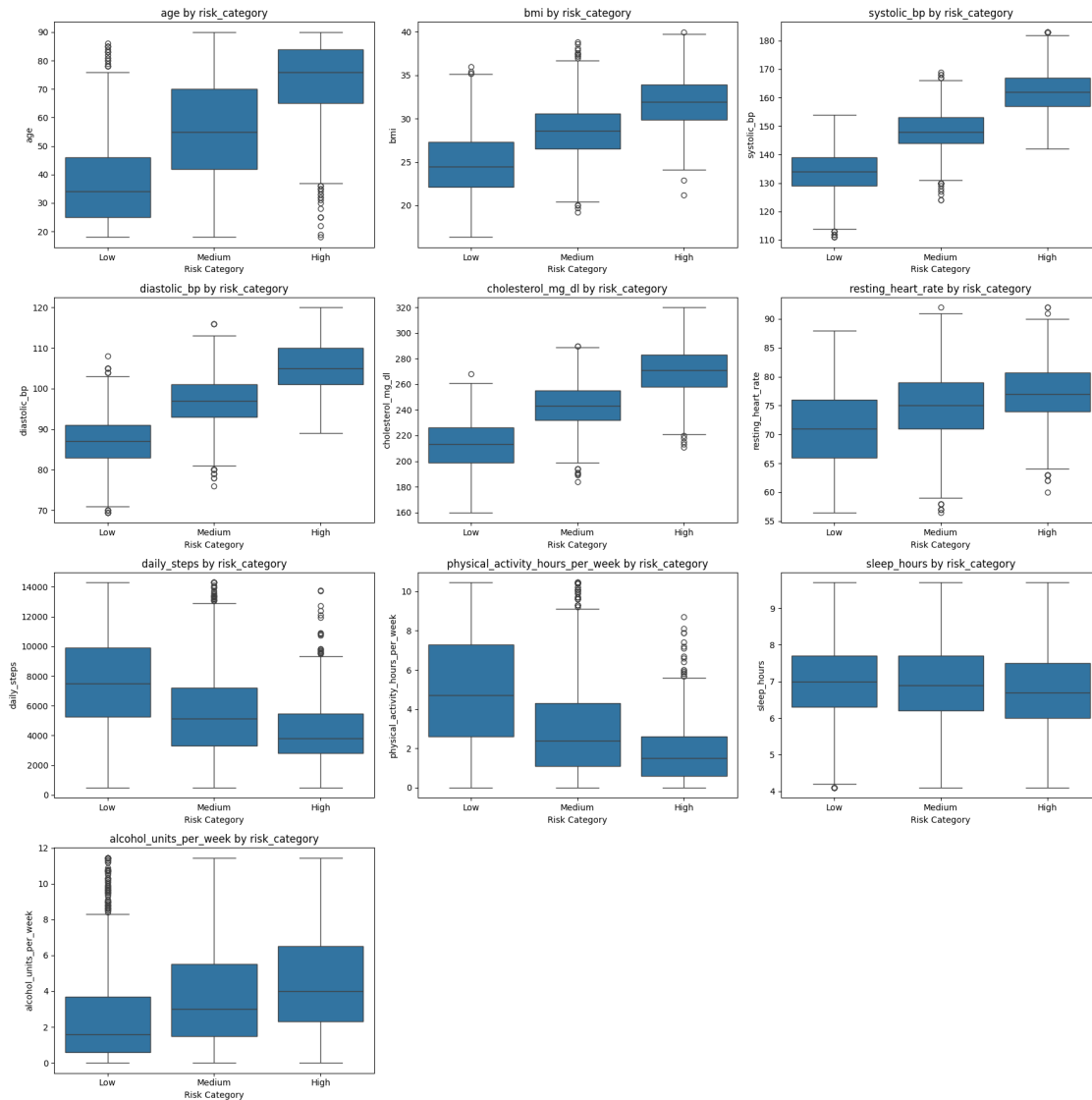
fig, axes = plt.subplots(4, 3, figsize=(18, 18))
axes = axes.flatten()

for i, col in enumerate(kolom_numerik):
    sns.boxplot(
        data=df_clean,
        x=target,
        y=col,
        order=['Low', 'Medium', 'High'],
        ax=axes[i]
    )

    axes[i].set_title(f'{col} by {target}')
    axes[i].set_xlabel('Risk Category')
    axes[i].set_ylabel(col)

# Hapus subplot kosong
for j in range(len(kolom_numerik), len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()
```



- Pasien kategori **High Risk** cenderung memiliki usia lebih tinggi dibandingkan kategori **Low** dan **Medium**.
- Nilai **bmi** meningkat dari kategori **Low**, **Medium**, hingga **High Risk**.
- Tekanan darah **systolic_bp** dan **diastolic_bp** terlihat semakin tinggi pada kategori risiko yang lebih tinggi.
- Kadar **cholesterol_mg_dl** juga meningkat seiring naiknya kategori risiko.
- **resting_heart_rate** pada kategori **High Risk** cenderung lebih tinggi dibandingkan kategori lainnya.
- **daily_steps** semakin rendah pada kategori risiko yang lebih tinggi.
- **physical_activity_hours_per_week** juga menurun dari **Low Risk** ke **High Risk**.
- **sleep_hours** terlihat relatif mirip antar kategori, sehingga perbedaannya tidak terlalu kuat.
- **alcohol_units_per_week** cenderung lebih tinggi pada kategori **High Risk**.
- Secara umum, pasien **High Risk** memiliki faktor klinis yang lebih buruk, seperti usia, BMI,

tekanan darah, kolesterol, dan detak jantung yang lebih tinggi.

- Sebaliknya, pasien **Low Risk** cenderung memiliki aktivitas fisik dan jumlah langkah harian yang lebih tinggi.

7.5 Korelasi Varibel Katagorik

```
[35]: from scipy.stats import chi2_contingency

kolom_kategorik = [
    'smoking_status',
    'stress_level',
    'family_history_heart_disease',
    'diet_quality_score',
    'risk_category'
]

def cramers_v(x, y):
    contingency_table = pd.crosstab(x, y)
    chi2 = chi2_contingency(contingency_table)[0]
    n = contingency_table.sum().sum()
    r, k = contingency_table.shape

    return np.sqrt(chi2 / (n * (min(r, k) - 1)))

categorical_corr_results = []

for i in range(len(kolom_kategorik)):
    for j in range(i + 1, len(kolom_kategorik)):
        col1 = kolom_kategorik[i]
        col2 = kolom_kategorik[j]

        score = cramers_v(df_clean[col1], df_clean[col2])

        categorical_corr_results.append({
            'Variable Pair': f'{col1} vs {col2}',
            "Cramer's V": round(score, 3)
        })

categorical_corr_df = pd.DataFrame(categorical_corr_results)
categorical_corr_df = categorical_corr_df.sort_values(
    by="Cramer's V",
    ascending=False
)

display(categorical_corr_df)
```

	Variable Pair	Cramer's V
9	diet_quality_score vs risk_category	0.355

3	smoking_status vs risk_category	0.276
6	stress_level vs risk_category	0.235
2	smoking_status vs diet_quality_score	0.175
0	smoking_status vs stress_level	0.162
8	family_history_heart_disease vs risk_category	0.140
5	stress_level vs diet_quality_score	0.135
4	stress_level vs family_history_heart_disease	0.036
7	family_history_heart_disease vs diet_quality_s...	0.035
1	smoking_status vs family_history_heart_disease	0.008

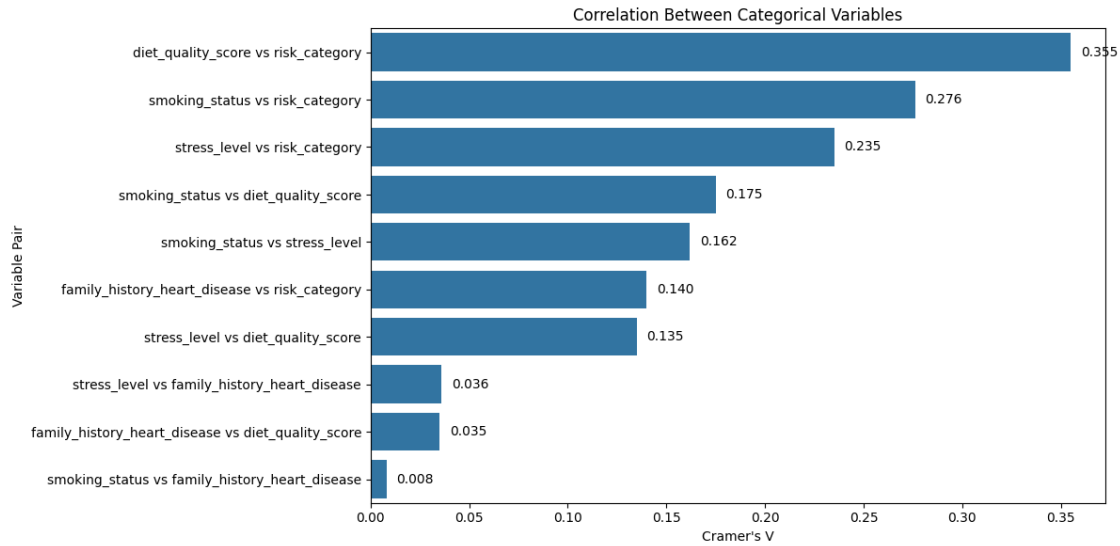
```
[36]: plt.figure(figsize=(12, 6))

sns.barplot(
    data=categorical_corr_df,
    x="Cramer's V",
    y='Variable Pair'
)

plt.title("Correlation Between Categorical Variables")
plt.xlabel("Cramer's V")
plt.ylabel("Variable Pair")

for i, value in enumerate(categorical_corr_df["Cramer's V"]):
    plt.text(
        value + 0.005,
        i,
        f'{value:.3f}',
        va='center'
    )

plt.tight_layout()
plt.show()
```



- Hubungan terkuat terdapat antara `diet_quality_score` dan `risk_category` dengan nilai **Cramer's V = 0,355**.
- Artinya, kualitas diet memiliki hubungan paling besar dengan kategori risiko dibandingkan variabel kategorik lainnya.
- `smoking_status` juga memiliki hubungan dengan `risk_category` sebesar **0,276**.
- `stress_level` memiliki hubungan dengan `risk_category` sebesar **0,235**.
- Hal ini menunjukkan bahwa pola makan, status merokok, dan tingkat stres cukup berkaitan dengan kategori risiko kardiovaskular.
- `family_history_heart_disease` memiliki hubungan yang lebih lemah dengan `risk_category`, yaitu **0,140**.
- Beberapa pasangan variabel memiliki nilai korelasi sangat kecil, seperti `smoking_status` dengan `family_history_heart_disease` sebesar **0,008**.
- Secara umum, variabel kategorik yang paling berpengaruh terhadap kategori risiko adalah **kualitas diet, status merokok, dan tingkat stres**.

```
[57]: import math
```

```
[58]: # Buat semua pasangan variabel kategorik
pairs = []

for i in range(len(kolom_kategorik)):
    for j in range(i + 1, len(kolom_kategorik)):
        pairs.append((kolom_kategorik[i], kolom_kategorik[j]))

n_cols = 3
n_rows = math.ceil(len(pairs) / n_cols)

fig, axes = plt.subplots(n_rows, n_cols, figsize=(21, 5 * n_rows))
axes = axes.flatten()
```

```

for idx, (col1, col2) in enumerate(pairs):
    crosstab_pct = pd.crosstab(
        df_clean[col1],
        df_clean[col2],
        normalize='index'
    ) * 100

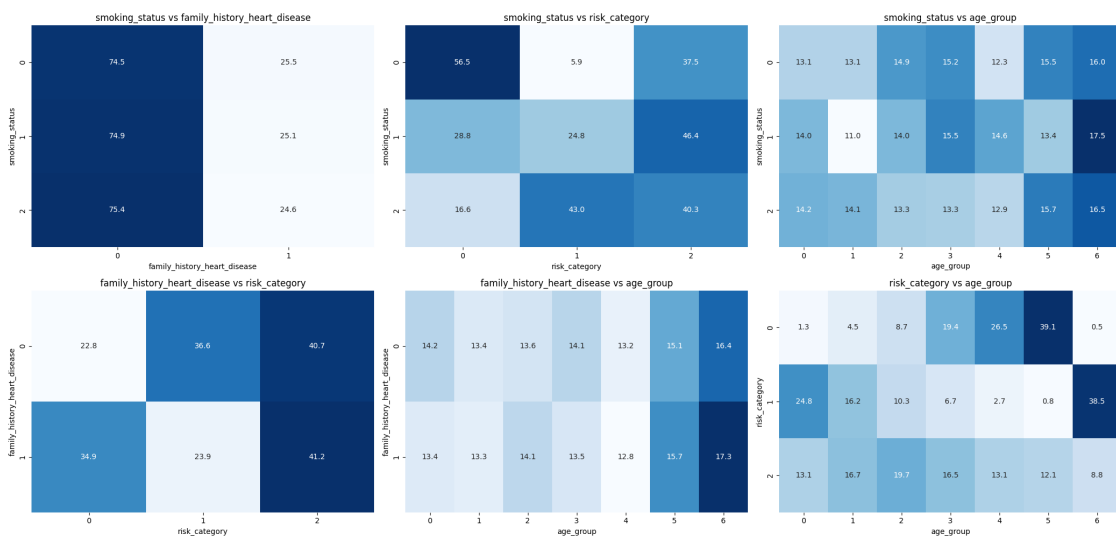
    sns.heatmap(
        crosstab_pct,
        annot=True,
        fmt='.1f',
        cmap='Blues',
        ax=axes[idx],
        cbar=False
    )

    axes[idx].set_title(f'{col1} vs {col2}')
    axes[idx].set_xlabel(col2)
    axes[idx].set_ylabel(col1)

# Hapus subplot kosong
for j in range(len(pairs), len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()

```



- smoking_status dan family_history_heart_disease memiliki pola yang hampir sama di setiap kategori.

- Hal ini menunjukkan bahwa status merokok tidak terlalu berkaitan dengan riwayat keluarga penyakit jantung.
- `smoking_status` terlihat memiliki hubungan dengan `risk_category`.
- Kelompok perokok cenderung memiliki proporsi risiko yang lebih tinggi dibandingkan kelompok tidak merokok.
- `family_history_heart_disease` juga berhubungan dengan `risk_category`.
- Pasien dengan riwayat keluarga penyakit jantung cenderung memiliki proporsi risiko tinggi yang lebih besar.
- `risk_category` memiliki pola yang berbeda pada setiap `age_group`.
- Kelompok usia tertentu memiliki proporsi risiko yang lebih tinggi dibandingkan kelompok usia lainnya.
- Secara umum, variabel yang paling terlihat berkaitan dengan kategori risiko adalah `smoking_status`, `family_history_heart_disease`, dan `age_group`.
- Pastikan angka kategori seperti 0, 1, dan 2 dijelaskan menggunakan keterangan label agar interpretasi lebih mudah dipahami.

7.5.1 Korelasi Varibel Katagorik dengan Target Katagorik

```
[59]: kolom_kategorik = [
        'smoking_status',
        'stress_level',
        'family_history_heart_disease',
        'diet_quality_score'
    ]

    target = 'risk_category'
```

```
[60]: import pandas as pd
import numpy as np
from scipy.stats import chi2_contingency
import matplotlib.pyplot as plt
import seaborn as sns

kolom_kategorik = [
    'smoking_status',
    'stress_level',
    'family_history_heart_disease',
    'diet_quality_score'
]

target = 'risk_category'

def cramers_v(x, y):
    confusion_matrix = pd.crosstab(x, y)
    chi2 = chi2_contingency(confusion_matrix)[0]
    n = confusion_matrix.sum().sum()
    r, k = confusion_matrix.shape
```

```

        return np.sqrt(chi2 / (n * (min(r, k) - 1)))

cramers_results = []

for col in kolom_kategorik:
    score = cramers_v(df_clean[col], df_clean[target])

    cramers_results.append({
        'Categorical Variable': col,
        "Cramer's V": round(score, 3)
    })

cramers_df = pd.DataFrame(cramers_results)
cramers_df = cramers_df.sort_values(by="Cramer's V", ascending=False)

display(cramers_df)

```

	Categorical Variable	Cramer's V
3	diet_quality_score	0.355
0	smoking_status	0.276
1	stress_level	0.235
2	family_history_heart_disease	0.140

```

[61]: plt.figure(figsize=(9, 5))

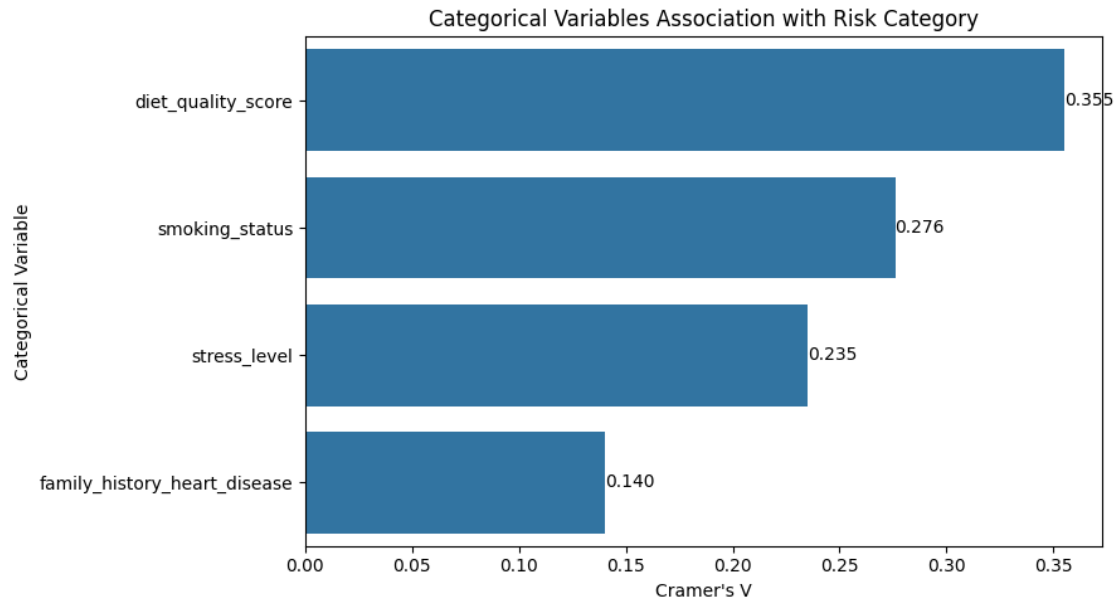
sns.barplot(
    data=cramers_df,
    x="Cramer's V",
    y='Categorical Variable'
)

plt.title("Categorical Variables Association with Risk Category")
plt.xlabel("Cramer's V")
plt.ylabel("Categorical Variable")

for i, value in enumerate(cramers_df["Cramer's V"]):
    plt.text(
        value,
        i,
        f'{value:.3f}',
        va='center',
        ha='left'
    )

plt.tight_layout()
plt.show()

```



7.5.2 Korelasi Varibel Katagorik dengan Target Numerik

```
[62]: kolom_kategorik = [
    'smoking_status',
    'stress_level',
    'family_history_heart_disease',
    'diet_quality_score'
]

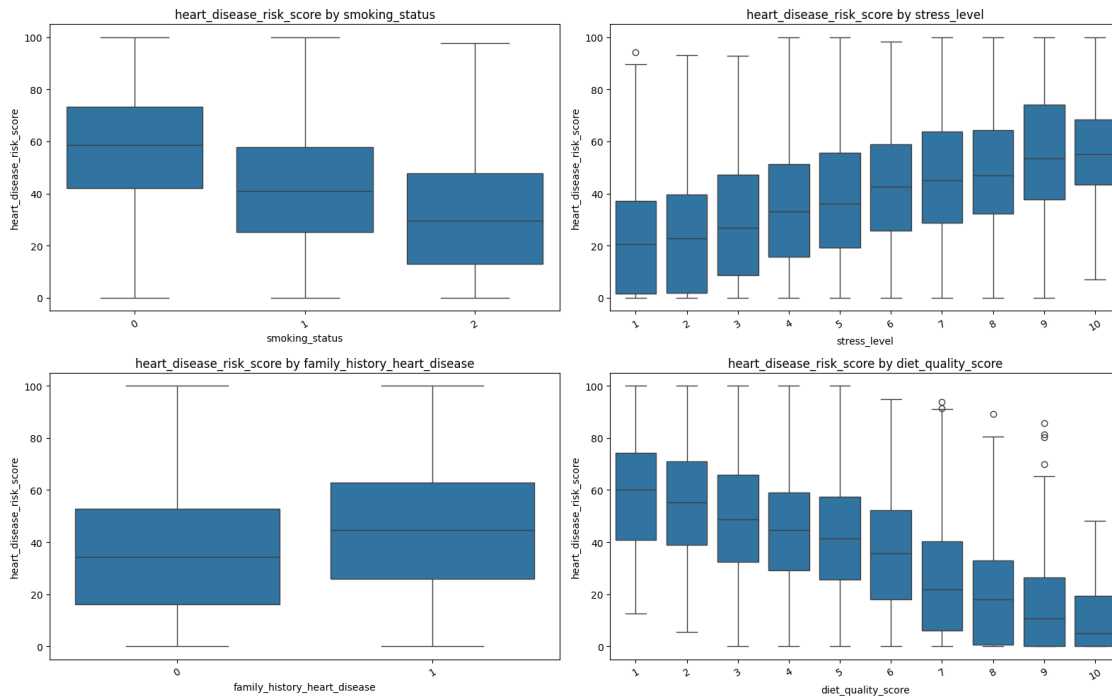
target = 'heart_disease_risk_score'

fig, axes = plt.subplots(2, 2, figsize=(16, 10))
axes = axes.flatten()

for i, col in enumerate(kolom_kategorik):
    sns.boxplot(
        data=df_clean,
        x=col,
        y=target,
        ax=axes[i]
    )

    axes[i].set_title(f'{target} by {col}')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel(target)
    axes[i].tick_params(axis='x', rotation=30)
```

```
plt.tight_layout()
plt.show()
```



- Variabel `diet_quality_score` memiliki hubungan paling kuat dengan `risk_category`, dengan nilai **Cramer's V = 0,355**.
- Hal ini menunjukkan bahwa kualitas diet cukup berkaitan dengan kategori risiko kardiovaskular.
- Variabel `smoking_status` memiliki hubungan sebesar **0,276**, sehingga status merokok juga cukup berhubungan dengan kategori risiko.
- Variabel `stress_level` memiliki hubungan sebesar **0,235**, menunjukkan bahwa tingkat stres juga berkaitan dengan risiko kardiovaskular.
- Variabel `family_history_heart_disease` memiliki hubungan paling lemah, yaitu **0,140**.
- Secara umum, variabel kategorik yang paling berhubungan dengan kategori risiko adalah **kualitas diet, status merokok, dan tingkat stres**.

8 EDA untuk Pertanyaan Bisnis

8.1 Business Questions

1. Berapa persentase pasien pada setiap kategori risiko kardiovaskular (Low, Medium, dan High) dalam dataset?
2. Kelompok usia mana yang memiliki proporsi pasien High Risk paling besar?
3. Bagaimana rata-rata tekanan darah, kadar kolesterol, dan `heart_disease_risk_score` pada setiap kategori risiko?

4. Bagaimana pola gaya hidup pasien berdasarkan kategori risiko, dilihat dari aktivitas fisik, jumlah langkah harian, durasi tidur, kualitas diet, dan konsumsi alkohol?
5. Bagaimana proporsi status merokok dan riwayat keluarga penyakit jantung pada setiap kategori risiko?
6. Fitur klinis dan gaya hidup apa yang paling menonjol pada pasien kategori High Risk berdasarkan hasil EDA?
7. Kelompok pasien mana yang perlu diprioritaskan dalam edukasi dan pencegahan penyakit jantung?
1. Berapa persentase pasien pada setiap kategori risiko kardiovaskular Low, Medium, dan High dalam dataset cardiovascular risk?
2. Kelompok usia mana yang memiliki proporsi pasien High Risk paling besar dalam dataset cardiovascular risk?
3. Kategori risiko mana yang memiliki rata-rata heart_disease_risk_score tertinggi dalam dataset cardiovascular risk?
4. Berapa rata-rata tekanan darah sistolik dan diastolik pada pasien kategori Low, Medium, dan High?
5. Berapa rata-rata kadar kolesterol pada setiap kategori risiko kardiovaskular?
6. Berapa rata-rata aktivitas fisik, jumlah langkah harian, durasi tidur, kualitas diet, dan konsumsi alkohol pada setiap kategori risiko?
7. Berapa proporsi pasien dengan status merokok Current, Former, dan Never pada setiap kategori risiko kardiovaskular?
8. Berapa proporsi pasien dengan riwayat keluarga penyakit jantung pada kategori Low, Medium, dan High?
9. Fitur klinis dan gaya hidup apa yang paling menonjol pada pasien dengan kategori High Risk berdasarkan hasil EDA?
10. Kelompok pasien mana yang perlu menjadi prioritas edukasi pencegahan berdasarkan kategori risiko, tekanan darah, kolesterol, aktivitas fisik, dan riwayat keluarga?
11. Seberapa besar pengaruh faktor gaya hidup terhadap peningkatan risiko penyakit jantung?
12. Bagaimana karakteristik individu dengan risiko tinggi dibandingkan dengan risiko rendah dan sedang?

8.1.1 1. Berapa persentase pasien pada setiap kategori risiko kardiovaskular Low, Medium, dan High dalam dataset cardiovascular risk?

```
[39]: risk_order = ['Low', 'Medium', 'High']

risk_counts = df_clean['risk_category'].value_counts().reindex(risk_order)
risk_percentage = (risk_counts / risk_counts.sum() * 100).round(2)

risk_distribution = pd.DataFrame({
```



```

    'Risk Category': risk_order,
    'Count': risk_counts.values,
    'Percentage (%)': risk_percentage.values
})

display(risk_distribution)

plt.figure(figsize=(8, 5))

sns.barplot(
    data=risk_distribution,
    x='Risk Category',
    y='Percentage (%)'
)

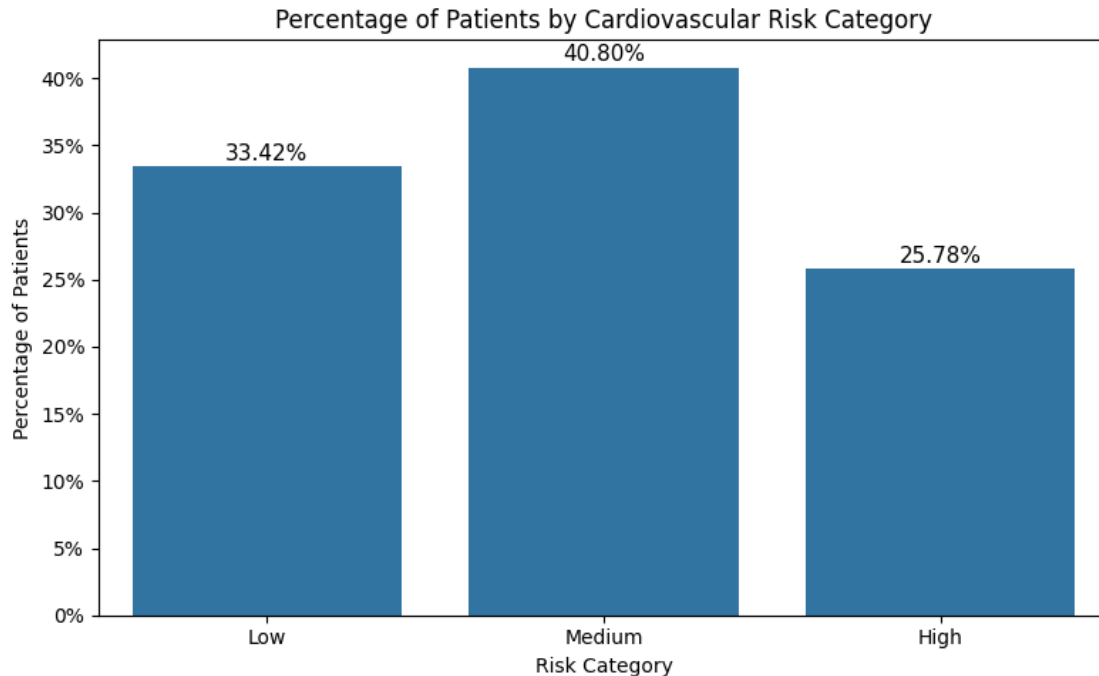
plt.title('Percentage of Patients by Cardiovascular Risk Category')
plt.xlabel('Risk Category')
plt.ylabel('Percentage of Patients')
plt.gca().yaxis.set_major_formatter(PercentFormatter(decimals=0))

for i, value in enumerate(risk_distribution['Percentage (%)']):
    plt.text(
        i,
        value + 0.5,
        f'{value:.2f}%',
        ha='center',
        fontsize=11
    )

plt.tight_layout()
plt.show()

```

	Risk Category	Count	Percentage (%)
0	Low	1838	33.42
1	Medium	2244	40.80
2	High	1418	25.78



Insight

- Kategori risiko terbanyak adalah **Medium Risk** sebesar **40.80%**.
- Kategori **Low Risk** sebesar **33.42%**.
- Kategori **High Risk** sebesar **25.78%**.
- Mayoritas pasien berada pada tingkat risiko **sedang**.
- Kelompok **High Risk** tetap perlu menjadi prioritas perhatian meskipun jumlahnya paling kecil.

8.1.2 2.Kelompok usia mana yang memiliki proporsi pasien High Risk paling besar dalam dataset cardiovascular risk?

```
[40]: # Membuat kelompok usia
df_clean['age_group'] = pd.cut(
    df_clean['age'],
    bins=[0, 30, 40, 50, 60, 70, 80, 100],
    labels=['<30', '30-39', '40-49', '50-59', '60-69', '70-79', '80+'],
    right=False
)

# Menghitung jumlah pasien High Risk per kelompok usia
high_risk_by_age = (
    df_clean[df_clean['risk_category'] == 'High']
    .groupby('age_group')
    .size()
)
```

```

# Menghitung total pasien per kelompok usia
total_by_age = df_clean.groupby('age_group').size()

# Menghitung proporsi High Risk per kelompok usia
high_risk_percentage = (high_risk_by_age / total_by_age * 100).fillna(0).
    ↪round(2)

# Membuat dataframe ringkasan
high_risk_age_df = pd.DataFrame({
    'Age Group': high_risk_percentage.index,
    'High Risk Percentage (%)': high_risk_percentage.values,
    'High Risk Count': high_risk_by_age.reindex(high_risk_percentage.index).
    ↪fillna(0).astype(int).values,
    'Total Patients': total_by_age.reindex(high_risk_percentage.index).values
})

display(high_risk_age_df)

# Visualisasi
plt.figure(figsize=(10, 5))

sns.barplot(
    data=high_risk_age_df,
    x='Age Group',
    y='High Risk Percentage (%)'
)

plt.title('High Risk Proportion by Age Group')
plt.xlabel('Age Group')
plt.ylabel('High Risk Percentage')
plt.gca().yaxis.set_major_formatter(PercentFormatter(decimals=0))

for i, value in enumerate(high_risk_age_df['High Risk Percentage (%)']):
    plt.text(
        i,
        value + 0.5,
        f'{value:.2f}%',
        ha='center',
        fontsize=10
    )

plt.tight_layout()
plt.show()

```

/tmp/ipykernel_10994/2149912408.py:12: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of

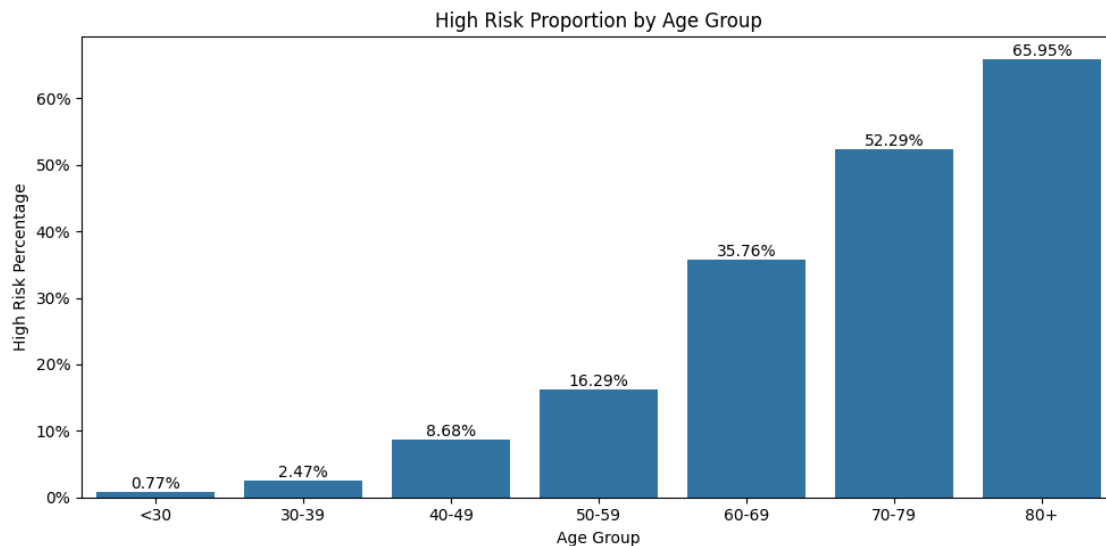
pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
.groupby('age_group')
```

/tmp/ipykernel_10994/2149912408.py:17: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
total_by_age = df_clean.groupby('age_group').size()
```

	Age Group	High Risk Percentage (%)	High Risk Count	Total Patients
0	<30	0.77	7	912
1	30-39	2.47	19	768
2	40-49	8.68	64	737
3	50-59	16.29	123	755
4	60-69	35.76	275	769
5	70-79	52.29	376	719
6	80+	65.95	554	840



Insight

- Proporsi **High Risk** paling besar terdapat pada kelompok usia **80+** sebesar **65.95%**.
- Kelompok usia **70–79** memiliki proporsi High Risk sebesar **52.29%**.
- Kelompok usia **60–69** mulai menunjukkan peningkatan signifikan, yaitu **35.76%**.
- Semakin tinggi kelompok usia, semakin besar proporsi pasien dengan kategori **High Risk**.
- Kelompok usia **60 tahun ke atas** perlu menjadi prioritas perhatian dalam pencegahan risiko kardiovaskular.

8.1.3 3. Bagaimana rata-rata tekanan darah, kadar kolesterol, dan heart_disease_risk_score pada setiap kategori risiko?

```
[41]: risk_order = ['Low', 'Medium', 'High']

avg_data = (
    df_clean
    .groupby('risk_category')[[
        'systolic_bp',
        'diastolic_bp',
        'cholesterol_mg_dl',
        'heart_disease_risk_score'
    ]]
    .mean()
    .reindex(risk_order)
    .round(2)
    .reset_index()
)

display(avg_data)

avg_data_melted = avg_data.melt(
    id_vars='risk_category',
    value_vars=[
        'systolic_bp',
        'diastolic_bp',
        'cholesterol_mg_dl',
        'heart_disease_risk_score'
    ],
    var_name='Health Indicator',
    value_name='Average Value'
)

plt.figure(figsize=(12, 6))

sns.barplot(
    data=avg_data_melted,
    x='risk_category',
    y='Average Value',
    hue='Health Indicator'
)

plt.title('Average Health Indicators by Risk Category')
plt.xlabel('Risk Category')
plt.ylabel('Average Value')

for container in plt.gca().containers:
```

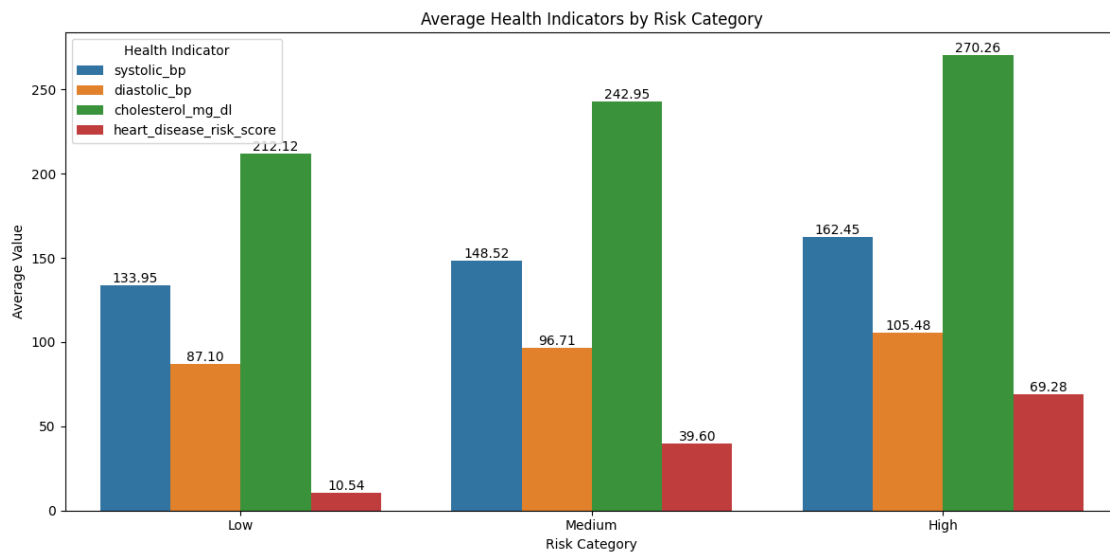
```
plt.gca().bar_label(container, fmt='%.2f')

plt.legend(title='Health Indicator')

plt.tight_layout()
plt.show()
```

	risk_category	systolic_bp	diastolic_bp	cholesterol_mg_dl	\
0	Low	133.95	87.10	212.12	
1	Medium	148.52	96.71	242.95	
2	High	162.45	105.48	270.26	

	heart_disease_risk_score
0	10.54
1	39.60
2	69.28



Insight

- Rata-rata `systolic_bp` meningkat dari Low (133.95), Medium (148.52), hingga High (162.45).
- Rata-rata `diastolic_bp` juga meningkat dari Low (87.10), Medium (96.71), hingga High (105.48).
- Rata-rata kolesterol meningkat dari Low (212.12 mg/dL), Medium (242.95 mg/dL), hingga High (270.26 mg/dL).
- Rata-rata `heart_disease_risk_score` meningkat dari Low (10.54), Medium (39.60), hingga High (69.28).
- Kategori High Risk memiliki rata-rata tekanan darah, kolesterol, dan skor risiko paling tinggi.
- Kategori Low Risk memiliki rata-rata tekanan darah, kolesterol, dan skor risiko paling rendah.

- Pola ini menunjukkan bahwa tekanan darah, kadar kolesterol, dan skor risiko penyakit jantung cenderung meningkat seiring meningkatnya kategori risiko kardiovaskular.
- Hasil ini juga menunjukkan bahwa `risk_category` sudah selaras dengan nilai `heart_disease_risk_score`.

8.1.4 4. Bagaimana pola gaya hidup pasien berdasarkan kategori risiko, dilihat dari aktivitas fisik, jumlah langkah harian, durasi tidur, kualitas diet, dan konsumsi alkohol?

```
[42]: risk_order = ['Low', 'Medium', 'High']

lifestyle_cols = [
    'physical_activity_hours_per_week',
    'daily_steps',
    'sleep_hours',
    'diet_quality_score',
    'alcohol_units_per_week'
]

avg_lifestyle = (
    df_clean
    .groupby('risk_category')[lifestyle_cols]
    .mean()
    .reindex(risk_order)
    .round(2)
    .reset_index()
)

display(avg_lifestyle)

label_map = {
    'physical_activity_hours_per_week': 'Physical Activity Hours per Week',
    'daily_steps': 'Daily Steps',
    'sleep_hours': 'Sleep Hours',
    'diet_quality_score': 'Diet Quality Score',
    'alcohol_units_per_week': 'Alcohol Units per Week'
}

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

for i, col in enumerate(lifestyle_cols):
    plot_data = avg_lifestyle[['risk_category', col]]

    sns.barplot(
        data=plot_data,
```

```

        x='risk_category',
        y=col,
        order=risk_order,
        ax=axes[i]
    )

    axes[i].set_title(f'Average {label_map[col]} by Risk Category')
    axes[i].set_xlabel('Risk Category')
    axes[i].set_ylabel(label_map[col])

    for container in axes[i].containers:
        axes[i].bar_label(container, fmt='%.2f')

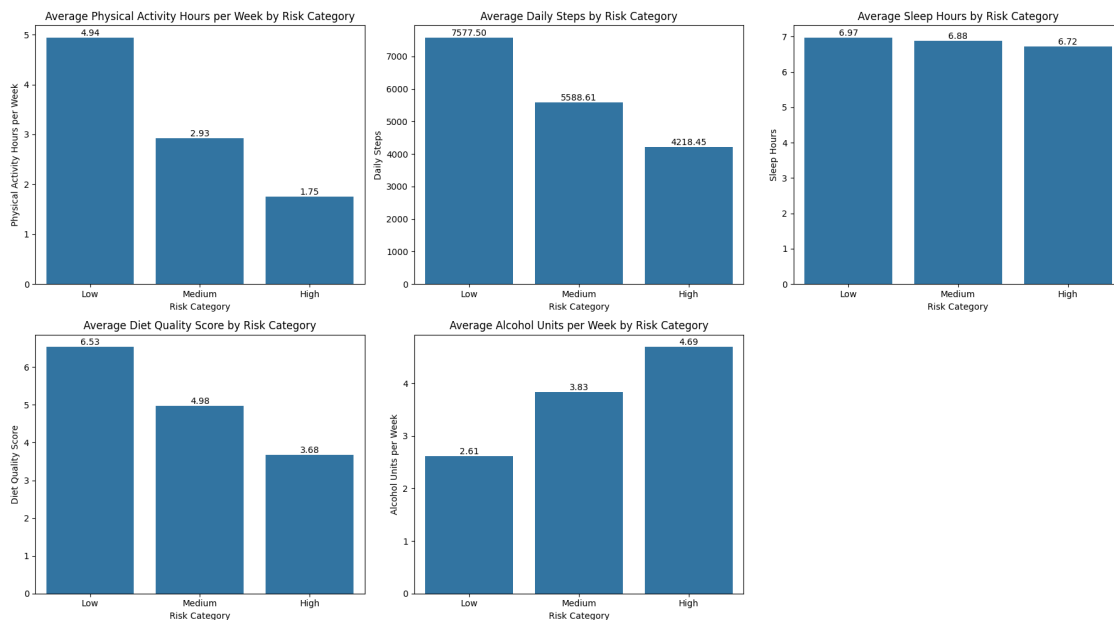
for j in range(len(lifestyle_cols), len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()

```

	risk_category	physical_activity_hours_per_week	daily_steps	sleep_hours	\
0	Low	4.94	7577.50	6.97	
1	Medium	2.93	5588.61	6.88	
2	High	1.75	4218.45	6.72	

	diet_quality_score	alcohol_units_per_week
0	6.53	2.61
1	4.98	3.83
2	3.68	4.69



Insight

- Rata-rata aktivitas fisik menurun dari **Low** (4.94 jam/minggu) ke **High** (1.75 jam/minggu).
- Rata-rata daily steps juga menurun dari **Low** (7577.50 langkah) ke **High** (4218.45 langkah).
- Rata-rata sleep hours sedikit menurun dari **Low** (6.97 jam) ke **High** (6.72 jam).
- Rata-rata diet quality score menurun dari **Low** (6.53) ke **High** (3.68).
- Rata-rata alcohol units per week meningkat dari **Low** (2.61) ke **High** (4.69).
- Pasien kategori **High Risk** cenderung memiliki aktivitas fisik lebih rendah, langkah harian lebih sedikit, kualitas diet lebih rendah, dan konsumsi alkohol lebih tinggi.

8.1.5 5. Bagaimana proporsi status merokok dan riwayat keluarga penyakit jantung pada setiap kategori risiko?

```
[43]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.ticker import PercentFormatter

risk_order = ['Low', 'Medium', 'High']
smoking_order = ['Never', 'Former', 'Current']
family_order = ['No', 'Yes']

combined_count = pd.concat(
    [
        pd.crosstab(
            df_clean['risk_category'],
            df_clean['smoking_status']
        ).reindex(index=risk_order, columns=smoking_order),

        pd.crosstab(
            df_clean['risk_category'],
            df_clean['family_history_heart_disease']
        ).reindex(index=risk_order, columns=family_order)
    ],
    axis=1
)

display(combined_count)

smoking_pct = pd.crosstab(
    df_clean['risk_category'],
    df_clean['smoking_status'],
    normalize='index'
).reindex(index=risk_order, columns=smoking_order) * 100
```

```

family_pct = pd.crosstab(
    df_clean['risk_category'],
    df_clean['family_history_heart_disease'],
    normalize='index'
).reindex(index=risk_order, columns=family_order) * 100

smoking_pct = smoking_pct.round(2)
family_pct = family_pct.round(2)

display(smoking_pct)
display(family_pct)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

smoking_pct.plot(
    kind='bar',
    stacked=True,
    ax=axes[0]
)

axes[0].set_title('Smoking Status Proportion by Risk Category')
axes[0].set_xlabel('Risk Category')
axes[0].set_ylabel('Percentage of Patients')
axes[0].yaxis.set_major_formatter(PercentFormatter(decimals=0))
axes[0].legend(title='Smoking Status')
axes[0].tick_params(axis='x', rotation=0)

family_pct.plot(
    kind='bar',
    stacked=True,
    ax=axes[1]
)

axes[1].set_title('Family History Proportion by Risk Category')
axes[1].set_xlabel('Risk Category')
axes[1].set_ylabel('Percentage of Patients')
axes[1].yaxis.set_major_formatter(PercentFormatter(decimals=0))
axes[1].legend(title='Family History')
axes[1].tick_params(axis='x', rotation=0)

plt.tight_layout()
plt.show()

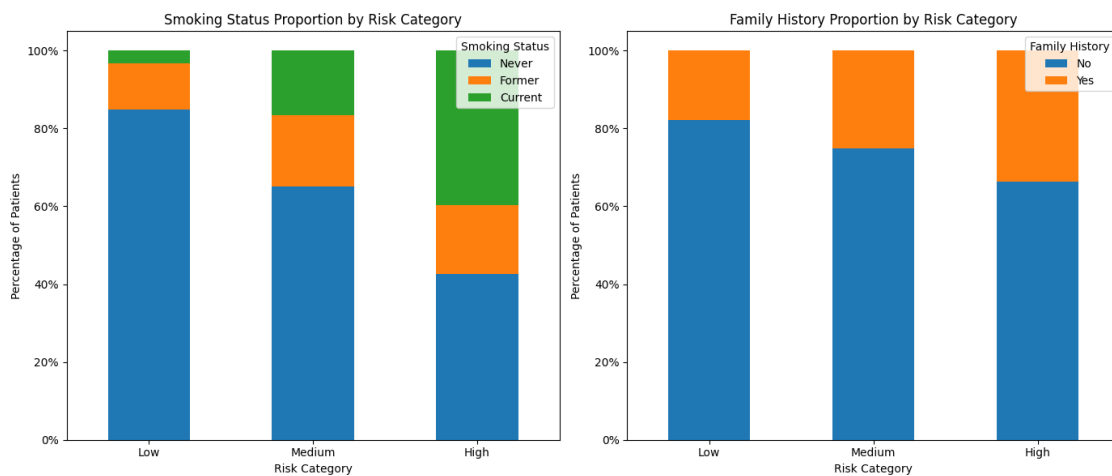
```

	Never	Former	Current	No	Yes
risk_category					
Low	1561	218	59	1512	326

Medium	1463	408	373	1681	563
High	603	253	562	941	477

smoking_status	Never	Former	Current
risk_category			
Low	84.93	11.86	3.21
Medium	65.20	18.18	16.62
High	42.52	17.84	39.63

family_history_heart_disease	No	Yes
risk_category		
Low	82.26	17.74
Medium	74.91	25.09
High	66.36	33.64



Insight

- Pada kategori Low Risk, mayoritas pasien berstatus *Never smoker* dan tidak memiliki riwayat keluarga penyakit jantung.
- Pada kategori Medium Risk, proporsi *Never smoker* masih paling besar, tetapi mulai menurun dibanding kategori Low Risk.
- Pada kategori High Risk, proporsi *Current smoker* meningkat paling besar dibanding kategori lain.
- Proporsi pasien dengan riwayat keluarga penyakit jantung (*Yes*) meningkat dari Low → Medium → High.
- Proporsi *Never smoker* menurun seiring meningkatnya kategori risiko.
- Proporsi pasien tanpa riwayat keluarga penyakit jantung menurun pada kategori risiko yang lebih tinggi.
- Kategori High Risk memiliki proporsi perokok aktif dan riwayat keluarga penyakit jantung paling besar.
- Pola ini menunjukkan bahwa kebiasaan merokok aktif dan riwayat keluarga penyakit jantung lebih banyak ditemukan pada pasien kategori High Risk.

8.1.6 6. Fitur klinis dan gaya hidup apa yang paling menonjol pada pasien kategori High Risk berdasarkan hasil EDA?

```
[44]: risk_order = ['Low', 'Medium', 'High']

feature_cols = [
    'age',
    'bmi',
    'systolic_bp',
    'diastolic_bp',
    'cholesterol_mg_dl',
    'resting_heart_rate',
    'daily_steps',
    'physical_activity_hours_per_week',
    'sleep_hours',
    'diet_quality_score',
    'alcohol_units_per_week'
]

# Rata-rata setiap fitur berdasarkan kategori risiko
avg_features_by_risk = (
    df_clean
    .groupby('risk_category')[feature_cols]
    .mean()
    .reindex(risk_order)
    .round(2)
)

display(avg_features_by_risk)
```

	age	bmi	systolic_bp	diastolic_bp	cholesterol_mg_dl	\
risk_category						
Low	37.04	24.80	133.95	87.10	212.12	
Medium	55.53	28.60	148.52	96.71	242.95	
High	73.07	31.87	162.45	105.48	270.26	

	resting_heart_rate	daily_steps	\
risk_category			
Low	70.96	7577.50	
Medium	74.67	5588.61	
High	77.20	4218.45	

	physical_activity_hours_per_week	sleep_hours	\
risk_category			
Low	4.94	6.97	
Medium	2.93	6.88	
High	1.75	6.72	

	diet_quality_score	alcohol_units_per_week
risk_category		
Low	6.53	2.61
Medium	4.98	3.83
High	3.68	4.69

```
[45]: # Rata-rata fitur pada seluruh pasien
overall_mean = df_clean[feature_cols].mean()

# Rata-rata fitur pada pasien High Risk
high_risk_mean = df_clean[df_clean['risk_category'] == 'High'][feature_cols].
    ↪mean()

# Selisih High Risk dibanding rata-rata keseluruhan
high_risk_comparison = pd.DataFrame({
    'Overall Mean': overall_mean,
    'High Risk Mean': high_risk_mean,
    'Difference': high_risk_mean - overall_mean,
    'Difference (%)': ((high_risk_mean - overall_mean) / overall_mean * 100)
}).round(2)

high_risk_comparison = high_risk_comparison.sort_values(
    by='Difference (%)',
    ascending=False
)

display(high_risk_comparison)
```

	Overall Mean	High Risk Mean	Difference \
age	53.87	73.07	19.20
alcohol_units_per_week	3.65	4.69	1.05
bmi	28.17	31.87	3.70
cholesterol_mg_dl	239.69	270.26	30.57
systolic_bp	147.24	162.45	15.21
diastolic_bp	95.76	105.48	9.72
resting_heart_rate	74.08	77.20	3.12
sleep_hours	6.87	6.72	-0.14
daily_steps	5900.01	4218.45	-1681.56
diet_quality_score	5.16	3.68	-1.48
physical_activity_hours_per_week	3.29	1.75	-1.55

	Difference (%)
age	35.64
alcohol_units_per_week	28.73
bmi	13.13
cholesterol_mg_dl	12.75
systolic_bp	10.33
diastolic_bp	10.15

resting_heart_rate	4.21
sleep_hours	-2.11
daily_steps	-28.50
diet_quality_score	-28.66
physical_activity_hours_per_week	-47.00

```
[46]: plt.figure(figsize=(10, 6))

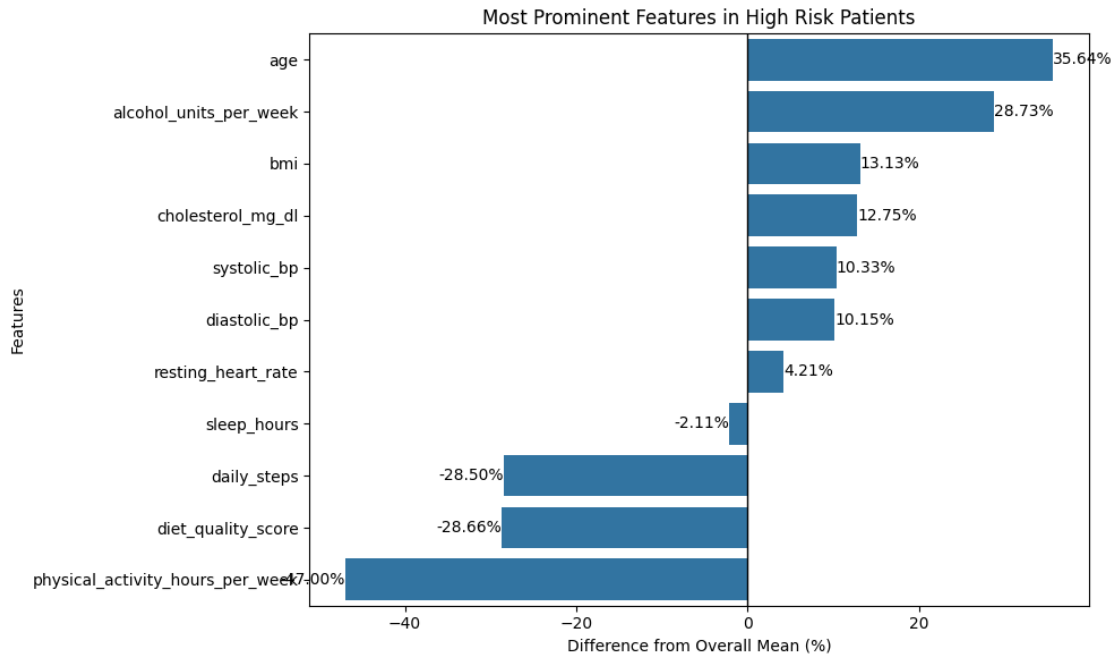
sns.barplot(
    data=high_risk_comparison.reset_index(),
    x='Difference (%)',
    y='index'
)

plt.axvline(0, color='black', linewidth=1)

plt.title('Most Prominent Features in High Risk Patients')
plt.xlabel('Difference from Overall Mean (%)')
plt.ylabel('Features')

for i, value in enumerate(high_risk_comparison['Difference (%)']):
    plt.text(
        value,
        i,
        f'{value:.2f}%',
        va='center',
        ha='left' if value >= 0 else 'right'
    )

plt.tight_layout()
plt.show()
```



Insight

- Fitur yang paling menonjol pada pasien **High Risk** adalah **age**, lebih tinggi **35.64%** dari rata-rata keseluruhan.
- **Alcohol units per week** juga lebih tinggi **28.73%** pada pasien High Risk.
- Fitur klinis seperti **BMI**, **cholesterol**, **systolic BP**, dan **diastolic BP** lebih tinggi pada pasien High Risk.
- **Physical activity hours per week** lebih rendah **47.00%** dibanding rata-rata keseluruhan.
- **Diet quality score** lebih rendah **28.66%** pada pasien High Risk.
- **Daily steps** lebih rendah **28.50%** pada pasien High Risk.
- Pasien **High Risk** cenderung lebih tua, konsumsi alkohol lebih tinggi, aktivitas fisik lebih rendah, pola makan lebih buruk, dan indikator klinis lebih tinggi.

8.1.7 7. Kelompok pasien mana yang perlu menjadi prioritas edukasi pencegahan berdasarkan kategori risiko, tekanan darah, kolesterol, aktivitas fisik, dan riwayat keluarga?

```
[47]: # Membuat salinan data
priority_df = df_clean.copy()

# Menentukan kondisi prioritas
priority_df['priority_group'] = 'Low Priority'

priority_df.loc[
    (
        (priority_df['risk_category'] == 'High') |
```

```

(priority_df['systolic_bp'] >= 140) |
(priority_df['diastolic_bp'] >= 90) |
(priority_df['cholesterol_mg_dl'] >= 240) |
(priority_df['physical_activity_hours_per_week'] < 2) |
(priority_df['family_history_heart_disease'] == 'Yes')
),
'priority_group'
] = 'Need Education Priority'

# Menampilkan jumlah dan persentase kelompok prioritas
priority_summary = pd.DataFrame({
    'Count': priority_df['priority_group'].value_counts(),
    'Percentage (%)': round(priority_df['priority_group'].
↪value_counts(normalize=True) * 100, 2)
})

display(priority_summary)

```

priority_group	Count	Percentage (%)
Need Education Priority	4822	87.67
Low Priority	678	12.33

```

[48]: priority_profile = (
    priority_df
    .groupby('priority_group')[
        [
            'systolic_bp',
            'diastolic_bp',
            'cholesterol_mg_dl',
            'physical_activity_hours_per_week',
            'heart_disease_risk_score'
        ]
    ]
    .mean()
    .round(2)
)

display(priority_profile)

```

priority_group	systolic_bp	diastolic_bp	cholesterol_mg_dl \
Low Priority	129.15	83.00	203.82
Need Education Priority	149.79	97.55	244.73

priority_group	physical_activity_hours_per_week \
Low Priority	1.50
Need Education Priority	1.50

Low Priority	6.01
Need Education Priority	2.91

	heart_disease_risk_score
priority_group	
Low Priority	4.46
Need Education Priority	42.19

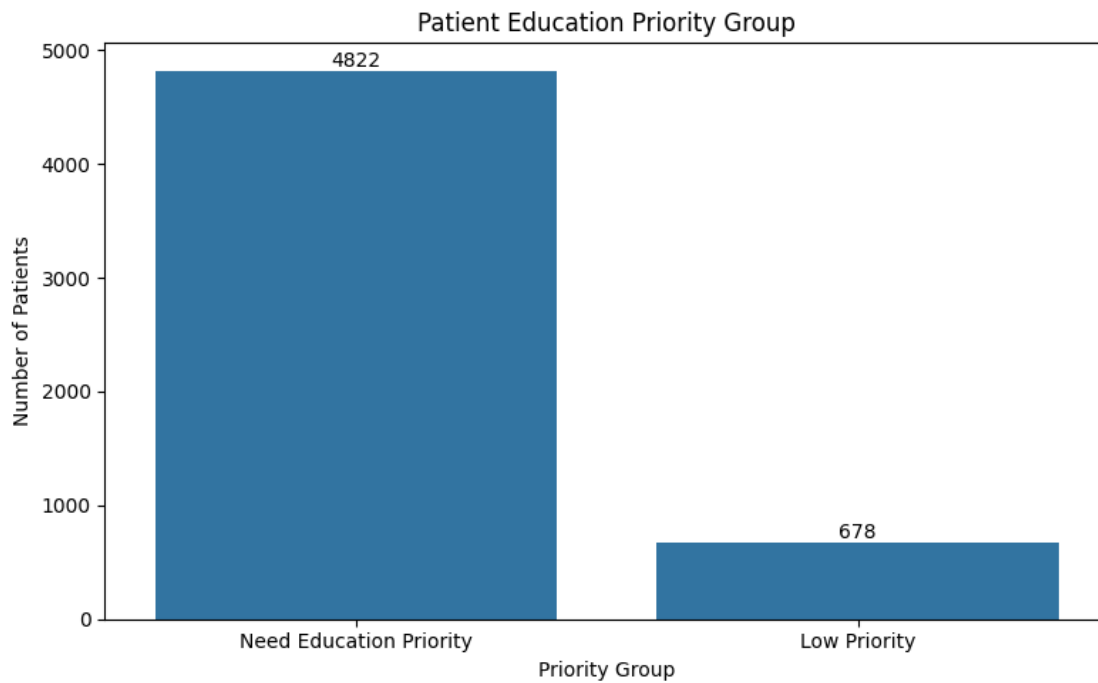
```
[49]: plt.figure(figsize=(8, 5))

sns.countplot(
    data=priority_df,
    x='priority_group'
)

plt.title('Patient Education Priority Group')
plt.xlabel('Priority Group')
plt.ylabel('Number of Patients')

for container in plt.gca().containers:
    plt.gca().bar_label(container)

plt.tight_layout()
plt.show()
```



Insight

- Sebanyak **4.822 pasien** masuk kelompok **Need Education Priority**.
- Sebanyak **678 pasien** masuk kelompok **Low Priority**.
- Mayoritas pasien membutuhkan edukasi pencegahan kardiovaskular.
- Kelompok prioritas edukasi jauh lebih besar dibanding kelompok prioritas rendah.
- Edukasi pencegahan perlu difokuskan pada pasien dengan risiko tinggi, tekanan darah tinggi, kolesterol tinggi, aktivitas fisik rendah, atau riwayat keluarga penyakit jantung.

9 FITUR ENGINEERING

```
[50]: # Cek data clean
display(df_clean.head())
print("Ukuran data clean:", df_clean.shape)

# Simpan data clean
df_clean.to_csv('cardiovascular_risk_dataset_clean.csv', index=False)

# Download data clean
from google.colab import files
files.download('cardiovascular_risk_dataset_clean.csv')
```

	Patient_ID	age	bmi	systolic_bp	diastolic_bp	cholesterol_mg_dl	\
0	1	62	25.0	142	93.0	247	
1	2	54	29.7	158	101.0	254	
2	3	46	36.2	170	113.0	276	
3	4	48	30.4	153	98.0	230	
4	5	46	25.3	139	87.0	206	

	resting_heart_rate	smoking_status	daily_steps	stress_level	\
0	72.0	Never	11565	3	
1	74.0	Current	4036	8	
2	80.0	Current	3043	9	
3	73.0	Former	5604	5	
4	69.0	Current	7464	1	

	physical_activity_hours_per_week	sleep_hours	family_history_heart_disease	\
0		5.6	8.2	No
1		0.5	6.7	No
2		0.4	4.1	No
3		0.6	8.0	No
4		2.0	6.1	No

	diet_quality_score	alcohol_units_per_week	heart_disease_risk_score	\
0	7		0.70	28.1
1	5		4.50	63.0
2	1		11.45	73.1
3	4		8.50	39.5

4	5	3.60	29.3
---	---	------	------

```

risk_category age_group
0      Medium    60-69
1      High     50-59
2      High     40-49
3      Medium    40-49
4      Medium    40-49

```

Ukuran data clean: (5500, 18)

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

```
[51]: df_clean.info()
```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 5500 entries, 0 to 5499

Data columns (total 18 columns):

#	Column	Non-Null Count	Dtype
0	Patient_ID	5500 non-null	int64
1	age	5500 non-null	int64
2	bmi	5500 non-null	float64
3	systolic_bp	5500 non-null	int64
4	diastolic_bp	5500 non-null	float64
5	cholesterol_mg_dl	5500 non-null	int64
6	resting_heart_rate	5500 non-null	float64
7	smoking_status	5500 non-null	object
8	daily_steps	5500 non-null	int64
9	stress_level	5500 non-null	int64
10	physical_activity_hours_per_week	5500 non-null	float64
11	sleep_hours	5500 non-null	float64
12	family_history_heart_disease	5500 non-null	object
13	diet_quality_score	5500 non-null	int64
14	alcohol_units_per_week	5500 non-null	float64
15	heart_disease_risk_score	5500 non-null	float64
16	risk_category	5500 non-null	object
17	age_group	5500 non-null	category

dtypes: category(1), float64(7), int64(7), object(3)

memory usage: 736.3+ KB

9.0.1 ENCODING

```

[52]: from sklearn.preprocessing import LabelEncoder
import pandas as pd

# Kolom kategorik yang akan diubah langsung

```

```

kolom_kategorik = [
    'smoking_status',
    'family_history_heart_disease',
    'risk_category',
    'age_group'
]

# Dictionary untuk menyimpan mapping encoding
encoding_mapping = {}

for col in kolom_kategorik:
    le = LabelEncoder()

    # Simpan mapping sebelum kolom diganti
    le.fit(df_clean[col].astype(str))
    encoding_mapping[col] = {
        label: int(code) for label, code in zip(le.classes_, le.transform(le.
↪classes_))
    }

    # Ubah langsung kolom aslinya
    df_clean[col] = le.transform(df_clean[col].astype(str))

# Tampilkan mapping encoding
for col, mapping in encoding_mapping.items():
    print(f"\nEncoding untuk kolom: {col}")
    for label, code in mapping.items():
        print(f"{code} = {label}")

```

Encoding untuk kolom: smoking_status

0 = Current
1 = Former
2 = Never

Encoding untuk kolom: family_history_heart_disease

0 = No
1 = Yes

Encoding untuk kolom: risk_category

0 = High
1 = Low
2 = Medium

Encoding untuk kolom: age_group

0 = 30-39
1 = 40-49

```

2 = 50-59
3 = 60-69
4 = 70-79
5 = 80+
6 = <30

```

```
[53]: df_clean.head()
```

```

[53]:   Patient_ID  age  bmi  systolic_bp  diastolic_bp  cholesterol_mg_dl  \
0          1   62  25.0          142          93.0          247
1          2   54  29.7          158          101.0          254
2          3   46  36.2          170          113.0          276
3          4   48  30.4          153          98.0          230
4          5   46  25.3          139          87.0          206

      resting_heart_rate  smoking_status  daily_steps  stress_level  \
0              72.0          2          11565          3
1              74.0          0          4036          8
2              80.0          0          3043          9
3              73.0          1          5604          5
4              69.0          0          7464          1

      physical_activity_hours_per_week  sleep_hours  \
0              5.6          8.2
1              0.5          6.7
2              0.4          4.1
3              0.6          8.0
4              2.0          6.1

      family_history_heart_disease  diet_quality_score  alcohol_units_per_week  \
0              0          7          0.70
1              0          5          4.50
2              0          1          11.45
3              0          4          8.50
4              0          5          3.60

      heart_disease_risk_score  risk_category  age_group
0              28.1          2          3
1              63.0          0          2
2              73.1          0          1
3              39.5          2          1
4              29.3          2          1

```

9.0.2 Membuat Fitur Baru

9.1 Feature Engineering

Feature engineering dilakukan untuk membuat fitur tambahan dari kolom yang sudah tersedia. Tujuannya adalah membantu model memahami pola risiko kardiovaskular dengan lebih baik, terutama dari sisi tekanan darah, BMI, kolesterol, aktivitas fisik, kualitas tidur, konsumsi alkohol, dan kombinasi faktor risiko klinis maupun gaya hidup.

9.1.1 Fitur Baru yang Dibuat

No	Nama Fitur	Deskripsi	Keterangan Nilai
1	<code>pulse_pressure</code>	Selisih antara tekanan darah sistolik dan diastolik.	Semakin tinggi nilainya, semakin besar perbedaan tekanan darah.
2	<code>blood_pressure_ratio</code>	Rasio antara tekanan darah sistolik dan diastolik.	Digunakan untuk melihat perbandingan tekanan darah pasien.
3	<code>hypertension_stage</code>	Kategori tingkat hipertensi berdasarkan tekanan darah sistolik dan diastolik.	0 = Normal, 1 = Elevated, 2 = Hypertension Stage 1, 3 = Hypertension Stage 2
4	<code>bmi_category</code>	Kategori BMI pasien berdasarkan nilai <code>bmi</code> .	0 = Underweight, 1 = Normal, 2 = Overweight, 3 = Obese
5	<code>cholesterol_category</code>	Kategori kadar kolesterol pasien berdasarkan nilai <code>cholesterol_mg_dl</code> .	0 = Normal, 1 = Borderline High, 2 = High
6	<code>activity_level</code>	Kategori tingkat aktivitas berdasarkan jumlah langkah harian.	0 = Low Activity, 1 = Moderate Activity, 2 = High Activity
7	<code>sleep_category</code>	Kategori durasi tidur pasien berdasarkan <code>sleep_hours</code> .	0 = Short Sleep, 1 = Normal Sleep, 2 = Long Sleep
8	<code>alcohol_category</code>	Kategori konsumsi alkohol mingguan berdasarkan <code>alcohol_units_per_week</code> .	0 = Low, 1 = Moderate, 2 = High

No	Nama Fitur	Deskripsi	Keterangan Nilai
9	<code>lifestyle_risk_score</code>	Skor gabungan faktor gaya hidup berisiko.	Semakin tinggi skor, semakin banyak indikator gaya hidup tidak sehat.
10	<code>clinical_risk_score</code>	Skor gabungan faktor klinis berisiko.	Semakin tinggi skor, semakin banyak indikator klinis yang berisiko.

9.1.2 Penjelasan Fitur Gabungan

`lifestyle_risk_score` Fitur ini dibuat dari beberapa indikator gaya hidup, yaitu:

- Jumlah langkah harian rendah
- Aktivitas fisik mingguan rendah
- Durasi tidur kurang
- Kualitas diet rendah
- Konsumsi alkohol tinggi
- Status merokok aktif

Semakin tinggi nilai `lifestyle_risk_score`, semakin banyak faktor gaya hidup berisiko yang dimiliki pasien.

`clinical_risk_score` Fitur ini dibuat dari beberapa indikator klinis, yaitu:

- BMI tinggi
- Tekanan darah sistolik tinggi
- Tekanan darah diastolik tinggi
- Kolesterol tinggi
- Resting heart rate tinggi
- Riwayat keluarga penyakit jantung

Semakin tinggi nilai `clinical_risk_score`, semakin banyak faktor klinis berisiko yang dimiliki pasien.

9.1.3 Tujuan Feature Engineering

Feature engineering ini bertujuan untuk:

- Mengubah informasi numerik menjadi kategori yang lebih mudah dipahami.
- Menambahkan fitur gabungan yang merepresentasikan tingkat risiko klinis dan gaya hidup.
- Membantu model machine learning dalam mengenali pola risiko kardiovaskular.
- Meningkatkan interpretasi hasil analisis dan pemodelan.

```
[54]: # 1. Pulse Pressure
# Selisih tekanan darah sistolik dan diastolik
df_clean['pulse_pressure'] = df_clean['systolic_bp'] - df_clean['diastolic_bp']
```

```

# 2. Blood Pressure Ratio
# Rasio tekanan darah sistolik terhadap diastolik
df_clean['blood_pressure_ratio'] = df_clean['systolic_bp'] /
    df_clean['diastolic_bp']

# 3. Hypertension Stage
# 0 = Normal
# 1 = Elevated / mulai tinggi
# 2 = Hypertension Stage 1
# 3 = Hypertension Stage 2
def categorize_hypertension(row):
    if row['systolic_bp'] < 120 and row['diastolic_bp'] < 80:
        return 0
    elif row['systolic_bp'] < 130 and row['diastolic_bp'] < 80:
        return 1
    elif row['systolic_bp'] < 140 or row['diastolic_bp'] < 90:
        return 2
    else:
        return 3

df_clean['hypertension_stage'] = df_clean.apply(categorize_hypertension, axis=1)

# 4. BMI Category
# 0 = Underweight
# 1 = Normal
# 2 = Overweight
# 3 = Obese
def categorize_bmi(bmi):
    if bmi < 18.5:
        return 0
    elif bmi < 25:
        return 1
    elif bmi < 30:
        return 2
    else:
        return 3

df_clean['bmi_category'] = df_clean['bmi'].apply(categorize_bmi)

# 5. Cholesterol Category
# 0 = Normal
# 1 = Borderline High
# 2 = High
def categorize_cholesterol(chol):

```



```

    if chol < 200:
        return 0
    elif chol < 240:
        return 1
    else:
        return 2

df_clean['cholesterol_category'] = df_clean['cholesterol_mg_dl'].
    ↪ apply(categorize_cholesterol)

# 6. Activity Level
# Berdasarkan jumlah langkah harian
# 0 = Low Activity
# 1 = Moderate Activity
# 2 = High Activity
def categorize_activity(steps):
    if steps < 5000:
        return 0
    elif steps < 10000:
        return 1
    else:
        return 2

df_clean['activity_level'] = df_clean['daily_steps'].apply(categorize_activity)

# 7. Sleep Category
# 0 = Short Sleep
# 1 = Normal Sleep
# 2 = Long Sleep
def categorize_sleep(hours):
    if hours < 6:
        return 0
    elif hours <= 8:
        return 1
    else:
        return 2

df_clean['sleep_category'] = df_clean['sleep_hours'].apply(categorize_sleep)

# 8. Alcohol Category
# 0 = Low
# 1 = Moderate
# 2 = High
def categorize_alcohol(units):

```

```

    if units < 3:
        return 0
    elif units <= 7:
        return 1
    else:
        return 2

df_clean['alcohol_category'] = df_clean['alcohol_units_per_week'].
    ↪ apply(categorize_alcohol)

# 9. Lifestyle Risk Score
# Semakin tinggi skor, semakin buruk gaya hidup
df_clean['lifestyle_risk_score'] = (
    (df_clean['daily_steps'] < 5000).astype(int) +
    (df_clean['physical_activity_hours_per_week'] < 2).astype(int) +
    (df_clean['sleep_hours'] < 6).astype(int) +
    (df_clean['diet_quality_score'] <= 4).astype(int) +
    (df_clean['alcohol_units_per_week'] > 7).astype(int) +
    (df_clean['smoking_status'] == 0).astype(int)
)

# 10. Clinical Risk Score
# Semakin tinggi skor, semakin banyak indikator klinis berisiko
df_clean['clinical_risk_score'] = (
    (df_clean['bmi'] >= 30).astype(int) +
    (df_clean['systolic_bp'] >= 140).astype(int) +
    (df_clean['diastolic_bp'] >= 90).astype(int) +
    (df_clean['cholesterol_mg_dl'] >= 240).astype(int) +
    (df_clean['resting_heart_rate'] >= 90).astype(int) +
    (df_clean['family_history_heart_disease'] == 1).astype(int)
)

df_clean.head()

```

```

[54]: Patient_ID  age  bmi  systolic_bp  diastolic_bp  cholesterol_mg_dl  \
0          1    62   25.0           142           93.0             247
1          2    54   29.7           158          101.0             254
2          3    46   36.2           170          113.0             276
3          4    48   30.4           153           98.0             230
4          5    46   25.3           139           87.0             206

      resting_heart_rate  smoking_status  daily_steps  stress_level  ...  \
0                72.0                2        11565             3  ...
1                74.0                0         4036             8  ...
2                80.0                0         3043             9  ...

```

3	73.0	1	5604	5 ...
4	69.0	0	7464	1 ...

	pulse_pressure	blood_pressure_ratio	hypertension_stage	bmi_category	\
0	49.0	1.526882	3	2	
1	57.0	1.564356	3	2	
2	57.0	1.504425	3	3	
3	55.0	1.561224	3	3	
4	52.0	1.597701	2	2	

	cholesterol_category	activity_level	sleep_category	alcohol_category	\
0	2	2	2	0	
1	2	0	1	1	
2	2	0	0	2	
3	1	1	1	2	
4	1	1	1	1	

	lifestyle_risk_score	clinical_risk_score
0	0	3
1	3	3
2	6	4
3	3	3
4	1	0

[5 rows x 28 columns]

```
[55]: df_clean.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5500 entries, 0 to 5499
Data columns (total 28 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Patient_ID                            5500 non-null   int64
1   age                                    5500 non-null   int64
2   bmi                                    5500 non-null   float64
3   systolic_bp                           5500 non-null   int64
4   diastolic_bp                           5500 non-null   float64
5   cholesterol_mg_dl                      5500 non-null   int64
6   resting_heart_rate                      5500 non-null   float64
7   smoking_status                          5500 non-null   int64
8   daily_steps                            5500 non-null   int64
9   stress_level                           5500 non-null   int64
10  physical_activity_hours_per_week        5500 non-null   float64
11  sleep_hours                             5500 non-null   float64
12  family_history_heart_disease            5500 non-null   int64
13  diet_quality_score                      5500 non-null   int64
```

```

14 alcohol_units_per_week      5500 non-null    float64
15 heart_disease_risk_score    5500 non-null    float64
16 risk_category               5500 non-null    int64
17 age_group                   5500 non-null    int64
18 pulse_pressure              5500 non-null    float64
19 blood_pressure_ratio        5500 non-null    float64
20 hypertension_stage          5500 non-null    int64
21 bmi_category                5500 non-null    int64
22 cholesterol_category        5500 non-null    int64
23 activity_level              5500 non-null    int64
24 sleep_category              5500 non-null    int64
25 alcohol_category            5500 non-null    int64
26 lifestyle_risk_score        5500 non-null    int64
27 clinical_risk_score          5500 non-null    int64
dtypes: float64(9), int64(19)
memory usage: 1.2 MB

```

10 DATA DICTIONARY

11 Data Dictionary

Data dictionary digunakan untuk menjelaskan setiap variabel yang terdapat dalam dataset. Tabel ini berisi nama variabel, tipe data, deskripsi, serta keterangan nilai atau satuan yang digunakan. Dengan adanya data dictionary, pembaca dapat memahami makna setiap kolom sebelum masuk ke tahap analisis dan pemodelan.

11.1 Data Dictionary Variabel Utama

No	Nama Variabel	Tipe Data	Deskripsi	Keterangan
1	Patient_ID	Integer	Identitas unik untuk setiap pasien	Digunakan sebagai ID pasien
2	age	Integer	Usia pasien	Dalam tahun
3	bmi	Float	Indeks massa tubuh pasien	Menggambarkan status berat badan pasien
4	systolic_bp	Integer	Tekanan darah sistolik pasien	Dalam satuan mmHg
5	diastolic_bp	Integer	Tekanan darah diastolik pasien	Dalam satuan mmHg
6	cholesterol_mg_dl	Integer	Kadar kolesterol pasien	Dalam satuan mg/dL
7	resting_heart_rate	Integer	Detak jantung pasien saat istirahat	Dalam satuan bpm
8	smoking_status	Object	Status merokok pasien	Current, Former, atau Never
9	daily_steps	Integer	Jumlah langkah harian pasien	Dalam langkah per hari
10	stress_level	Integer	Tingkat stres pasien	Skala 1 sampai 10

No	Nama Variabel	Tipe Data	Deskripsi	Keterangan
11	physical_activity_hours_per_week	Float	Durasi aktivitas fisik pasien per minggu	Dalam jam per minggu
12	sleep_hours	Float	Durasi tidur pasien	Dalam jam per hari
13	family_history_heart_disease	Object	Riwayat keluarga dengan penyakit jantung	Yes atau No
14	diet_quality_score	Integer	Skor kualitas pola makan pasien	Skala 1 sampai 10
15	alcohol_units_per_week	Float	Jumlah konsumsi alkohol pasien per minggu	Dalam unit per minggu
16	heart_disease_risk_score	Float	Skor risiko penyakit jantung	Rentang nilai 0 sampai 100
17	risk_category	Object	Kategori risiko penyakit jantung	Low, Medium, atau High

11.2 Data Dictionary Variabel Hasil Feature Engineering

No	Nama Variabel	Tipe Data	Deskripsi	Keterangan
1	pulse_pressure	Integer/Float	Selisih antara tekanan darah sistolik dan diastolik	$\text{systolic_bp} - \text{diastolic_bp}$
2	blood_pressure_ratio	Float	Rasio antara tekanan darah sistolik dan diastolik	$\text{systolic_bp} / \text{diastolic_bp}$
3	hypertension_stage	Integer/Kategori	Kategori tingkat hipertensi pasien	Dibuat berdasarkan tekanan darah sistolik dan diastolik
4	bmi_category	Kategori	Kategori BMI pasien	Mengelompokkan BMI ke dalam kategori tertentu
5	cholesterol_level	Kategori	Kategori kadar kolesterol pasien	Dibuat berdasarkan rentang kadar kolesterol
6	activity_level	Kategori	Kategori tingkat aktivitas fisik pasien	Dibuat berdasarkan aktivitas fisik atau jumlah langkah harian
7	sleep_category	Kategori	Kategori durasi tidur pasien	Mengelompokkan durasi tidur pasien
8	alcohol_category	Kategori	Kategori konsumsi alkohol pasien	Dibuat berdasarkan jumlah unit alkohol per minggu
9	lifestyle_risk_score	Integer/Float	Skor risiko berdasarkan faktor gaya hidup	Melibatkan faktor seperti aktivitas, tidur, alkohol, merokok, dan diet
10	clinical_risk_score	Integer/Float	Skor risiko berdasarkan faktor klinis	Melibatkan faktor seperti usia, BMI, tekanan darah, kolesterol, dan detak jantung
11	age_group	Kategori	Kelompok usia pasien	Contoh: <30, 30-39, 40-49, 50-59, 60-69, 70-79, dan 80+

12 SAVE DATA

```
[56]: output_path = '/content/cardiovascular_risk_dataset_feature_engineered.csv'
df_clean.to_csv(output_path, index=False)

# Download file
from google.colab import files
files.download(output_path)
```

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

13 UBAH JADI PDF

```
[66]: # Install package untuk export PDF
!apt-get update -qq
!apt-get install -y pandoc texlive-xetex texlive-fonts-recommended
↪texlive-plain-generic > /dev/null
```

W: Skipping acquire of configured file 'main/source/Sources' as repository
'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide
it (sources.list entry misspelt?)
~C

```
[69]: # Ubah nama file sesuai nama notebook kamu
!jupyter nbconvert --to pdf "CLEANING_DATA.ipynb"
```

```
[NbConvertApp] Converting notebook CLEANING_DATA.ipynb to pdf
[NbConvertApp] Support files will be in CLEANING_DATA_files/
[NbConvertApp] Making directory ./CLEANING_DATA_files
[NbConvertApp] Writing 236628 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no
citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 2284479 bytes to CLEANING_DATA.pdf
```

```
[68]: !jupyter nbconvert --to html --no-input --embed-images "/content/CLEANING_DATA.
↪ipynb" --output "/content/CLEANING_DATA_REPORT.html"
```

```
[NbConvertApp] Converting notebook /content/CLEANING_DATA.ipynb to html
[NbConvertApp] WARNING | Alternative text is missing on 19 image(s).
[NbConvertApp] Writing 3670492 bytes to /content/CLEANING_DATA_REPORT.html
```

Python Code

```
1  # DASHBOARD STREAMLIT
2
3  import streamlit as st
4  import pandas as pd
5  import numpy as np
6  import plotly.express as px
7  from plotly.subplots import make_subplots
8  import plotly.graph_objects as go
9
10
11 # =====
12 # PAGE CONFIG
13 st.set_page_config(
14     page_title="Dashboard Cardiovascular Risk Analysis",
15     page_icon="❤️",
16     layout="wide",
17     initial_sidebar_state="expanded"
18 )
19
20
21 # =====
22 # DATA PATH
23 # =====
24 DATA_PATH = "cardiovascular_risk_dataset_clean.csv"
25
26
27 # =====
28 # CONSTANTS
29 # =====
30 RISK_ORDER = ["Low", "Medium", "High"]
31 AGE_ORDER = ["<30", "30-39", "40-49", "50-59", "60-69", "70-79", "80+"]
32 FAMILY_ORDER = ["No", "Yes"]
33 BMI_ORDER = ["Underweight", "Normal", "Overweight", "Obese"]
34
35 COLORS = {
36     "navy": "#0B1F5B",
37     "blue": "#2563EB",
38     "green": "#22A55A",
39     "orange": "#F59E0B",
40     "red": "#EF4444",
41     "purple": "#7C3AED",
42     "teal": "#14B8A6",
43     "bg": "#F5F8FD",
44     "card": "#FFFFFF",
45     "border": "#DCE6F2",
46     "text": "#1E293B",
47     "muted": "#64748B",
48 }
49
50 RISK_COLORS = {
51     "Low": COLORS["green"],
52     "Medium": COLORS["orange"],
53     "High": COLORS["red"],
54 }
55
56
57 # =====
58 # CSS
```

```

59 # =====
60 st.markdown(
61     f"""
62     <style>
63         html, body, [data-testid="stAppViewContainer"] {{
64             background-color: {COLORS["bg"]};
65             color: {COLORS["text"]};
66         }}
67
68         [data-testid="stHeader"] {{
69             background: rgba(0,0,0,0);
70         }}
71
72         [data-testid="stToolbar"], #MainMenu, footer {{
73             display: none !important;
74         }}
75
76         .block-container {{
77             max-width: 1600px;
78             padding-top: 1rem;
79             padding-bottom: 2rem;
80             padding-left: 1.5rem;
81             padding-right: 1.5rem;
82         }}
83
84         .header-box {{
85             background: linear-gradient(135deg, #FFFFFF 0%, #F0F6FF 100%);
86             border: 1px solid {COLORS["border"]};
87             border-radius: 22px;
88             padding: 24px 28px;
89             box-shadow: 0 6px 22px rgba(15,23,42,0.05);
90             margin-bottom: 18px;
91             text-align: center;
92         }}
93
94         .header-title {{
95             font-size: 38px;
96             font-weight: 900;
97             color: {COLORS["navy"]};
98             line-height: 1.1;
99             margin-bottom: 6px;
100         }}
101
102         .header-subtitle {{
103             font-size: 16px;
104             color: {COLORS["muted"]};
105             font-weight: 500;
106         }}
107
108         .filter-title {{
109             font-size: 18px;
110             font-weight: 900;
111             color: {COLORS["navy"]};
112             margin-bottom: 8px;
113         }}
114
115         .kpi-card {{
116             background: white;
117             border: 1px solid {COLORS["border"]};
118             border-radius: 18px;
119             padding: 20px 18px;
120             box-shadow: 0 5px 16px rgba(15,23,42,0.05);

```



```

121     min-height: 135px;
122 }}
123
124 .kpi-label {{
125     font-size: 14px;
126     font-weight: 800;
127     color: {COLORS["navy"]};
128     margin-bottom: 10px;
129     line-height: 1.25;
130 }}
131
132 .kpi-value {{
133     font-size: 34px;
134     font-weight: 900;
135     line-height: 1;
136 }}
137
138 .kpi-unit {{
139     font-size: 14px;
140     font-weight: 700;
141     color: {COLORS["muted"]};
142     margin-top: 8px;
143 }}
144
145 .section-title {{
146     font-size: 22px;
147     font-weight: 900;
148     color: {COLORS["navy"]};
149     margin-top: 18px;
150     margin-bottom: 12px;
151 }}
152
153 .chart-title {{
154     font-size: 17px;
155     font-weight: 900;
156     color: {COLORS["navy"]};
157     margin-bottom: 10px;
158     line-height: 1.25;
159 }}
160
161 .chart-badge {{
162     display: inline-flex;
163     width: 26px;
164     height: 26px;
165     border-radius: 50%;
166     background: {COLORS["blue"]};
167     color: white;
168     align-items: center;
169     justify-content: center;
170     font-size: 13px;
171     font-weight: 900;
172     margin-right: 8px;
173 }}
174
175 .priority-item {{
176     background: #F8FBFF;
177     border: 1px solid #E8EEF7;
178     border-radius: 14px;
179     padding: 14px 14px;
180     margin-bottom: 12px;
181 }}
182

```

```

183 .priority-title {{
184     font-size: 14px;
185     font-weight: 900;
186     color: {COLORS["navy"]};
187     margin-bottom: 4px;
188 }}
189
190 .priority-sub {{
191     font-size: 13px;
192     font-weight: 600;
193     color: {COLORS["muted"]};
194     line-height: 1.35;
195 }}
196
197 .stSelectbox label {{
198     color: {COLORS["navy"]} !important;
199     font-size: 14px !important;
200     font-weight: 800 !important;
201 }}
202
203 div[data-baseweb="select"] > div {{
204     background: white !important;
205     border: 1px solid {COLORS["border"]} !important;
206     border-radius: 12px !important;
207     min-height: 44px !important;
208     color: {COLORS["text"]} !important;
209 }}
210
211 div[data-baseweb="select"] * {{
212     color: {COLORS["text"]} !important;
213 }}
214
215 [data-testid="stVerticalBlockBorderWrapper"] {{
216     background: white !important;
217     border-radius: 18px !important;
218     border: 1px solid {COLORS["border"]} !important;
219     box-shadow: 0 5px 16px rgba(15,23,42,0.05) !important;
220     padding: 10px !important;
221 }}
222
223 [data-testid="stRadio"] label {{
224     color: #0B1F5B !important;
225     font-weight: 800 !important;
226 }}
227
228 [data-testid="stRadio"] label p {{
229     color: #0B1F5B !important;
230     font-size: 15px !important;
231     font-weight: 800 !important;
232 }}
233
234 [data-testid="stRadio"] div[role="radiogroup"] > label {{
235     background: #FFFFFF !important;
236     border: 1px solid #DCE6F2 !important;
237     border-radius: 12px !important;
238     padding: 10px 12px !important;
239     margin-bottom: 8px !important;
240 }}
241
242 [data-testid="stRadio"] div[role="radiogroup"] > label:hover {{
243     background: #EEF6FF !important;
244     border-color: #2563EB !important;

```

```

245     }}
246
247
248     [data-testid="stSlider"] {{
249         background: #FFFFFF !important;
250         border: 1px solid #DCE6F2 !important;
251         border-radius: 12px !important;
252         padding: 4px 16px 0px 16px !important;
253         min-height: 40px !important;
254         height: 40px !important;
255     }}
256
257     [data-testid="stSlider"] label {{
258         display: none !important;
259     }}
260
261     [data-testid="stSlider"] > div {{
262         padding-top: 0px !important;
263         padding-bottom: 0px !important;
264     }}
265
266     [data-testid="stSlider"] [data-baseweb="slider"] {{
267         margin-top: -4px !important;
268 }}
269
270
271     .lifestyle-note {{
272         background: #F8FBFF;
273         border: 1px solid #E8EEF7;
274         border-radius: 14px;
275         padding: 14px 18px;
276         color: #64748B;
277         font-size: 15px;
278         font-weight: 600;
279         margin-top: 10px;
280         text-align: center;
281     }}
282
283     .lifestyle-note b {{
284         color: #0B1F5B;
285     }}
286
287 </style>
288 "",
289 unsafe_allow_html=True
290 )
291
292
293 # =====
294 # LOAD DATA
295 # =====
296 @st.cache_data
297 def load_data(path):
298     df = pd.read_csv(path)
299
300     numeric_cols = [
301         "age",
302         "bmi",
303         "systolic_bp",
304         "diastolic_bp",
305         "cholesterol_mg_dl",
306         "resting_heart_rate",

```

```

307     "daily_steps",
308     "stress_level",
309     "physical_activity_hours_per_week",
310     "sleep_hours",
311     "diet_quality_score",
312     "alcohol_units_per_week",
313     "heart_disease_risk_score",
314 ]
315
316 for col in numeric_cols:
317     if col in df.columns:
318         df[col] = pd.to_numeric(df[col], errors="coerce")
319
320 if "age_group" not in df.columns:
321     df["age_group"] = pd.cut(
322         df["age"],
323         bins=[0, 30, 40, 50, 60, 70, 80, 120],
324         labels=AGE_ORDER,
325         right=False,
326     ).astype(str)
327
328 if "bmi_category" not in df.columns:
329     df["bmi_category"] = pd.cut(
330         df["bmi"],
331         bins=[0, 18.5, 25, 30, 100],
332         labels=BMI_ORDER,
333         right=False,
334     ).astype(str)
335
336 return df
337
338
339 df = load_data(DATA_PATH)
340
341
342 # =====
343 # HELPER FUNCTIONS
344 # =====
345 def safe_mean(dataframe, col):
346     if dataframe.empty or col not in dataframe.columns:
347         return 0
348     return dataframe[col].mean()
349
350
351 def style_fig(fig, height=420, showlegend=True, margin=None):
352     if margin is None:
353         margin = dict(l=45, r=35, t=70, b=60)
354
355     fig.update_layout(
356         height=height,
357         paper_bgcolor="white",
358         plot_bgcolor="white",
359         margin=margin,
360         font=dict(
361             family="Arial",
362             size=12,
363             color=COLORS["text"],
364         ),
365         showlegend=showlegend,
366         legend=dict(
367             orientation="h",
368             yanchor="bottom",

```

```

369         y=1.08,
370         xanchor="left",
371         x=0,
372         font=dict(
373             size=12,
374             color=COLORS["text"],
375         ),
376         title=dict(
377             font=dict(
378                 size=12,
379                 color=COLORS["navy"],
380             )
381         ),
382         bgcolor="rgba(255,255,255,0)",
383     ),
384 )
385
386 fig.update_xaxes(
387     showgrid=False,
388     linecolor="#D5E1EE",
389     tickfont=dict(size=11, color=COLORS["text"]),
390     title_font=dict(size=12, color=COLORS["text"]),
391 )
392
393 fig.update_yaxes(
394     gridcolor="#EEF3F8",
395     linecolor="#D5E1EE",
396     tickfont=dict(size=11, color=COLORS["text"]),
397     title_font=dict(size=12, color=COLORS["text"]),
398 )
399
400 return fig
401
402
403 def chart_title(letter, title):
404     st.markdown(
405         f"""
406         <div class="chart-title">
407             <span class="chart-badge">{letter}</span>{title}
408         </div>
409         """,
410         unsafe_allow_html=True,
411     )
412
413
414 def kpi_card(label, value, unit, color):
415     st.markdown(
416         f"""
417         <div class="kpi-card">
418             <div class="kpi-label">{label}</div>
419             <div class="kpi-value" style="color:{color};">{value}</div>
420             <div class="kpi-unit">{unit}</div>
421         </div>
422         """,
423         unsafe_allow_html=True,
424     )
425
426
427 def make_small_lifestyle_chart(data, column, title, y_title, color_map):
428     fig = px.bar(
429         data,
430         x="risk_category",

```

```

431         y=column,
432         color="risk_category",
433         text=data[column].round(1),
434         color_discrete_map=color_map,
435         category_orders={"risk_category": RISK_ORDER},
436     )
437
438     fig.update_traces(
439         textposition="outside",
440         marker_line_width=0.8,
441         marker_line_color="white",
442     )
443
444     fig.update_layout(
445         height=330,
446         paper_bgcolor="white",
447         plot_bgcolor="white",
448         margin=dict(l=45, r=25, t=30, b=45),
449         font=dict(family="Arial", size=12, color=COLORS["text"]),
450         showlegend=False,
451         xaxis_title="Kategori Risiko",
452         yaxis_title=y_title,
453         title=dict(
454             text=title,
455             font=dict(size=18, color=COLORS["navy"]),
456             x=0.5,
457             xanchor="center",
458         ),
459     )
460
461     fig.update_xaxes(
462         showgrid=False,
463         linecolor="#D5E1EE",
464         tickfont=dict(size=11, color=COLORS["text"]),
465         title_font=dict(size=12, color=COLORS["text"]),
466     )
467
468     fig.update_yaxes(
469         gridcolor="#EEF3F8",
470         linecolor="#D5E1EE",
471         tickfont=dict(size=11, color=COLORS["text"]),
472         title_font=dict(size=12, color=COLORS["text"]),
473     )
474
475     return fig
476
477
478 def make_activity_progress_chart(data, column, title, x_title, color_map, max_value=None):
479     temp = data[["risk_category", column]].copy()
480     temp["risk_category"] = pd.Categorical(
481         temp["risk_category"],
482         categories=RISK_ORDER,
483         ordered=True
484     )
485     temp = temp.sort_values("risk_category", ascending=False)
486
487     if max_value is None:
488         max_value = max(6, float(temp[column].max()) + 1)
489
490     fig = go.Figure()
491
492     fig.add_trace(

```

```

493         go.Bar(
494             x=[max_value] * len(temp),
495             y=temp["risk_category"],
496             orientation="h",
497             marker=dict(
498                 color="#EAF0F7",
499                 line=dict(width=0),
500             ),
501             hoverinfo="skip",
502             showlegend=False,
503         )
504     )
505
506     fig.add_trace(
507         go.Bar(
508             x=temp[column],
509             y=temp["risk_category"],
510             orientation="h",
511             marker=dict(
512                 color=[color_map[r] for r in temp["risk_category"]],
513                 line=dict(width=0),
514             ),
515             text=temp[column].round(1),
516             textposition="outside",
517             textfont=dict(size=13, color=COLORS["text"]),
518             showlegend=False,
519         )
520     )
521
522     fig.update_layout(
523         bargmode="overlay",
524         height=330,
525         paper_bgcolor="white",
526         plot_bgcolor="white",
527         margin=dict(l=80, r=60, t=45, b=50),
528         font=dict(family="Arial", size=12, color=COLORS["text"]),
529         title=dict(
530             text=title,
531             font=dict(size=18, color=COLORS["navy"]),
532             x=0.5,
533             xanchor="center",
534         ),
535         xaxis_title=x_title,
536         yaxis_title="",
537         xaxis=dict(range=[0, max_value]),
538     )
539
540     fig.update_xaxes(
541         showgrid=False,
542         linecolor="#D5E1EE",
543         tickfont=dict(size=11, color=COLORS["text"]),
544         title_font=dict(size=12, color=COLORS["text"]),
545     )
546
547     fig.update_yaxes(
548         showgrid=False,
549         linecolor="#D5E1EE",
550         tickfont=dict(size=12, color=COLORS["text"]),
551     )
552
553     return fig
554

```

```

555 def make_sleep_lollipop_chart(data, column, title, x_title, color_map):
556     temp = data[["risk_category", column]].copy()
557     temp = temp.dropna(subset=[column])
558
559     temp["risk_category"] = pd.Categorical(
560         temp["risk_category"],
561         categories=RISK_ORDER,
562         ordered=True
563     )
564     temp = temp.sort_values("risk_category", ascending=False)
565
566     fig = go.Figure()
567
568     if temp.empty:
569         fig.update_layout(
570             height=330,
571             paper_bgcolor="white",
572             plot_bgcolor="white",
573             title=dict(
574                 text=title,
575                 font=dict(size=18, color=COLORS["navy"]),
576                 x=0.5,
577                 xanchor="center",
578             ),
579         )
580     return fig
581
582     x_min = max(0, float(temp[column].min()) - 0.3)
583     x_max = float(temp[column].max()) + 0.3
584
585     for risk in temp["risk_category"].astype(str):
586         value = float(temp.loc[temp["risk_category"].astype(str) == risk, column].iloc[0])
587         color = color_map[risk]
588
589         fig.add_trace(
590             go.Scatter(
591                 x=[x_min, value],
592                 y=[risk, risk],
593                 mode="lines",
594                 line=dict(
595                     color=color,
596                     width=5,
597                 ),
598                 showlegend=False,
599                 hoverinfo="skip",
600             )
601         )
602
603         fig.add_trace(
604             go.Scatter(
605                 x=[value],
606                 y=[risk],
607                 mode="markers+text",
608                 marker=dict(
609                     size=18,
610                     color=color,
611                     line=dict(color="white", width=2),
612                 ),
613                 text=[f"{value:.1f} jam"],
614                 textposition="middle right",
615                 textfont=dict(size=14, color=color, family="Arial"),
616                 showlegend=False,

```



```

617         hoverinfo="skip",
618     )
619 )
620
621 fig.update_layout(
622     height=330,
623     paper_bgcolor="white",
624     plot_bgcolor="white",
625     margin=dict(l=75, r=85, t=55, b=50),
626     font=dict(family="Arial", size=14, color=COLORS["text"]),
627     title=dict(
628         text=title,
629         font=dict(size=18, color=COLORS["navy"]),
630         x=0.5,
631         xanchor="center",
632     ),
633     xaxis_title=x_title,
634     yaxis_title="",
635     xaxis=dict(range=[x_min, x_max]),
636 )
637
638 fig.update_xaxes(
639     showgrid=True,
640     gridcolor="#EEF3F8",
641     linecolor="#D5E1EE",
642     tickfont=dict(size=13, color=COLORS["text"]),
643     title_font=dict(size=14, color=COLORS["text"]),
644 )
645
646 fig.update_yaxes(
647     categoryorder="array",
648     categoryarray=list(reversed(RISK_ORDER)),
649     showgrid=False,
650     linecolor="#D5E1EE",
651     tickfont=dict(size=13, color=COLORS["text"]),
652 )
653
654 return fig
655
656
657 def make_diet_gauge_chart(data, column, title, color_map, max_score=10):
658     temp = data[["risk_category", column]].copy()
659     temp = temp.dropna(subset=[column])
660
661     temp["risk_category"] = pd.Categorical(
662         temp["risk_category"],
663         categories=RISK_ORDER,
664         ordered=True
665     )
666     temp = temp.sort_values("risk_category")
667
668     available_risks = [
669         risk for risk in RISK_ORDER
670         if risk in temp["risk_category"].astype(str).tolist()
671     ]
672
673     fig = go.Figure()
674
675     if len(available_risks) == 0:
676         fig.update_layout(
677             height=330,
678             paper_bgcolor="white",

```

```

679         plot_bgcolor="white",
680         title=dict(
681             text=title,
682             font=dict(size=18, color=COLORS["navy"]),
683             x=0.5,
684             xanchor="center",
685         ),
686     )
687     return fig
688
689     if len(available_risks) == 1:
690         domains = {
691             available_risks[0]: (0.30, 0.70)
692         }
693     elif len(available_risks) == 2:
694         domains = {
695             available_risks[0]: (0.10, 0.45),
696             available_risks[1]: (0.55, 0.90),
697         }
698     else:
699         domains = {
700             "Low": (0.00, 0.30),
701             "Medium": (0.35, 0.65),
702             "High": (0.70, 1.00),
703         }
704
705     for risk in available_risks:
706         value = float(temp.loc[temp["risk_category"].astype(str) == risk, column].iloc[0])
707
708         fig.add_trace(
709             go.Indicator(
710                 mode="gauge+number",
711                 value=value,
712                 domain={"x": domains[risk], "y": [0.15, 0.95]},
713                 title={
714                     "text": risk,
715                     "font": {"size": 18, "color": COLORS["navy"]},
716                 },
717                 number={
718                     "font": {"size": 38, "color": color_map[risk]},
719                     "valueformat": ".1f",
720                 },
721                 gauge={
722                     "axis": {
723                         "range": [0, max_score],
724                         "tickwidth": 0,
725                         "tickfont": {"size": 11, "color": COLORS["muted"]},
726                     },
727                     "bar": {"color": color_map[risk], "thickness": 0.35},
728                     "bgcolor": "#EAF0F7",
729                     "borderwidth": 0,
730                     "steps": [
731                         {"range": [0, max_score], "color": "#EAF0F7"}
732                     ],
733                 },
734             )
735         )
736
737     fig.update_layout(
738         height=330,
739         paper_bgcolor="white",
740         plot_bgcolor="white",

```

```

741     margin=dict(l=35, r=35, t=60, b=35),
742     font=dict(family="Arial", size=14, color=COLORS["text"]),
743     title=dict(
744         text=title,
745         font=dict(size=18, color=COLORS["navy"]),
746         x=0.5,
747         xanchor="center",
748     ),
749     annotations=[
750         dict(
751             text="Skor Diet (0-10). Semakin tinggi skor, semakin baik kualitas diet.",
752             x=0.5,
753             y=0.02,
754             xref="paper",
755             yref="paper",
756             showarrow=False,
757             font=dict(size=14, color=COLORS["muted"]),
758         )
759     ],
760 )
761
762 return fig
763
764
765 def make_alcohol_dot_chart(data, column, title, x_title, color_map):
766     temp = data[["risk_category", column]].copy()
767     temp = temp.dropna(subset=[column])
768
769     temp["risk_category"] = pd.Categorical(
770         temp["risk_category"],
771         categories=RISK_ORDER,
772         ordered=True
773     )
774     temp = temp.sort_values("risk_category")
775
776     if temp.empty:
777         fig = go.Figure()
778         fig.update_layout(
779             height=330,
780             paper_bgcolor="white",
781             plot_bgcolor="white",
782             title=dict(
783                 text=title,
784                 font=dict(size=18, color=COLORS["navy"]),
785                 x=0.5,
786                 xanchor="center",
787             ),
788         )
789         return fig
790
791     available_risks = [
792         risk for risk in RISK_ORDER
793         if risk in temp["risk_category"].astype(str).tolist()
794     ]
795
796     fig = go.Figure()
797
798     max_unit = max(6, int(np.ceil(temp[column].max())) + 1)
799     x_positions = list(range(max_unit + 1))
800
801     for risk in available_risks:
802         value = float(temp.loc[temp["risk_category"].astype(str) == risk, column].iloc[0])

```

```

803     full_units = int(np.floor(value))
804     color = color_map[risk]
805
806     marker_colors = [
807         color if x <= full_units and x > 0 else "#EAF0F7"
808         for x in x_positions
809     ]
810
811     fig.add_trace(
812         go.Scatter(
813             x=x_positions,
814             y=[risk] * len(x_positions),
815             mode="markers",
816             marker=dict(
817                 size=18,
818                 symbol="square",
819                 color=marker_colors,
820                 line=dict(color="white", width=1),
821             ),
822             showlegend=False,
823             hoverinfo="skip",
824         )
825     )
826
827     fig.add_trace(
828         go.Scatter(
829             x=[value],
830             y=[risk],
831             mode="text",
832             text=[f"{value:.1f}"],
833             textposition="middle right",
834             textfont=dict(size=18, color=color, family="Arial"),
835             showlegend=False,
836             hoverinfo="skip",
837         )
838     )
839
840
841     fig.update_layout(
842         height=330,
843         paper_bgcolor="white",
844         plot_bgcolor="white",
845         margin=dict(l=75, r=85, t=55, b=50),
846         font=dict(family="Arial", size=14, color=COLORS["text"]),
847         title=dict(
848             text=title,
849             font=dict(size=18, color=COLORS["navy"]),
850             x=0.5,
851             xanchor="center",
852         ),
853         xaxis_title=x_title,
854         yaxis_title="",
855         xaxis=dict(range=[-0.5, max_unit + 1]),
856     )
857
858     fig.update_xaxes(
859         showgrid=True,
860         gridcolor="#EEF3F8",
861         linecolor="#D5E1EE",
862         dtick=1,
863         tickfont=dict(size=13, color=COLORS["text"]),
864         title_font=dict(size=14, color=COLORS["text"]),

```

```

865 )
866
867 fig.update_yaxes(
868     categoryorder="array",
869     categoryarray=available_risks,
870     showgrid=False,
871     linecolor="#D5E1EE",
872     tickfont=dict(size=13, color=COLORS["text"]),
873 )
874
875 return fig
876
877
878 def make_scatter_matrix_dashboard(data, numeric_features, target_col):
879     n_cols = 3
880     n_rows = int(np.ceil(len(numeric_features) / n_cols))
881
882     subplot_titles = [
883         f"Risk Score vs {feature}"
884         for feature in numeric_features
885     ]
886
887     fig = make_subplots(
888         rows=n_rows,
889         cols=n_cols,
890         subplot_titles=subplot_titles,
891         horizontal_spacing=0.08,
892         vertical_spacing=0.12,
893     )
894
895     for i, feature in enumerate(numeric_features):
896         row = i // n_cols + 1
897         col = i % n_cols + 1
898
899         temp = data[[feature, target_col]].dropna()
900
901         fig.add_trace(
902             go.Scatter(
903                 x=temp[feature],
904                 y=temp[target_col],
905                 mode="markers",
906                 marker=dict(
907                     size=5,
908                     color="rgba(90, 88, 170, 0.35)",
909                     line=dict(width=0),
910                 ),
911                 name=feature,
912                 showlegend=False,
913             ),
914             row=row,
915             col=col,
916         )
917
918         if len(temp) >= 2 and temp[feature].nunique() > 1:
919             x = temp[feature].values
920             y = temp[target_col].values
921
922             coef = np.polyfit(x, y, 1)
923             poly_fn = np.poly1d(coef)
924
925             x_line = np.linspace(np.nanmin(x), np.nanmax(x), 100)
926             y_line = poly_fn(x_line)

```

```

927         fig.add_trace(
928             go.Scatter(
929                 x=x_line,
930                 y=y_line,
931                 mode="lines",
932                 line=dict(color="#F97316", width=2),
933                 name=f"Trend {feature}",
934                 showlegend=False,
935             ),
936             row=row,
937             col=col,
938         )
939
940
941     fig.update_xaxes(title_text=feature, row=row, col=col)
942     fig.update_yaxes(title_text="Heart Disease Risk Score", row=row, col=col)
943
944     fig.update_layout(
945         height=360 * n_rows,
946         title=dict(
947             text="Relationship Between Numeric Features and Heart Disease Risk Score",
948             x=0.5,
949             xanchor="center",
950             font=dict(size=22, color=COLORS["navy"]),
951         ),
952         paper_bgcolor="white",
953         plot_bgcolor="white",
954         font=dict(family="Arial", size=11, color=COLORS["text"]),
955         margin=dict(l=40, r=40, t=80, b=40),
956     )
957
958     fig.update_xaxes(
959         showgrid=False,
960         linecolor="#D5E1EE",
961         tickfont=dict(size=10, color=COLORS["text"]),
962         title_font=dict(size=11, color=COLORS["text"]),
963     )
964
965     fig.update_yaxes(
966         gridcolor="#EEF3F8",
967         linecolor="#D5E1EE",
968         tickfont=dict(size=10, color=COLORS["text"]),
969         title_font=dict(size=11, color=COLORS["text"]),
970     )
971
972     return fig
973
974
975 # =====
976 # HEADER
977 # =====
978 st.markdown(
979     """
980     <div class="header-box">
981         <div class="header-title"> Dashboard Risiko Penyakit Jantung</div>
982         <div class="header-subtitle">
983             Analisis risiko kardiovaskular berdasarkan faktor klinis dan gaya hidup pasien
984         </div>
985     </div>
986     """,
987     unsafe_allow_html=True,
988 )

```

```

989 # =====
990 # LEFT MENU NAVIGATION
991 # =====
992 menu_col, content_col = st.columns([1.2, 5])
993
994
995 with menu_col:
996     st.markdown(
997         """
998         <div style="
999             background:white;
1000             border:1px solid #DCE6F2;
1001             border-radius:18px;
1002             padding:18px 16px;
1003             box-shadow:0 5px 16px rgba(15,23,42,0.05);
1004             margin-bottom:18px;
1005         ">
1006         <div style="
1007             font-size:18px;
1008             font-weight:900;
1009             color:#0B1F5B;
1010             margin-bottom:12px;
1011         ">
1012             Menu Dashboard
1013         </div>
1014     </div>
1015     """,
1016     unsafe_allow_html=True
1017 )
1018
1019 page = st.radio(
1020     label="",
1021     options=[
1022         "Dashboard",
1023         "Relationship Features",
1024         "Prioritas Edukasi"
1025     ],
1026     label_visibility="collapsed"
1027 )
1028
1029
1030 with content_col:
1031
1032     # =====
1033     # FILTERS
1034     # =====
1035     st.markdown('<div class="filter-title">Filter Data</div>', unsafe_allow_html=True)
1036
1037     f1, f2, f3, f4 = st.columns(4)
1038
1039     with f1:
1040         selected_risk = st.selectbox(
1041             "Kategori Risiko",
1042             ["Semua"] + [x for x in RISK_ORDER if x in
df["risk_category"].dropna().astype(str).unique().tolist()],
1043         )
1044
1045     with f2:
1046         selected_family = st.selectbox(
1047             "Riwayat Keluarga",
1048             ["Semua"] + [x for x in FAMILY_ORDER if x in
df["family_history_heart_disease"].dropna().astype(str).unique().tolist()],

```

```

1049 )
1050
1051 with f3:
1052     selected_age = st.selectbox(
1053         "Age",
1054         ["Semua"] + [x for x in AGE_ORDER if x in
df["age_group"].dropna().astype(str).unique().tolist()],
1055     )
1056
1057
1058 with f4:
1059     bmi_min = float(df["bmi"].min())
1060     bmi_max = float(df["bmi"].max())
1061
1062     st.markdown(
1063         '<div style="color:#0B1F5B; font-size:14px; font-weight:400; margin-bottom:6px;">BMI</div>',
1064         unsafe_allow_html=True
1065     )
1066
1067     selected_bmi_range = st.slider(
1068         "BMI",
1069         min_value=round(bmi_min, 1),
1070         max_value=round(bmi_max, 1),
1071         value=(round(bmi_min, 1), round(bmi_max, 1)),
1072         step=0.1,
1073         format="%.1f",
1074         label_visibility="collapsed"
1075     )
1076
1077 # =====
1078 # APPLY FILTER
1079 # =====
1080 filtered = df.copy()
1081
1082 if selected_risk != "Semua":
1083     filtered = filtered[filtered["risk_category"].astype(str) == selected_risk]
1084
1085 if selected_family != "Semua":
1086     filtered = filtered[filtered["family_history_heart_disease"].astype(str) == selected_family]
1087
1088 if selected_age != "Semua":
1089     filtered = filtered[filtered["age_group"].astype(str) == selected_age]
1090
1091 filtered = filtered[
1092     filtered["bmi"].between(
1093         selected_bmi_range[0],
1094         selected_bmi_range[1]
1095     )
1096 ]
1097
1098 if filtered.empty:
1099     st.warning("Tidak ada data yang sesuai dengan filter yang dipilih.")
1100     st.stop()
1101
1102
1103 # =====
1104 # KPI
1105 # =====
1106 st.divider()
1107
1108 total_patients = len(filtered)
1109 avg_cholesterol = safe_mean(filtered, "cholesterol_mg_dl")

```



```

1110 avg_steps = safe_mean(filtered, "daily_steps")
1111 avg_heart_risk = safe_mean(filtered, "heart_disease_risk_score")
1112 avg_activity = safe_mean(filtered, "physical_activity_hours_per_week")
1113
1114 k1, k2, k3, k4, k5 = st.columns(5)
1115
1116 with k1:
1117     kpi_card("Jumlah Pasien", f"{total_patients:,.0f}", "pasien", COLORS["blue"])
1118
1119 with k2:
1120     kpi_card("Rata-rata Kolesterol", f"{avg_cholesterol:,.0f}", "mg/dL", COLORS["green"])
1121
1122 with k3:
1123     kpi_card("Rata-rata Langkah Harian", f"{avg_steps:,.0f}", "langkah/hari", COLORS["orange"])
1124
1125 with k4:
1126     kpi_card("Rata-rata Heart Risk", f"{avg_heart_risk:,.1f}", "score", COLORS["red"])
1127
1128 with k5:
1129     kpi_card("Rata-rata Aktivitas Seminggu", f"{avg_activity:,.1f}", "jam/minggu", COLORS["purple"])
1130
1131
1132 st.info(
1133     f"Data ditampilkan berdasarkan filter aktif: "
1134     f"Kategori Risiko = {selected_risk}, "
1135     f"Riwayat Keluarga = {selected_family}, "
1136     f"Age = {selected_age}, "
1137     f"BMI = {selected_bmi_range[0]:.1f} - {selected_bmi_range[1]:.1f}."
1138 )
1139
1140 st.divider()
1141
1142
1143 # =====
1144 # CHART DATA
1145 # =====
1146 risk_count = (
1147     filtered["risk_category"]
1148     .value_counts()
1149     .reindex(RISK_ORDER)
1150     .fillna(0)
1151     .reset_index()
1152 )
1153 risk_count.columns = ["risk_category", "count"]
1154
1155
1156 age_risk_score = (
1157     filtered
1158     .groupby("age_group", observed=False)["heart_disease_risk_score"]
1159     .mean()
1160     .reindex(AGE_ORDER)
1161     .fillna(0)
1162     .reset_index()
1163 )
1164 age_risk_score.columns = ["age_group", "avg_risk_score"]
1165
1166
1167 clinical_avg = (
1168     filtered.groupby("risk_category", observed=False)[
1169         ["systolic_bp", "diastolic_bp", "cholesterol_mg_dl", "heart_disease_risk_score"]
1170     ]
1171     .mean()

```

```

1172         .reindex(RISK_ORDER)
1173         .reset_index()
1174     )
1175
1176     clinical_long = clinical_avg.melt(
1177         id_vars="risk_category",
1178         var_name="indikator",
1179         value_name="rata_rata",
1180     )
1181
1182     clinical_long["indikator"] = clinical_long["indikator"].map(
1183         {
1184             "systolic_bp": "Sistolik",
1185             "diastolic_bp": "Diastolik",
1186             "cholesterol_mg_dl": "Kolesterol",
1187             "heart_disease_risk_score": "Heart Risk",
1188         }
1189     )
1190
1191
1192     lifestyle_avg = (
1193         filtered.groupby("risk_category", observed=False)[
1194             [
1195                 "physical_activity_hours_per_week",
1196                 "daily_steps",
1197                 "sleep_hours",
1198                 "diet_quality_score",
1199                 "alcohol_units_per_week",
1200             ]
1201         ]
1202         .mean()
1203         .reindex(RISK_ORDER)
1204         .reset_index()
1205     )
1206
1207
1208     family_prop = pd.crosstab(
1209         filtered["risk_category"],
1210         filtered["family_history_heart_disease"],
1211         normalize="index",
1212     ).reindex(index=RISK_ORDER, columns=FAMILY_ORDER).fillna(0).mul(100).reset_index()
1213
1214     family_long = family_prop.melt(
1215         id_vars="risk_category",
1216         var_name="family_history",
1217         value_name="persentase",
1218     )
1219
1220
1221     feature_cols = [
1222         "age",
1223         "bmi",
1224         "cholesterol_mg_dl",
1225         "systolic_bp",
1226         "diastolic_bp",
1227         "physical_activity_hours_per_week",
1228         "diet_quality_score",
1229         "daily_steps",
1230     ]
1231
1232     feature_labels = {
1233         "age": "Age",

```

```

1234     "bmi": "BMI",
1235     "cholesterol_mg_dl": "Kolesterol",
1236     "systolic_bp": "Sistolik",
1237     "diastolic_bp": "Diastolik",
1238     "physical_activity_hours_per_week": "Aktivitas Fisik",
1239     "diet_quality_score": "Diet Score",
1240     "daily_steps": "Langkah Harian",
1241 }
1242
1243 low_mean = df[df["risk_category"] == "Low"][feature_cols].mean()
1244 high_mean = df[df["risk_category"] == "High"][feature_cols].mean()
1245
1246 feature_delta = ((high_mean - low_mean) / low_mean * 100).replace([np.inf, -np.inf], np.nan).dropna()
1247 feature_delta = feature_delta.rename("delta_pct").reset_index().rename(columns={"index": "feature"})
1248 feature_delta["feature_label"] = feature_delta["feature"].map(feature_labels)
1249 feature_delta = feature_delta.sort_values("delta_pct", ascending=True)
1250 feature_delta["direction"] = np.where(feature_delta["delta_pct"] >= 0, "Naik", "Turun")
1251
1252
1253 # =====
1254 # FIGURES
1255 # =====
1256 fig_risk = px.pie(
1257     risk_count,
1258     names="risk_category",
1259     values="count",
1260     hole=0.55,
1261     color="risk_category",
1262     color_discrete_map=RISK_COLORS,
1263 )
1264
1265 fig_risk.update_traces(
1266     textposition="inside",
1267     texttemplate="%{percent:.1%}",
1268     marker=dict(line=dict(color="white", width=2)),
1269     domain=dict(x=[0.00, 0.72], y=[0.00, 1.00]),
1270 )
1271
1272 fig_risk.update_layout(
1273     height=420,
1274     paper_bgcolor="white",
1275     plot_bgcolor="white",
1276     margin=dict(l=10, r=10, t=25, b=10),
1277     font=dict(size=12, color=COLORS["text"]),
1278     legend=dict(
1279         orientation="v",
1280         x=0.78,
1281         y=0.86,
1282         font=dict(size=12, color=COLORS["text"]),
1283         title=dict(text="Kategori Risiko"),
1284     ),
1285     annotations=[
1286         dict(
1287             text=f"<b>Total</b><br>{risk_count['count'].sum():.0f}",
1288             x=0.36,
1289             y=0.50,
1290             showarrow=False,
1291             font=dict(size=16, color=COLORS["navy"]),
1292             xanchor="center",
1293             yanchor="middle",
1294             align="center",
1295         )

```

```

1296     ],
1297 )
1298
1299
1300 fig_age = px.box(
1301     filtered,
1302     x="age_group",
1303     y="heart_disease_risk_score",
1304     color="age_group",
1305     category_orders={"age_group": AGE_ORDER},
1306     points="outliers",
1307 )
1308
1309 fig_age.update_layout(
1310     xaxis_title="Kelompok Usia",
1311     yaxis_title="Heart Disease Risk Score",
1312     showlegend=False,
1313 )
1314
1315 fig_age.update_traces(
1316     marker=dict(size=4),
1317     line=dict(width=1.5),
1318 )
1319
1320 fig_age = style_fig(
1321     fig_age,
1322     height=420,
1323     showlegend=False,
1324     margin=dict(l=55, r=30, t=55, b=60),
1325 )
1326
1327 fig_clinical = px.bar(
1328     clinical_long,
1329     x="risk_category",
1330     y="rata_rata",
1331     color="indikator",
1332     barmode="group",
1333     text=clinical_long["rata_rata"].round(1),
1334     color_discrete_map={
1335         "Sistolik": COLORS["blue"],
1336         "Diastolik": COLORS["teal"],
1337         "Kolesterol": COLORS["purple"],
1338         "Heart Risk": COLORS["orange"],
1339     },
1340 )
1341
1342 fig_clinical.update_traces(textposition="outside")
1343
1344 fig_clinical.update_layout(
1345     xaxis_title="Kategori Risiko",
1346     yaxis_title="Rata-rata",
1347     legend_title_text="Indikator Klinis",
1348 )
1349
1350 fig_clinical = style_fig(
1351     fig_clinical,
1352     height=420,
1353     showlegend=True,
1354     margin=dict(l=55, r=30, t=75, b=60),
1355 )
1356
1357 fig_family = px.bar(

```

```

1358     family_long,
1359     x="persentase",
1360     y="risk_category",
1361     color="family_history",
1362     orientation="h",
1363     text=family_long["persentase"].round(1).astype(str) + "%",
1364     color_discrete_map={
1365         "No": COLORS["blue"],
1366         "Yes": COLORS["red"],
1367     },
1368     category_orders={
1369         "risk_category": RISK_ORDER,
1370         "family_history": FAMILY_ORDER,
1371     },
1372 )
1373
1374 fig_family.update_traces(
1375     textposition="inside",
1376     insidetextfont=dict(color="white", size=12),
1377 )
1378
1379 fig_family.update_layout(
1380     barmode="stack",
1381     xaxis=dict(range=[0, 100], ticksuffix="%"),
1382     xaxis_title="Persentase (%)",
1383     yaxis_title="Kategori Risiko",
1384     legend_title_text="Riwayat Keluarga",
1385 )
1386
1387 fig_family = style_fig(
1388     fig_family,
1389     height=420,
1390     showlegend=True,
1391     margin=dict(l=90, r=30, t=75, b=60),
1392 )
1393
1394
1395 fig_feature = px.bar(
1396     feature_delta,
1397     x="delta_pct",
1398     y="feature_label",
1399     orientation="h",
1400     text=feature_delta["delta_pct"].map(lambda x: f"{x:+.1f}%"),
1401     color="direction",
1402     color_discrete_map={
1403         "Naik": COLORS["red"],
1404         "Turun": COLORS["blue"],
1405     },
1406 )
1407
1408 fig_feature.update_traces(textposition="outside", cliponaxis=False)
1409 fig_feature.add_vline(x=0, line_width=1, line_color="#94A3B8")
1410
1411 max_abs = max(60, np.abs(feature_delta["delta_pct"]).max() + 10)
1412
1413 fig_feature.update_layout(
1414     xaxis=dict(range=[-max_abs, max_abs], ticksuffix="%"),
1415     xaxis_title="Perubahan Relatif High Risk vs Low Risk",
1416     yaxis_title="",
1417 )
1418
1419 fig_feature = style_fig(

```

```

1420     fig_feature,
1421     height=420,
1422     showlegend=False,
1423     margin=dict(l=140, r=50, t=40, b=60),
1424 )
1425
1426
1427 # Lifestyle separated figures
1428 fig_activity = make_activity_progress_chart(
1429     lifestyle_avg,
1430     "physical_activity_hours_per_week",
1431     "Rata-rata Aktivitas Fisik",
1432     "Jam per Minggu",
1433     RISK_COLORS,
1434     max_value=6,
1435 )
1436
1437 fig_steps = make_small_lifestyle_chart(
1438     lifestyle_avg,
1439     "daily_steps",
1440     "Rata-rata Langkah Harian",
1441     "Langkah per Hari",
1442     RISK_COLORS,
1443 )
1444
1445 fig_sleep = make_sleep_lollipop_chart(
1446     lifestyle_avg,
1447     "sleep_hours",
1448     "Rata-rata Durasi Tidur",
1449     "Jam per Hari",
1450     RISK_COLORS,
1451 )
1452
1453 fig_diet = make_diet_gauge_chart(
1454     lifestyle_avg,
1455     "diet_quality_score",
1456     "Skor Kualitas Diet",
1457     RISK_COLORS,
1458     max_score=10,
1459 )
1460
1461 fig_alcohol = make_alcohol_dot_chart(
1462     lifestyle_avg,
1463     "alcohol_units_per_week",
1464     "Konsumsi Alkohol per Kategori Risiko",
1465     "Unit per Minggu",
1466     RISK_COLORS,
1467 )
1468
1469
1470 # Scatterplot relationship figures
1471 scatter_features = [
1472     "age",
1473     "bmi",
1474     "systolic_bp",
1475     "diastolic_bp",
1476     "cholesterol_mg_dl",
1477     "resting_heart_rate",
1478     "daily_steps",
1479     "physical_activity_hours_per_week",
1480     "sleep_hours",
1481     "alcohol_units_per_week",

```

```

1482 ]
1483
1484 scatter_features = [
1485     col for col in scatter_features
1486     if col in filtered.columns and "heart_disease_risk_score" in filtered.columns
1487 ]
1488
1489 fig_scatter = make_scatter_matrix_dashboard(
1490     filtered,
1491     scatter_features,
1492     "heart_disease_risk_score",
1493 )
1494
1495
1496 # =====
1497 # PAGE 1: DASHBOARD
1498 # =====
1499 if page == "Dashboard":
1500
1501     row1_col1, row1_col2 = st.columns(2)
1502
1503     with row1_col1:
1504         with st.container(border=True):
1505             chart_title("A", "Persentase Pasien per Kategori Risiko")
1506             st.plotly_chart(fig_risk, use_container_width=True, config={"displayModeBar": False})
1507
1508     with row1_col2:
1509         with st.container(border=True):
1510             chart_title("B", "Distribusi Heart Risk Score berdasarkan Kelompok Usia")
1511             st.plotly_chart(fig_age, use_container_width=True, config={"displayModeBar": False})
1512
1513
1514     row2_col1, row2_col2 = st.columns(2)
1515
1516     with row2_col1:
1517         with st.container(border=True):
1518             chart_title("C", "Rata-rata Tekanan Darah, Kolesterol, dan Heart Risk")
1519             st.plotly_chart(fig_clinical, use_container_width=True, config={"displayModeBar": False})
1520
1521     with row2_col2:
1522         with st.container(border=True):
1523             chart_title("D", "Riwayat Keluarga Penyakit Jantung")
1524             st.plotly_chart(fig_family, use_container_width=True, config={"displayModeBar": False})
1525
1526
1527     with st.container(border=True):
1528         chart_title("E", "Fitur yang Menonjol pada Pasien High Risk")
1529         st.plotly_chart(fig_feature, use_container_width=True, config={"displayModeBar": False})
1530
1531
1532 # =====
1533 # POLA GAYA HIDUP DIPISAH
1534 # =====
1535 with st.container(border=True):
1536     chart_title("F", "Pola Gaya Hidup per Kategori Risiko")
1537
1538     g1, g2, g3 = st.columns(3)
1539
1540     with g1:
1541         st.plotly_chart(fig_activity, use_container_width=True, config={"displayModeBar": False})
1542
1543     with g2:

```

```

1544         st.plotly_chart(fig_steps, use_container_width=True, config={"displayModeBar": False})
1545
1546     with g3:
1547         st.plotly_chart(fig_sleep, use_container_width=True, config={"displayModeBar": False})
1548
1549
1550     g4, g5 = st.columns(2)
1551
1552     with g4:
1553         st.plotly_chart(fig_diet, use_container_width=True, config={"displayModeBar": False})
1554
1555     with g5:
1556         st.plotly_chart(fig_alcohol, use_container_width=True, config={"displayModeBar": False})
1557
1558     # =====
1559     # PAGE 2: RELATIONSHIP FEATURES
1560     # =====
1561     elif page == "Relationship Features":
1562
1563         st.markdown(
1564             '<div class="section-title">Relationship Between Numeric Features and Heart Disease Risk
Score</div>',
1565             unsafe_allow_html=True,
1566         )
1567
1568         with st.container(border=True):
1569             st.plotly_chart(fig_scatter, use_container_width=True, config={"displayModeBar": False})
1570
1571
1572     # =====
1573     # PAGE 3: PRIORITAS EDUKASI
1574     # =====
1575     elif page == "Prioritas Edukasi":
1576
1577         st.markdown(
1578             '<div class="section-title">Prioritas Edukasi Pencegahan</div>',
1579             unsafe_allow_html=True
1580         )
1581
1582         with st.container(border=True):
1583             st.markdown(
1584                 """
1585                 <div class="priority-item">
1586                     <div class="priority-title">1. Prioritaskan pasien kategori High Risk.</div>
1587                     <div class="priority-sub">Kelompok ini paling membutuhkan monitoring dan edukasi
pencegahan.</div>
1588                 </div>
1589
1590                 <div class="priority-item">
1591                     <div class="priority-title">2. Kontrol tekanan darah dan kolesterol secara rutin.</div>
1592                     <div class="priority-sub">Dua indikator ini dominan pada pasien berisiko tinggi.</div>
1593                 </div>
1594
1595                 <div class="priority-item">
1596                     <div class="priority-title">3. Dorong aktivitas fisik dan peningkatan langkah harian.
</div>
1597                     <div class="priority-sub">Gaya hidup kurang aktif berkaitan dengan risiko yang lebih
tinggi.</div>
1598                 </div>
1599
1600                 <div class="priority-item">
1601                     <div class="priority-title">4. Perbaiki kualitas diet dan tidur pasien.</div>

```



```
1602         <div class="priority-sub">Pola hidup sehat membantu menurunkan faktor risiko
kardiovaskular.</div>
1603     </div>
1604
1605     <div class="priority-item">
1606         <div class="priority-title">5. Fokus pada pasien dengan riwayat keluarga penyakit
jantung.</div>
1607         <div class="priority-sub">Kelompok ini perlu skrining dan edukasi lebih dini.</div>
1608     </div>
1609     "",
1610     unsafe_allow_html=True,
1611 )
1612
```

```
import pandas as pd
import numpy as np

# Library statistik dan visualisasi
from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns

# Load data
df_ab = pd.read_excel('/content/Data Website.xlsx')

# Tampilkan 5 data pertama
df_ab.head()
```

	usia2	BMI	tekanan darah	kolestrol	denyut jantung	langkah harian	durasi tidur	riwayat penyakit	kualiatas diet	aktivitas fisik	tingkat stress	alkohol	skor risiko
0	21	22.49	120/80	200	72	6000	6	tidak	5	3.5	6	0	0.05
1	21	22.49	110/80	200	65	6000	9	iya	9	3.5	6	0	0.12
2	21	31.25	102/90	200	89	4000	3	iya	3	1.0	6	16	0.22
3	30	13.15	103/90	109	65	4000	6	tidak	9	2.0	6	11	0.04

Langkah berikutnya: [Buat kode dengan df_ab](#) [New interactive sheet](#)

```
# Cek informasi dataset
df_ab.info()
```

```
# Cek nama kolom
df_ab.columns
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   usia2                  30 non-null     int64
1   BMI                    30 non-null     float64
2   tekanan darah          30 non-null     object
3   kolesterol              30 non-null     int64
4   denyut jantung          30 non-null     int64
5   langkah harian          30 non-null     int64
6   durasi tidur            30 non-null     int64
7   riwayat penyakit        30 non-null     object
8   kualiatas diet          30 non-null     int64
9   aktivitas fisik          30 non-null     float64
10  tingkat stress          30 non-null     int64
11  alkohol                 30 non-null     int64
12  skor risiko             30 non-null     float64
dtypes: float64(3), int64(8), object(2)
memory usage: 3.2+ KB
Index(['usia2', 'BMI', 'tekanan darah', 'kolesterol', 'denyut jantung',
      'langkah harian', 'durasi tidur', 'riwayat penyakit', 'kualiatas diet',
      'aktivitas fisik', 'tingkat stress', 'alkohol', 'skor risiko'],
      dtype='object')
```

✓ A/B Testing 1: Perbandingan Skor Risiko Berdasarkan Riwayat Penyakit

A/B Testing dilakukan untuk membandingkan skor risiko antara dua kelompok input berdasarkan variabel `riwayat penyakit`.

- Kelompok A: `riwayat penyakit = tidak`
- Kelompok B: `riwayat penyakit = iya`
- Output yang dibandingkan: `skor risiko`

```
# Rapikan nama kolom
df_ab.columns = df_ab.columns.str.strip().str.lower()

# Rapikan isi kolom riwayat penyakit
df_ab['riwayat penyakit'] = (
    df_ab['riwayat penyakit']
    .astype(str)
    .str.lower()
    .str.strip()
)

# Buat kelompok A dan B
group_A = df_ab[df_ab['riwayat penyakit'] == 'tidak']['skor risiko']
group_B = df_ab[df_ab['riwayat penyakit'] == 'iya']['skor risiko']

# A/B Testing menggunakan Mann-Whitney U Test
alpha = 0.05
```

```
test_stat, p_value = stats.mannwhitneyu(
    group_A,
    group_B,
    alternative='two-sided'
)

print("Metode uji: Mann-Whitney U Test")
print("Test Statistic:", test_stat)
print("P-value:", p_value)

if p_value < alpha:
    print("Kesimpulan: Terdapat perbedaan signifikan skor risiko antara Kelompok A dan Kelompok B.")
else:
    print("Kesimpulan: Tidak terdapat perbedaan signifikan skor risiko antara Kelompok A dan Kelompok B.")
```

Metode uji: Mann-Whitney U Test
Test Statistic: 47.5
P-value: 0.014989066277655365
Kesimpulan: Terdapat perbedaan signifikan skor risiko antara Kelompok A dan Kelompok B.

▼ **A/B Testing: Perbandingan Skor Risiko Berdasarkan Kolesterol**

A/B Testing dilakukan untuk membandingkan skor risiko antara dua kelompok input berdasarkan variabel `kolesterol`.

- Kelompok A: `kolesterol < 200` atau kolesterol normal
- Kelompok B: `kolesterol >= 200` atau kolesterol tinggi
- Output yang dibandingkan: `skor risiko`

Tujuan pengujian ini adalah untuk mengetahui apakah terdapat perbedaan skor risiko antara kelompok dengan kolesterol normal dan kelompok dengan kolesterol tinggi.

```
# Rapiakan nama kolom
df_ab.columns = df_ab.columns.str.strip().str.lower()

# Buat kelompok A dan B berdasarkan kolesterol
group_A = df_ab[df_ab['kolesterol'] < 200]['skor risiko']
group_B = df_ab[df_ab['kolesterol'] >= 200]['skor risiko']

# A/B Testing menggunakan Mann-Whitney U Test
alpha = 0.05

test_stat, p_value = stats.mannwhitneyu(
    group_A,
    group_B,
    alternative='two-sided'
)

print("A/B Testing: Kolesterol terhadap Skor Risiko")
print("Kelompok A: Kolesterol normal (< 200)")
print("Kelompok B: Kolesterol tinggi (>= 200)")
print("Jumlah data Kelompok A:", len(group_A))
print("Jumlah data Kelompok B:", len(group_B))
print("Rata-rata skor risiko Kelompok A:", round(group_A.mean(), 4))
print("Rata-rata skor risiko Kelompok B:", round(group_B.mean(), 4))
print("Selisih rata-rata B - A:", round(group_B.mean() - group_A.mean(), 4))
print("Metode uji: Mann-Whitney U Test")
print("Test Statistic:", test_stat)
print("P-value:", round(p_value, 4))

if p_value < alpha:
    print("Kesimpulan: Terdapat perbedaan signifikan skor risiko antara kelompok kolesterol normal dan kolesterol tinggi.")
else:
    print("Kesimpulan: Tidak terdapat perbedaan signifikan skor risiko antara kelompok kolesterol normal dan kolesterol tinggi.")
```

A/B Testing: Kolesterol terhadap Skor Risiko
Kelompok A: Kolesterol normal (< 200)
Kelompok B: Kolesterol tinggi (>= 200)
Jumlah data Kelompok A: 11
Jumlah data Kelompok B: 19
Rata-rata skor risiko Kelompok A: 0.1536
Rata-rata skor risiko Kelompok B: 0.2663
Selisih rata-rata B - A: 0.1127
Metode uji: Mann-Whitney U Test
Test Statistic: 55.5
P-value: 0.0368
Kesimpulan: Terdapat perbedaan signifikan skor risiko antara kelompok kolesterol normal dan kolesterol tinggi.

Mulai coding atau buat kode dengan AI.

